

响应式圣经

Reactive Programming Bible

尼恩 编著

实现响应式编程自由

疯狂创客圈Java高并发社群（50+群）出品

CSDN @40岁资深老架构师尼恩

前言

全链路异步化改造的基础是响应式编程

随着业务的发展，微服务应用的流量越来越大，使用到的资源也越来越多。

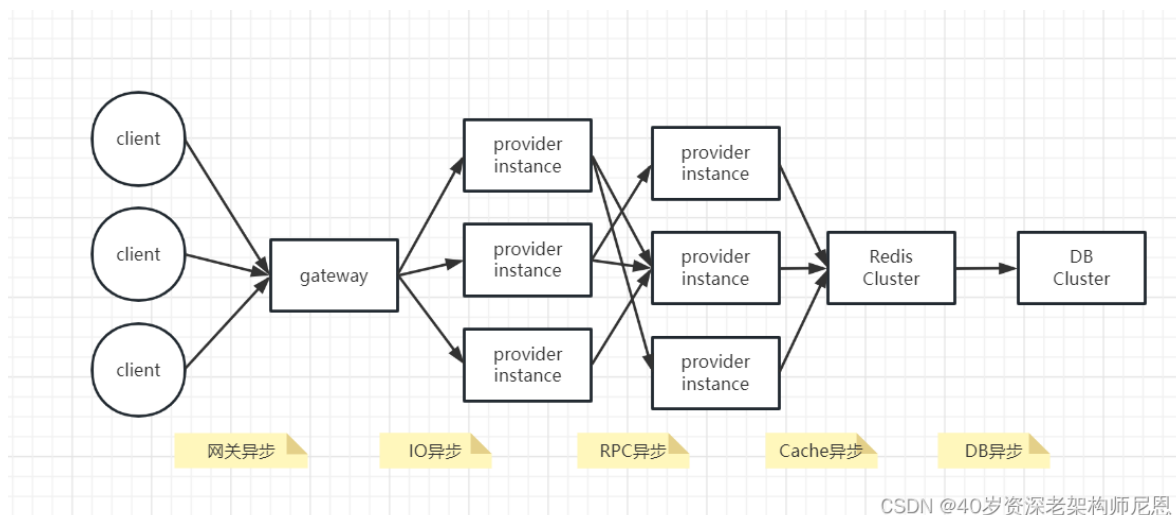
在微服务架构下，大量的应用都是 SpringCloud 分布式架构，这种架构总体上是**全链路同步模式**。

全链路同步模式不仅造成了资源的极大浪费，并且在流量发生激增波动的时候，受制于系统资源而无法快速的扩容。

全球后疫情时代，降本增效是大背景。如何降本增效？

可以通过技术升级，**全链路同步模式**，升级为 **全链路异步模式**。

先回顾一下全链路同步模式架构图



全链路同步模式，如何升级为 **全链路异步模式**，就是一个一个环节的异步化。

40岁老架构师尼恩，持续深化自己的3高架构知识宇宙，当然首先要去完成一次牛逼的**全链路异步模式微服务实操**，下面是尼恩的实操过程、效果、压测数据(性能足足提升10倍多)。

全链路异步模式改造 具体的内容，请参考尼恩的深度文章：[全链路异步，让你的 SpringCloud 性能优化10倍+](#)

并且，上面的文章，作为尼恩 全链路异步的架构知识，收录在《[尼恩Java面试宝典](#)》V46版的架构专题中

全链路异步化改造，性能提升十倍是大好事，那么，全链路同步模式改造的问题是什么呢？

全链路异步化改造的技术基础，是响应式编程，关键在于：**响应式编程的知识太难**。

古语说：蜀道难难于上青天。

很多小伙伴认为：响应式编程，比蜀道还难？

所以，40岁老架构师 使用20年的编程功力，给大家呈上一篇，学习响应式编程 的超级长文，也是一篇超级、超级详细，超级超级全面，并且不断迭代的文章：

《**响应式圣经：10W字实现响应式编程自由**》

此文，目前为V2版本，新版本是基于V1老版本是尼恩之前写的一篇深受好评的博客文章

[Flux、Mono、Reactor 实战（史上最全）](#)

后续，尼恩会一直不断迭代，为大家拉涅出一本史上最棒的《**响应式圣经**》，帮助大家实现响应式编程自由。

从此：蜀道，不再难。

注：本文以 PDF 持续更新，最新尼恩 架构笔记、面试题 的PDF文件，请从这里获取：[码云](#)

Reactive programming 响应式编程概述

背景知识

为了应对高并发服务器端开发场景，在2009年，微软提出了一个更优雅地实现异步编程的方式——Reactive Programming，我们称之为响应式编程。

随后，Netflix 和LightBend 公司提供了RxJava 和Akka Stream 等技术，使得Java 平台也有了能够实现响应式编程的框架。

在2017年9月28日，Spring 5 正式发布。

Spring 5 发布最大的意义在于，它将响应式编程技术的普及向前推进了一大步。

而同时，作为在背后支持Spring 5 响应式编程的框架Spring Reactor，也进入了里程碑式的3.1.0 版本。

什么是响应式编程

响应式编程是一种面向数据流和变化传播的编程范式。

这意味着可以在编程语言中很方便地表达静态或动态的数据流，而相关的计算模型会自动将变化的值通过数据流进行传播。

响应式编程基于reactor（Reactor 是一个运行在 Java8 之上的响应式框架）的思想，当你做一个带有一定延迟的才能够返回的io操作时，不会阻塞，而是立刻返回一个流，并且订阅这个流，当这个流上产生了返回数据，可以立刻得到通知并调用回调函数处理数据。

电子表格程序就是响应式编程的一个例子。有些单元含有公式，比如可以包含字面值或类似" $B1+C1$ "的公式，这些单元格的值，会依据其他单元格的值的变化而变化。

响应式传播核心特点之一：变化传播。

一个单元格变化之后，会像多米诺骨牌一样，导致直接和间接引用它的其他单元格均发生相应变化。

从API视角，看什么是 Reactive programming 响应式编程

有了 Reactive Streams 这种标准和规范，利用规范可以进行响应式编程。

那再了解下什么是 Reactive programming 响应式编程。

响应式编程是基于异步和事件驱动的非阻塞程序，只是垂直通过在 JVM 内启动少量线程扩展，而不是水平通过集群扩展。

异步调用 +IO Reactor 事件驱动，可以避免将 CPU 浪费在等待网络 IO 和磁盘 IO 上，实现提高资源使用率。

Reactive programming就是一个编程范例，具体项目中如何体现呢？

响应式项目编程实战中，通过基于 Reactive Streams 规范实现的框架Spring Reactor 去实战。

Spring Reactor 一般提供两种响应式 API：

- Mono：实现发布者，并返回 0 或 1 个元素

- `Flux`：实现发布者，并返回 N 个元素

响应式编程->异步非阻塞

上面讲了响应式编程是什么：

响应式编程（reactive programming）是一种基于数据流（data stream）和变化传递（propagation of change）的声明式（declarative）的编程范式

也讲解了数据流/变化传递/声明式是什么意思，但说到响应式编程就离不开**异步非阻塞**。

从Spring官网介绍WebFlux的信息我们就可以发现 `asynchronous, nonblocking` 这样的字样，因为**响应式编程它是异步的**，也可以理解成**变化传递它是异步执行的**。

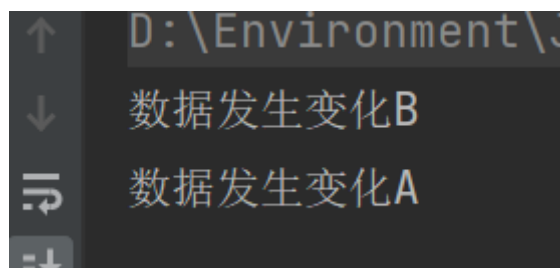
基于Java8观察者模式

Observable类：此类表示可观察对象，或模型视图范例中的“数据”。

它可以被子类实现以表示应用程序想要观察的对象。

```
//想要观察的对象 ObserverDemo
public class ObserverDemo extends Observable {
    public static void main(String[] args) {
        ObserverDemo observerDemo = new ObserverDemo();
        //添加观察者
        observerDemo.addObserver((o, arg) -> {
            System.out.println("数据发生变化A");
        });
        observerDemo.addObserver((o, arg) -> {
            System.out.println("数据发生变化B");
        });
        observerDemo.setChanged(); //将此Observable对象标记为已更改
        observerDemo.notifyObservers(); //如果该对象发生了变化，则通知其所有观察者
    }
}
```

启动程序测试：



创建一个Observable

rxjava中，可以使用Observable.create() 该方法接收一个ObservableOnSubscribe对象

```

observable<Integer> observable = Observable.create(new
observable.OnSubscribe<Integer>() {
    @Override
    public void call(Subscriber<? super Integer> subscriber) {

    }
});

```

来个大点的例子：

```

observable<Integer> observable=Observable.create(new
observable.OnSubscribe<Integer>() {
    @Override
    public void call(Subscriber<? super Integer> subscriber) {
        for(int i=0;i<5;i++){
            subscriber.onNext(i);
        }
        subscriber.onCompleted();
    }
});
//Observable.subscribe(Observer), Observer订阅了Observable
Subscription subscribe = observable.subscribe(new Observer<Integer>() {
    @Override
    public void onCompleted() {
        Log.e(TAG, "完成");
    }

    @Override
    public void onError(Throwable e) {
        Log.e(TAG, "异常");
    }

    @Override
    public void onNext(Integer integer) {
        Log.e(TAG, "接收Obsverable中发射的值: " + integer);
    }
});

```

输出：

```

接收Obsverable中发射的值: 0
接收Obsverable中发射的值: 1
接收Obsverable中发射的值: 2
接收Obsverable中发射的值: 3
接收Obsverable中发射的值: 4

```

从上面的例子可以看出，在Observer订阅了Observable后，

Observer作为OnSubscribe中call方法的参数传入，从而调用了Observer的相关方法

基于 Reactor 实现

Reactor 是一个运行在 Java8 之上满足 Reactice 规范的响应式框架，它提供了一组响应式风格的 API。

Reactor 有两个核心类： `Flux<T>` 和 `Mono<T>`，这两个类都实现 Publisher 接口。

- Flux 类似 Rxjava 的 Observable，它可以触发零到多个事件，并根据实际情况结束处理或触发错误。
- Mono 最多只触发一个事件，所以可以把 Mono 用于在异步任务完成时发出通知。



Mono & Flux

Flux 和 Mono 都是数据流的发布者，使用 Flux 和 Mono 都可以发出三种数据信号：元素值，错误信号，完成信号；错误信号和完成信号都代表终止信号，终止信号用于告诉订阅者数据流结束了，错误信号终止数据流同时把错误信息传递给订阅者。

三种信号的特点：

- 错误信号和完成信号都是终止信号，不能共存
- 如果没有发送任何元素值，而是直接发送错误或者完成信号，表示是空数据流
- 如果没有错误信号，也没有完成信号，表示是无限数据流

引入依赖

```
<dependency>
  <groupId>org.projectreactor</groupId>
  <artifactId>reactor-core</artifactId>
</dependency>
```

just 和 subscribe方法

just(): 创建Flux序列，并声明指定数据流

subscribe(): 订阅Flux序列，只有进行订阅后才回触发数据流，不订阅就什么都不会发生

```
public class TestReactor {
    public static void main(String[] args) {
        //just(): 创建Flux序列，并声明数据流，
        Flux<Integer> integerFlux = Flux.just(1, 2, 3, 4); //整形
        //subscribe(): 订阅Flux序列，只有进行订阅后才回触发数据流，不订阅就什么都不会发生
        integerFlux.subscribe(System.out::println);

        Flux<String> stringFlux = Flux.just("hello", "world"); //字符串
        stringFlux.subscribe(System.out::println);

        //fromArray(), fromIterable() 和 fromStream(): 可以从一个数组、Iterable 对象或
        Stream 对象中创建Flux序列
    }
}
```

```

Integer[] array = {1,2,3,4};
Flux.fromArray(array).subscribe(System.out::println);

List<Integer> integers = Arrays.asList(array);
Flux.fromIterable(integers).subscribe(System.out::println);

Stream<Integer> stream = integers.stream();
Flux.fromStream(stream).subscribe(System.out::println);
    }
}

```

启动测试：

```

09:59:51.01
1
2
3
4
hello
world
1
2
3
4
1
2
3
4
1
2
3
4

```

Reactive Streams（响应式流）的特点

要搞清楚这两个概念，必须说一下响应流规范。

它是响应式编程的基石。他具有以下特点：

- 响应流必须是无阻塞的。
- 响应流必须是一个数据流。
- 它必须可以异步执行。
- 并且它也应该能够处理背压。

- **即时响应性：**

只要有可能，[系统](#)就会及时地做出响应。即时响应是可用性和实用性的基石，而更加重要的是，即时响应意味着可以快速地检测到问题并且有效地对其进行处理。即时响应的系统专注于提供快速而一致的响应时间，确立可靠的反馈上限，以提供一致的服务质量。这种一致的行为转而将简化错误处理、建立最终用户的信任并促使用户与系统作进一步的互动。

- **回弹性：**

系统在出现[失败](#)时依然保持即时响应性。这不仅适用于高可用的、任务关键型系统——任何不具备回弹性的系统都将会在发生失败之后丢失即时响应性。回弹性是通过[复制](#)、[遏制](#)、[隔离](#)以及[委托](#)来实现的。失败的扩散被遏制在了每个[组件](#)内部，与其他组件相互隔离，从而确保系统某部分的失败不会危及整个系统，并能独立恢复。每个组件的恢复都被委托给了另一个（外部的）组件，此外，在必要时可以通过复制来保证高可用性。（因此）组件的客户端不再承担组件失败的处理。

- **弹性：**

系统在不断变化的工作负载之下依然保持即时响应性。反应式系统可以对输入（负载）的速率变化做出反应，比如通过增加或者减少被分配用于服务这些输入（负载）的[资源](#)。这意味着设计上并没有争用点和中央瓶颈，得以进行组件的分片或者复制，并在它们之间分布输入（负载）。通过提供相关的实时性能指标，反应式系统能支持预测式以及反应式的伸缩算法。这些系统可以在常规的硬件以及软件平台上实现成本高效的[弹性](#)。

- **消息驱动：**

反应式系统依赖异步的、消息传递，从而确保了松耦合、隔离、[位置透明](#)的组件之间有着明确边界。

这一边界还提供了将[失败](#)作为消息委托出去的手段。

使用显式的消息传递，可以通过在系统中塑造并监视消息流队列，并在必要时应用[回压](#)，从而实现负载管理、弹性以及流量控制。

使用位置透明的消息传递作为通信的手段，使得跨集群或者在单个主机中使用相同的结构成分和语义来管理失败成为了可能。

[非阻塞](#)的通信使得接收者可以只在活动时才消耗[资源](#)，从而减少系统开销。

Reactive Streams（响应式流）的过程

一般由以下组成：

一般由以下组成：

- `publisher`：发布者，发布元素到订阅者
- `subscriber`：订阅者，消费元素
- `subscription`：订阅，在发布者中，订阅被创建时，将与订阅者共享
- `processor`：处理器，发布者与订阅者之间处理数据，包含了发布者与订阅者的共同体

publisher接口规范

```
public interface Publisher<T> {  
    void subscribe(Subscriber<? super T> var1); //添加订阅者  
}
```

subscriber接口规范

```
public interface Subscriber<T> {  
  
    void onSubscribe(Subscription var1);  
    void onNext(T var1);  
    void onError(Throwable var1);  
  
    void onComplete();  
}
```

subscription接口规范

```
public interface Subscription {  
    void request(long var1);  
    void cancel();  
}
```

processor接口规范

```
public interface Processor<T, R> extends Subscriber<T>, Publisher<R> {  
}
```

Reactor 流框架的组件和使用

- Reactor 框架是 Pivotal 公司（开发 Spring 等技术公司）开发的
- 实现了 Reactive Programming 思想，符合 Reactive Streams 规范（Reactive Streams 是由 Netflix、TypeSafe、Pivotal 等公司发起的）的一项技术
- 侧重于 server 端的响应式编程框架
- Reactor 框架主要有两个主要的模块：reactor-core 和 reactor-ipc。前者主要负责 Reactive Programming 相关的核心 API 的实现，后者负责高性能网络通信的实现，目前是基于 Netty 实现的。

Java 原有的异步编程方式

- `Callback`：异步方法采用一个 callback 作为参数，当结果出来后回调这个 callback，例如 swings 的 `EventListener`
- `Future`：异步方法返回一个 `Future<T>`，此时结果并不是立刻可以拿到，需要处理结束之后才可以使用

Future 局限

- 多个 Future 组合不易
- 调用 `Future#get` 时仍然会阻塞
- 缺乏对多个值以及进一步的出错处理

Reactor 的 Publisher

- `Mono` 实现了 `org.reactivestreams.Publisher` 接口，代表 0 到 1 个元素的响应式序列。
- `Flux` 同样实现了 `org.reactivestreams.Publisher` 接口，代表 0 到 N 个元素的结果。

Publisher/Flux 和 Mono 三大流

由于响应流的特点，我们不能再返回一个简单的 POJO 对象来表示结果了。

必须返回一个类似 Java 中的 `Future` 的概念，在有结果可用时通知消费者进行消费响应。

Reactive Stream 规范中这种被定义为 `Publisher<T>`

Publisher 发射着基础类

`Publisher<T>` 是一个可以提供 0-N 个序列元素的提供者，并根据其订阅者 `Subscriber<? super T>` 的需求推送元素。

一个 `Publisher<T>` 可以支持多个订阅者，并可以根据订阅者的逻辑进行推送序列元素。

下面这个Excel计算就能说明一些 `Publisher<T>` 的特点。

	A	B
1	1	
2	2	
3	3	
4	4	
5	5	
6	6	
7	7	
8	8	
9	9	
10	45	求和
11	5.00	平均
12	9	最大值
13	1	最小值

A1-A9就可以看做 `Publisher<T>` 及其提供的元素序列。

A10-A13分别是求和函数 `SUM(A1:A9)`、平均函数 `AVERAGE(A1:A9)`、最大值函数 `MAX(A1:A9)`、最小值函数 `MIN(A1:A9)`，

A10-A13可以看作订阅者 `Subscriber`。

假如说我们没有**A10-A13**，那么**A1-A9**就没有实际意义，它们并不产生 计算。

这也是响应式的一个重要特点：**当没有订阅时发布者什么也不做。**

而Flux和Mono都是 `Publisher<T>` 在`Reactor 3` 实现类。

`Publisher<T>` 提供了 `subscribe` 方法，允许增加订阅的消费者，订阅之后，消费者在有结果可用时进行消费。

如果没有消费者，`Publisher<T>` 不会做任何事情，所以，Publisher根据消费情况进行响应。

`Publisher<T>` 可能返回零或者多个，甚至可能是无限的，为了更加清晰表示期待的结果就引入了两个实现模型Mono和Flux。

深入Publishers 发布者（发射序列）

异步处理将I/O或计算与调用该操作的线程进行解耦。

异步处理返回结果的未来句柄，通常是 `java.util.concurrent.Future` 或类似的东西，它返回单个对象，集合或异常。

使用Java 8或Promise模式，可以设置future的线性链接，以便发出后续的异步请求。一旦需要条件处理，就必须中断并同步异步流。尽管这种方法是可行的，但它并未充分利用异步处理的优势。

`Publisher<T>` 支持值甚至是无限流的发射序列，而不仅是单个标量值的发射（如`Future`那样）。一旦开始处理流而不是单个值，你将非常感谢这个事实。

Project Reactor的词汇表使用两种类型：`Mono` 和 `Flux`，它们都是发布者。

`Mono` 可以发出0到1个事件，而 `Flux` 可以发出0到N个事件。

`Publisher<T>` 不会偏向某些特定的并发性或异步性来源，也不会偏向于在 `ThreadPool` 中运行基础代码的执行方式-**同步还是异步**。作为 `Publisher<T>` 的消费者，你将实际的实现留给了生产者，生产者可以在以后修改它而无需修改代码。

`Publisher<T>` 的最后一个关键点是，底层处理不是在获取 `Publisher<T>` 时开始的，而是在观察者订阅或向 `Publisher<T>` 发出信号的那一刻开始的。

这与 `java.util.concurrent.Future` 至关重要，后者在创建/获取（created/obtained）时在某个地方启动。因此，如果没有观察者订阅 `Publisher<T>`，则将不会发生任何事情。

Publisher 和 Iterable (pull) 的对比

event	Iterable (pull)	Publisher (push)
retrieve data	<code>T next()</code>	<code>onNext(T)</code>
discover error	throws Exception	<code>onError(Exception)</code>
complete	<code>!hasNext()</code>	<code>onCompleted()</code>

Publisher 如何使用

与发布者合作时，你要做的第一件事就是消费它们。

消费发布者意味着订阅它。

这是一个订阅并打印所有发出的项目的示例：

```
Flux.just("Ben", "Michael", "Mark").subscribe(new Subscriber<String>() {
    public void onSubscribe(Subscription s) {
        s.request(3);
    }

    public void onNext(String s) {
        System.out.println("Hello " + s + "!");
    }

    public void onError(Throwable t) {
    }

    public void onComplete() {
        System.out.println("Completed");
    }
});
```

该示例打印以下行：

```
Hello BenHello MichaelHello MarkCompleted
```

你可以看到订阅者（或观察者）收到每个事件的通知，并且还接收到已完成的事件。

`Publisher<T>` 会发出项目（items），直到引发异常或 `Publisher<T>` 完成调用 `onCompleted` 的发出为止。在那之后不再发出其他元素。

对 `subscribe` 的调用会注册一个允许取消的 `Subscription`，因此不会接收其他事件。

一旦订阅者从 `Publisher<T>` 中取消订阅，发布者便可以与取消订阅和免费资源进行互操作。

实现 `Subscriber<T>` 需要实现多种方法，因此让我们将代码重写为更简单的形式：

```
Flux.just("Ben", "Michael", "Mark").doOnNext(new Consumer<String>() {
    public void accept(String s) {
        System.out.println("Hello " + s + "!");
    }
}).doOnComplete(new Runnable() {
    public void run() {
        System.out.println("Completed");
    }
}).subscribe();
```

或者，使用Java 8 Lambdas甚至更简单：

```
Flux.just("Ben", "Michael", "Mark")
    .doOnNext(s -> System.out.println("Hello " + s + "!"))
    .doOnComplete(() -> System.out.println("Completed"))
    .subscribe();
```

你可以使用运算符控制 `Subscriber` 处理的元素。

如果仅对前 `N` 个元素感兴趣，`take()` 运算符将限制发射项目的数量。

```
Flux.just("Ben", "Michael", "Mark") //
    .doOnNext(s -> System.out.println("Hello " + s + "!"))
    .doOnComplete(() -> System.out.println("Completed"))
    .take(2)
    .subscribe();
```

该示例打印以下行：

```
Hello Ben
Hello Michael
Completed
```

请注意，一旦发出预期的元素计数，`take`操作符就会从 `Publisher<T>` 隐式取消其订阅。

可以通过另一个`Flux`或`Subscriber`来完成对 `Publisher<T>` 的订阅。

除非要实现自定义`Publisher`，否则请始终使用`Subscriber`。

上例中使用的订阅者`Consumer`不处理异常，因此一旦引发异常，你将看到如下堆栈跟踪：

```
Exception in thread "main" reactor.core.Exceptions$BubblingException:
java.lang.RuntimeException: Example exception
    at reactor.core.Exceptions.bubble(Exceptions.java:96)
    at reactor.core.publisher.Operators.onErrorDropped(Operators.java:296)
    at
    reactor.core.publisher.LambdaSubscriber.onError(LambdaSubscriber.java:117)
    ...
Caused by: java.lang.RuntimeException: Example exception
    at demos.lambda$example3Lambda$4(demos.java:87)
    at
    reactor.core.publisher.FluxPeekFuseable$PeekFuseableSubscriber.onNext(FluxPeekFu
seable.java:157)
    ... 23 more
```

始终建议从一开始就实施错误处理程序。在某些时候，事情可能并且会出错。

完全实现的订阅者声明 `onCompleted` 和 `onError` 方法，使你可以对以下事件作出反应：

```
Flux.just("Ben", "Michael", "Mark").subscribe(new Subscriber<String>() {
    public void onSubscribe(Subscription s) {
        s.request(3);
    }

    public void onNext(String s) {
        System.out.println("Hello " + s + "!");
    }

    public void onError(Throwable t) {
        System.out.println("onError: " + e);
    }

    public void onComplete() {
        System.out.println("Completed");
    }
});
```

从push到pull

上面的示例说明了如何以一种非阻塞式或非阻塞式执行的方式设置发布者。

`Flux<T>` 可以显式转换为 `Iterable<T>` 或与 `block()` 同步。

开始表达代码内部执行的本质时，请避免在代码中调用 `block()`。

调用 `block()` 消除了应用程序反应链（reactive chain）的所有非阻塞优势。

```
String last = Flux.just("Ben", "Michael", "Mark").last().block();
System.out.println(last);
```

该示例打印以下行：

```
Mark
```

阻塞调用可用于同步发布者链（chain），并找到进入普通且众所周知的 `pull` 模式的方法。

```
List<String> list = Flux.just("Ben", "Michael", "Mark").collectList().block();
System.out.println(list);
```

`toList` 运算符收集所有发出的元素，并将列表通过 `BlockingPublisher<T>` 传递。

该示例打印以下行：

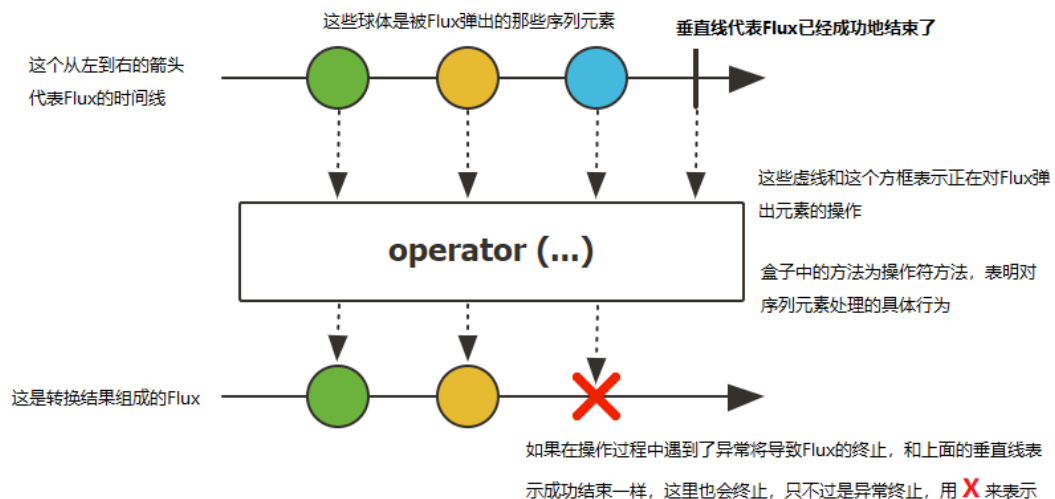
```
[Ben, Michael, Mark]
```

Flux 流（多个元素的发射序列）

Flux 是一个发出(emit) 0-N 个元素组成的异步序列的 `Publisher<T>`，可以被 `onComplete` 信号或者 `onError` 信号所终止。

在响应流规范中存在三种给下游消费者调用的方法 `onNext`，`onComplete`，和 `onError`。

下面这张图表示了Flux的抽象模型：



以上的讲解对于初次接触反应式编程的依然是难以理解的，所以这里有一个循序渐进的理解过程。

有些类比并不是很妥当，但是对于你循序渐进的理解这些新概念还是有帮助的。

传统数据处理

我们在平常是这么写的：

```
public List<User> allUsers() {
    return Arrays.asList(new User("A"), new User("B"));
}
```

我们通过迭代返回值 `List` 来 `get` 这些元素进行再处理（消费），不管有没有消费者，菜品都会生产出来。

流式数据处理

在Java 8中我们可以改写为流的表示：

```
public Stream<User> allUsers() {
    return Stream.of(new User("A"), new User("B"));
}
```

反应式数据处理

在Reactor中我们又可以改写为Flux表示：

```
public Flux<User> allUsers(){
    return Flux.just(new User("A"), new User("B"));
}
```

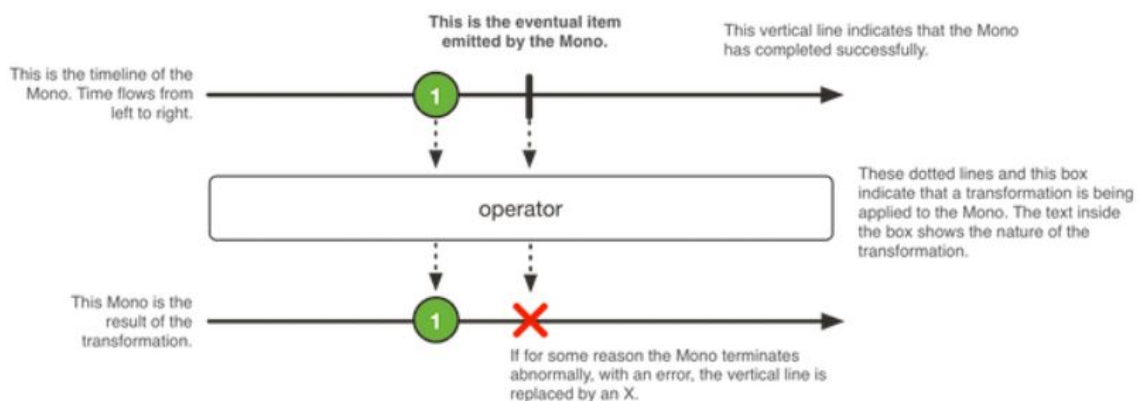
这时候食客来了，发生了订阅，厨师才开始做。

Flux 的创建Demo

```
Flux ints = Flux.range(1, 4);
Flux seq1 = Flux.just("bole1", "bole2", "bole3");
List iterable = Arrays.asList("bole_01", "bole_02", "bole_03");
Flux seq2 = Flux.fromIterable(iterable);
seq2.subscribe(i -> System.out.println(i));
```

Mono<T>

Mono 是一个发出(emit) 0-1 个元素的 Publisher<T>, 可以被 onComplete 信号或者 onError 信号所终止。



mono 整体和Flux差不多，只不过这里只会发出0-1个元素。也就是说不是有就是没有。

象Flux一样，我们来看看Mono的演化过程以帮助理解。

传统数据处理

```
public User currentUser () {  
    return isAuthenticated ? new User("felord.cn", "reactive") : null;  
}
```

直接返回符合条件的对象或者null`。

Optional的处理方式

```
public Optional<User> currentUser () {  
    return isAuthenticated ? Optional.of(new User("felord.cn", "reactive"))  
        : Optional.empty();  
}
```

这个Optional我觉得就有反应式的那种味儿了，当然它并不是反应式。当我们不从返回值Optional取其中具体的对象时，我们不清楚里面到底有没有，但是Optional是一定客观存在的,不会出现NPE问题。

反应式数据处理

```
public Mono<User> currentUser () {  
    return isAuthenticated ? Mono.just(new User("felord.cn", "reactive"))  
        : Mono.empty();  
}
```

和Optional有点类似的机制，当然Mono不是为了解决NPE问题的，它是为了处理响应流中单个值（也可能是void）而存在的。

Mono的创建Demo

```
Mono data = Mono.just("bole");  
Mono noData = Mono.empty();  
m.subscribe(i -> System.out.println(i));
```

函数编程

反应式编程，常常和函数式编程结合，这就是让大家困扰的地方

函数编程接口

接口函数名	说明
BiConsumer	表示接收两个输入参数和不返回结果的操作。
BiFunction	表示接受两个参数，并产生一个结果的函数。
BinaryOperator	表示在相同类型的两个操作数的操作，生产相同类型的操作数的结果。
BiPredicate	代表两个参数谓词（布尔值函数）。
BooleanSupplier	代表布尔值结果的提供者。
Consumer	表示接受一个输入参数和不返回结果的操作。
DoubleBinaryOperator	代表在两个double值操作数的运算，并产生一个double值结果。
DoubleConsumer	表示接受一个double值参数，不返回结果的操作。
DoubleFunction	表示接受double值参数，并产生一个结果的函数。
DoublePredicate	代表一个double值参数谓词（布尔值函数）。
DoubleSupplier	表示表示接受double值参数，并产生一个结果的函数。值结果的提供者。
DoubleToIntFunction	表示接受一个double值参数，不返回结果的操作。
DoubleFunction	表示接受double值参数，并产生一个结果的函数。
DoublePredicate	代表一个double值参数谓词（布尔值函数）。
DoubleSupplier	DoubleToIntFunction
DoubleToIntFunction	表示接受double值参数，并产生一个int值结果的函数。
DoubleToLongFunction	表示上产生一个double值结果的单个double值操作数的操作。
Function	代表接受一个double值参数，并产生一个long值结果的函数。
DoubleUnaryOperator	表示上产生一个double值结果的单个double值操作数的操作。
Function	表示接受一个参数，并产生一个结果的函数。
IntConsumer	表示接受单个int值的参数并没有返回结果的操作。
IntFunction	表示接受一个int值参数，并产生一个结果的函数。
IntPredicate	表示一个整数值参数谓词（布尔值函数）。

接口函数名	说明
IntSupplier	代表整型值的结果的提供者。
IntToLongFunction	表示接受一个int值参数，并产生一个long值结果的函数。
IntUnaryOperator	表示产生一个int值结果的单个int值操作数的运算。
LongBinaryOperator	表示在两个long值操作数的操作，并产生一个ObjLongConsumer值结果。
LongFunction	表示接受long值参数，并产生一个结果的函数。
LongPredicate	代表一个long值参数谓词（布尔值函数）。
LongSupplier	表示long值结果的提供者。
LongToDoubleFunction	表示接受double参数，并产生一个double值结果的函数。
LongToIntFunction	表示接受long值参数，并产生一个int值结果的函数。
LongUnaryOperator	表示上产生一个long值结果单一的long值操作数的操作。
ObjDoubleConsumer	表示接受对象值和double值参数，并且没有返回结果的操作。
ObjIntConsumer	表示接受对象值和整型值参数，并返回没有结果的操作。
ObjLongConsumer	表示接受对象值和整型值参数，并返回没有结果的操作。
ObjLongConsumer	表示接受对象值和double值参数，并且没有返回结果的操作。
ObjIntConsumer	表示接受对象值和整型值参数，并返回没有结果的操作。
ObjLongConsumer	表示接受对象的值和long值的说法，并没有返回结果的操作。
Predicate	代表一个参数谓词（布尔值函数）。
Supplier	表示一个提供者的结果。
ToDoubleBiFunction	表示接受两个参数，并产生一个double值结果的功能。
ToDoubleFunction	代表一个产生一个double值结果的功能。
ToIntBiFunction	表示接受两个参数，并产生一个int值结果的函数。
ToIntFunction	代表产生一个int值结果的功能。
ToLongBiFunction	表示接受两个参数，并产生long值结果的功能。
ToLongFunction	代表一个产生long值结果的功能。
UnaryOperator	表示上产生相同类型的操作数的结果的单个操作数的操作。

常用函数编程示例

Consumer ：有一个输入 无输出 函数接口

```

Product product=new Product();

//类名+静态方法 有一个输入T, 没有输出
Consumer consumer1 = Product->Product.nameOf(product);//lambda
consumer1.accept(product);

Consumer consumer = Product::nameOf;//方法引用
consumer.accept(product);

```

Function<T,R> : 一个输入 一个输出 函数接口

```

//对象+方法 一个输入T 一个输出R
Function<Integer, Integer> function = product::reduceStock;
System.out.println("剩余库存: " + function.apply(10));

//带参数的构造函数
Function<Integer,Product> function1=Product::new;
System.out.println("新对象:" +function1.apply(200));

```

Predicate : 一个输入T, 一个输出 Boolean

```

//Predicate 一个输入T 一个输出Boolean
Predicate predicate= i -> product.isEnough(i);//lambda
System.out.println("库存是否足够: "+predicate.test(100));
Predicate predicate1= product::isEnough;//方法引用
System.out.println("库存是否足够: "+predicate1.test(100));

```

UnaryOperator : 一元操作符 ,输入输出都是T

```

//一元操作符 输入和输出T
UnaryOperator integerUnaryOperator =product::reduceStock;
System.out.println("剩余库存: " + integerUnaryOperator.apply(20));
IntUnaryOperator intUnaryOperator = product::reduceStock;
System.out.println("剩余库存: " + intUnaryOperator.applyAsInt(30));

```

Supplier : 没有输入 只有输出

```

//无参数构造函数
Supplier supplier = Product::new;
System.out.println("创建新对象:" + supplier.get());

```

```

Supplier supplier1=()->product.getStock();
System.out.println("剩余库存:" + supplier1.get());

```

BiFunction: 二元操作符 两个输入<T,U> 一个输出

```

//类名+方法
BiFunction<Product, Integer, Integer> binaryOperator = Product::reduceStock;
System.out.println(" 剩余库存(BiFunction): " + binaryOperator.apply(product, 10));

```

BinaryOperator : 二元操作符, 二个输入, 一个输出

```
//BinaryOperator binaryOperator1=(x,y)->product.reduceStock(x,y);
BinaryOperator binaryOperator1=product::reduceStock;
System.out.println(" 剩余库存(BinaryOperator): "
+binaryOperator1.apply(product.getStock(),10));
```

Flux类中的静态方法:

简单的创建方法

just():

可以指定序列中包含的全部元素。创建出来的Flux序列在发布这些元素之后会自动结束

fromArray(), fromIterable(), fromStream():

可以从一个数组, Iterable对象或Stream对象中穿件Flux对象

empty():

创建一个不包含任何元素, 只发布结束消息的序列

error(Throwable error):

创建一个只包含错误消息的序列

never():

传建一个不包含任务消息通知的序列

range(int start, int count):

创建包含从start起始的count个数量的Integer对象的序列

interval(Duration period)和interval(Duration delay, Duration period):

创建一个包含了从0开始递增的Long对象的序列。其中包含的元素按照指定的间隔来发布。除了间隔时间之外, 还可以指定起始元素发布之前的延迟时间

intervalMillis(long period)和intervalMillis(long delay, long period):

与interval()方法相同, 但该方法通过毫秒数来指定时间间隔和延迟时间

例子

```
Flux.just("Hello", "World").subscribe(System.out::println);
Flux.fromArray(new Integer[]{1, 2, 3}).subscribe(System.out::println);
Flux.empty().subscribe(System.out::println);
Flux.range(1, 10).subscribe(System.out::println);
Flux.interval(Duration.of(10,
ChronoUnit.SECONDS)).subscribe(System.out::println);
Flux.intervalMillis(1000).subscirbe(System.out::println);
```

复杂的序列创建 generate()

当序列的生成需要复杂的逻辑时, 则应该使用generate()或create()方法。

generate()方法通过同步和逐一的方式来产生Flux序列。

序列的产生是通过调用所提供的SynchronousSink对象的next(), complete()和error(Throwable)方法来完成的。

逐一生成的含义是在具体的生成逻辑中，next()方法只能最多被调用一次。

在某些情况下，序列的生成可能是有状态的，需要用到某些状态对象，此时可以使用

```
generate(Callable<S> stateSupplier, BiFunction<S, SynchronousSink<T>, S> generator),
```

其中stateSupplier用来提供初始的状态对象。

在进行序列生成时，状态对象会作为generator使用的第一个参数传入，可以在对应的逻辑中对状态对象进行修改以供下一次生成时使用。

```
Flux.generate(sink -> {
    sink.next("Hello");
    sink.complete();
}).subscribe(System.out::println);

final Random random = new Random();
Flux.generate(ArrayList::new, (list, sink) -> {
    int value = random.nextInt(100);
    list.add(value);
    sink.next(value);
    if( list.size() == 10 )
        sink.complete();
    return list;
}).subscribe(System.out::println);
```

复杂的序列创建 create()

create()方法与generate()方法的不同之处在于所使用的是FluxSink对象。

FluxSink支持同步和异步的消息产生，并且可以在一次调用中产生多个元素。

```
Flux.create(sink -> {
    for(int i = 0; i < 10; i++)
        sink.next(i);
    sink.complete();
}).subscribe(System.out::println);
```

Mono静态方法

Mono类包含了与Flux类中相同的静态方法：just(), empty()和never()等。

除此之外，Mono还有一些独有的静态方法：

- fromCallable(), fromCompletionStage(), fromFuture(), fromRunnable()和fromSupplier(): 分别从Callable, CompletionStage, CompletableFuture, Runnable和Supplier中创建Mono

- `delay(Duration duration)`和`delayMillis(long duration)`：创建一个Mono序列，在指定的延迟时间之后，产生数字0作为唯一值
- `ignoreElements(Publisher<T> source)`：创建一个Mono序列，忽略作为源的Publisher中的所有元素，只产生消息
- `justOrEmpty(Optional<? extends T> data)`和`justOrEmpty(T data)`：从一个Optional对象或可能为null的对象中创建Mono。只有Optional对象中包含之或对象不为null时，Mono序列才产生对应的元素

例子：

```
Mono.fromSupplier(() -> "Hello").subscribe(System.out::println);
Mono.justOrEmpty(Optional.of("Hello")).subscribe(System.out::println);
Mono.create(sink -> sink.success("Hello")).subscribe(System.out::println);
```

操作符

操作符buffer和bufferTimeout

这两个操作符的作用是把当前流中的元素收集到集合中，并把集合对象作为流中的新元素。

```
Flux.range(1, 100).buffer(20).subscribe(System.out::println);
```

在进行收集时可以指定不同的条件：所包含的元素的最大数量或收集的时间间隔。

方法`buffer()`仅使用一个条件，而`bufferTimeout()`可以同时指定两个条件。

指定时间间隔时可以使用`Duration`对象或毫秒数，即使用`bufferMillis()`或`bufferTimeoutMillis()`两个方法。

除了元素数量和时间间隔外，还可以通过`bufferUntil`和`bufferWhile`操作符来进行收集。这两个操作符的参数时表示每个集合中的元素所要满足的条件的Predicate对象。

`bufferUntil`会一直收集直到Predicate返回true。

使得Predicate返回true的那个元素可以选择添加到当前集合或下一个集合中；`bufferWhile`则只有当Predicate返回true时才会收集。一旦为false，会立即开始下一次收集。

```
Flux.intervalMillis(100).bufferMillis(1001).take(2).toStream().forEach(System.out::println);
Flux.range(1, 10).bufferUntil(i -> i%2 == 0).subscribe(System.out::println);
Flux.range(1, 10).bufferWhile(i -> i%2 == 0).subscribe(System.out::println);
```

操作符Filter

对流中包含的元素进行过滤，只留下满足Predicate指定条件的元素。

```
Flux.range(1, 10).filter(i -> i%2 == 0).subscribe(System.out::println);
```

操作符zipWith

zipWith操作符把当前流中的元素与另一个流中的元素按照一对一的方式进行合并。在合并时可以不做任何处理，由此得到的是一个元素类型为Tuple2的流；也可以通过一个BiFunction函数对合并的元素进行处理，所得到的流的元素类型为该函数的返回值。

```
Flux.just("a", "b").zipWith(Flux.just("c", "d")).subscribe(System.out::println);
Flux.just("a", "b").zipWith(Flux.just("c", "d"), (s1, s2) -> String.format("%s-%s", s1, s2)).subscribe(System.out::println);
```

操作符take

take系列操作符用来从当前流中提取元素。提取方式如下：

take(long n), take(Duration timespan)和takeMillis(long timespan)：按照指定的数量或时间间隔来提取

takeLast(long n)：提取流中的最后N个元素

takeUntil(Predicate<? super T> predicate)：提取元素直到Predicate返回true

takeWhile(Predicate<? super T> continuePredicate)：当Predicate返回true时才进行提取

takeUntilOther(Publisher<?> other)：提取元素知道另外一个流开始产生元素

```
Flux.range(1, 1000).take(10).subscribe(System.out::println);
Flux.range(1, 1000).takeLast(10).subscribe(System.out::println);
Flux.range(1, 1000).takeWhile(i -> i < 10).subscribe(System.out::println);
Flux.range(1, 1000).takeUntil(i -> i == 10).subscribe(System.out::println);
```

操作符reduce和reduceWith

reduce和reduceWith操作符对流中包含的所有元素进行累计操作，得到一个包含计算结果的Mono序列。累计操作是通过一个BiFunction来表示的。在操作时可以指定一个初始值。若没有初始值，则序列的第一个元素作为初始值。

```
Flux.range(1, 100).reduce((x, y) -> x + y).subscribe(System.out::println);
Flux.range(1, 100).reduceWith(() -> 100, (x + y) -> x + y).subscribe(System.out::println);
```

操作符merge和mergeSequential

merge和mergeSequential操作符用来把多个流合并成一个Flux序列。merge按照所有流中元素的实际产生序列来合并，而mergeSequential按照所有流被订阅的顺序，以流为单位进行合并。

```
Flux.merge(Flux.intervalMillis(0, 100).take(5), Flux.intervalMillis(50, 100).take(5)).toStream().forEach(System.out::println);
Flux.mergeSequential(Flux.intervalMillis(0, 100).take(5), Flux.intervalMillis(50, 100).take(5)).toStream().forEach(System.out::println);
```

操作符flatMap和flatMapSequential

flatMap和flatMapSequential操作符把流中的每个元素转换成一个流，再把所有流中的元素进行合并。flatMapSequential和flatMap之间的区别与mergeSequential和merge是一样的。

```
Flux.just(5, 10).flatMap(x -> Flux.intervalMillis(x * 10, 100)).take(x)).toStream().forEach(System.out::println);
```

操作符concatMap

concatMap操作符的作用也是把流中的每个元素转换成一个流，再把所有流进行合并。concatMap会根据原始流中的元素顺序依次把转换之后的流进行合并，并且concatMap堆转换之后的流的订阅是动态进行的，而flatMapSequential在合并之前就已经订阅了所有的流。

```
Flux.just(5, 10).concatMap(x -> Flux.intervalMillis(x * 10, 100)).take(x)).toStream().forEach(System.out::println);
```

操作符combineLatest

combineLatest操作符把所有流中的最新产生的元素合并成一个新的元素，作为返回结果流中的元素。只要其中任何一个流中产生了新的元素，合并操作就会被执行一次，结果流中就会产生新的元素。

```
Flux.combineLatest(Arrays::toString, Flux.intervalMillis(100).take(5), Flux.intervalMillis(50, 100).take(5)).toStream().forEach(System.out::println);
```

消息处理

当需要处理Flux或Mono中的消息时，可以通过subscribe方法来添加相应的订阅逻辑。

在调用subscribe方法时可以指定需要处理的消息类型。

```
Flux.just(1, 2).concatWith(Mono.error(new IllegalStateException())).subscribe(System.out::println, System.err::println);

Flux.just(1, 2).concatWith(Mono.error(new IllegalStateException())).onErrorReturn(0).subscribe(System.out::println);
```

第2种可以通过switchOnError()方法来使用另外的流来产生元素。

```
Flux.just(1, 2).concatWith(Mono.error(new IllegalStateException())).switchOnError(Mono.just(0)).subscribe(System.out::println);
```

第三种是通过onErrorResumeWith()方法来根据不同的异常类型来选择要使用的产生元素的流。

```
Flux.just(1, 2).concatWith(Mono.error(new
IllegalArgumentException())).onErrorResumeWith(e -> {
    if(e instanceof IllegalStateException)
        return Mono.just(0);
    else if(e instanceof IllegalArgumentException)
        return Mono.just(-1);
    return Mono.empty();
}).subscribe(System.out::println);
```

当出现错误时还可以使用retry操作符来进行重试。重试的动作是通过重新订阅序列来实现的。在使用retry操作时还可以指定重试的次数。

```
Flux.just(1, 2).concatWith(Mono.error(new
IllegalStateException())).retry(1).subscribe(System.out::println);
```

测试

StepVerifier的作用是可以对序列中包含的元素进行逐一验证。通过StepVerifier.create()方法对一个流进行包装之后再验证。expectNext()方法用来声明测试时所期待的流中的下一个元素的值，而verifyComplete()方法则验证流是否正常结束。verifyError()来验证流由于错误而终止。

```
StepVerifier.create(Flux.just(a,
b)).expectNext("a").expectNext("b").verifyComplete();
```

使用StepVerifier.withVirtualTime()方法可以创建出使用虚拟时钟的StepVerifier。通过thenAwait(Duration)方法可以让虚拟时钟前进。

```
StepVerifier.withVirtualTime(() -> Flux.interval(Duration.ofHours(4),
Duration.ofDays(1)).take(2))
    .expectSubscription()
    .expectNoEvent(Duration.ofHours(4))
    .expectNext(0L)
    .thenAwait(Duration.ofDays(1))
    .expectNext(1L)
    .verifyComplete();
```

TestPublisher的作用在于可以控制流中元素的产生，甚至是违反反应流规范的情况。通过create()方法创建一个新的TestPublisher对象，然后使用next()方法来产生元素，使用complete()方法来结束流。

```
final TestPublisher<String> testPublisher = TestPublisher.create();
testPublisher.next("a");
testPublisher.next("b");
testPublisher.complete();

StepVerifier.create(testPublisher)
    .expectNext("a")
    .expectNext("b")
    .expectComplete();
```

调试

在调试模式启用之后，所有的操作符在执行时都会保存额外的与执行链相关的信息。当出现错误时，这些信息会被作为异常堆栈信息的一部分输出。

```
Hooks.onOperator(providedHook -> providedHook.operatorStacktrace());
```

也可以通过checkpoint操作符来对特定的流处理链来启用调试模式。

```
Flux.just(1, 0).map(x -> 1/x).checkpoint("test").subscribe(System.out::println);
```

日志记录

可以通过添加log操作把流相关的事件记录在日志中，

```
Flux.range(1, 2).log("Range").subscribe(System.out::println);
```

冷热序列

冷序列的含义是不论订阅者在何时订阅该序列，总是能收到序列中产生的全部消息。

热序列是在持续不断的产生消息，订阅者只能获取到在其订阅之后产生的消息。

```
final Flux<Long> source =  
    Flux.intervalMillis(1000).take(10).publish().autoConnect();  
source.subscribe();  
Thread.sleep(5000);  
source.toStream().forEach(System.out::println);
```

map、flatMap 以及 flatMapSequential 区别：

```
//Map 的方法签名  
<V> Flux<V> map(Function<? super T, ? extends V> mapper)  
  
//FlatMap的方法签名  
<R> Flux<R> flatMap(Function<? super T, ? extends Publisher<? extends R>>  
    mapper)
```

map:

通过对每个元素应用 转换函数来 同步 转换此Flux发出的元素，并且是一对一的转换流元素。

```
reactor.core.publisher.Flux<T>
public final <V> Flux<V> map(@NotNull java.util.function.Function<? super T, ? extends V> mapper)
```

Transform the items emitted by this **Flux** by applying a synchronous function to each item.

Params: mapper – the synchronous transforming **Function**

Type parameters: <V> – the transformed type

Returns: a transformed **Flux**

Inferred annotations: @org.jetbrains.annotations.NotNull

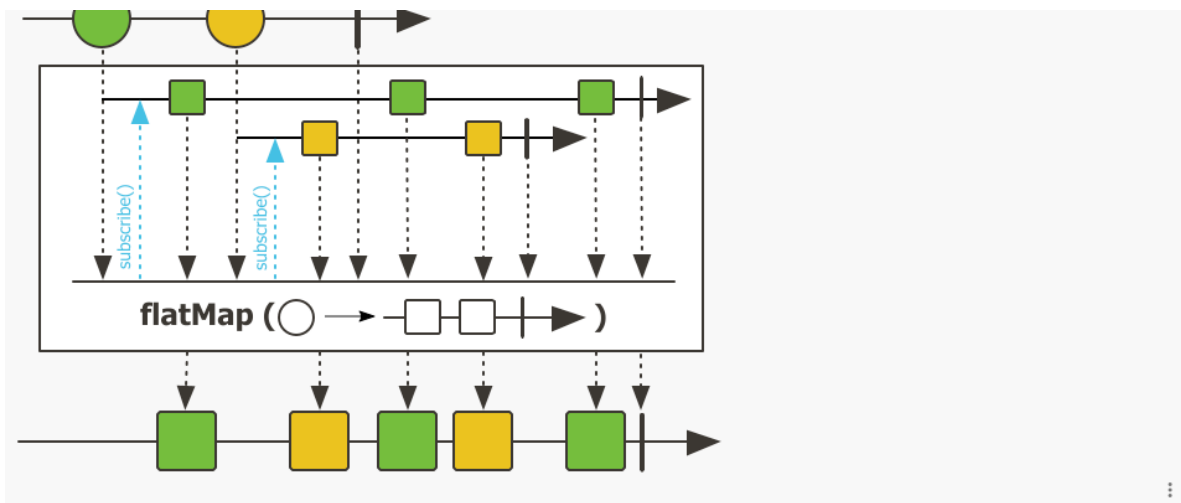
Gradle: io.projectreactor:reactor-core:3.3.11.RELEASE

flatMap:

将此Flux发出的元素 异步 转换为 Publisher，然后通过合并将这些内部Publisher，最终扁平化为单个 Flux。

Flux可能包含N个元素，所以flatMap是一对多的转换。

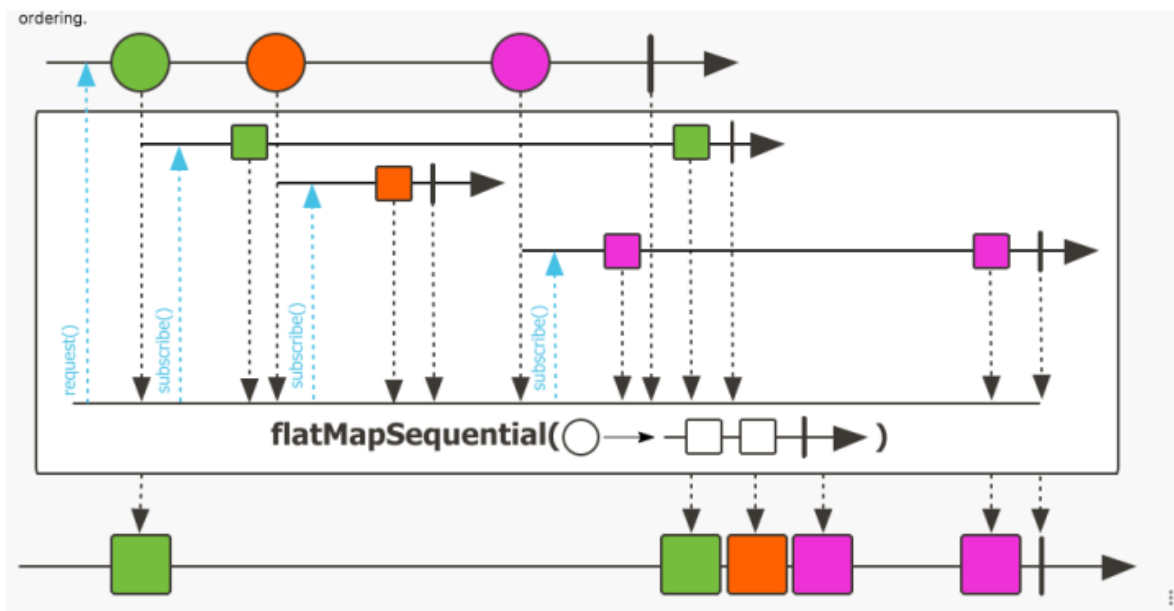
Flux 将各个 publisher 合并的过程中，不会保持 源Flux发布 的顺序，可能出现交错。



flatMapSequential:

将此Flux发出的元素异步转换为 Publisher，然后通过合并将这些内部发布者扁平化为单个Flux。

与 flatMap 不同的是 flatMapSequential 在合并 publisher 时，会 按源元素的顺序合并它们。



map是一个同步运算符，它只是一种将一个值转换为另一个值的方法。

flatMap可以是同步的，也可以是异步的，这取决于flatMap中调用的方法怎么使用。

示例1：

```
void demo() {
    //1. Flux.interval 按时生产Long的无限流
    final Flux<Long> flux = Flux.interval(Duration.of(100, ChronoUnit.MILLIS))
        .map(log()) //2. 调用方法log，订阅后log()会在当前线程执行
        .flatMap(logOfFlux()); //3. 通过flatMap调用 log()，会在当前线程执行
    //完成flux的定义，调用subscribe后才会真正开始执行
    flux.subscribe();
}

Function<Long, Long> log() {
    return aLong -> {
        log.info("num is {}", aLong);
        return aLong;
    };
}

Function<Long, Flux<Long>> logOfFlux() {
    return aLong -> {
        log.info("num is {}", aLong);
        return Flux.just(aLong);
    };
}
```

在我们的示例代码中，该 flatMap 操作是同步的，因为我们使用Flux.just()方法发出元素。

下面我们会介绍如何在 flatMap 中实现异步操作。

上边代码中讲到了 Publisher 调用 subscribe 后才会真正开始执行，所以 subscribe 中的代码并不一定会执行。

当 Mono 是空序列时：

```
void monoOfEmpty() {
    Mono.empty()
        .map(m -> func())
        .subscribe(message -> {
            responseObserver.onNext(message);
            responseObserver.onCompleted();
        });
}
```

会有同学喜欢在 subscribe 中处理响应（例如rpc），但这个场景中responseObserver.onCompleted()不会被执行。

正确的做法应该是：

```
void monoOfEmpty() {
    Mono.empty()
        .map(m -> func())
        .doOnSuccess(message -> responseObserver.onCompleted())
        .subscribe(responseObserver::onNext);
}
```

如何选择操作符？

本节的内容来自 [Reactor 3 参考文档——如何选择操作符](#)。

如果一个操作符是专属于 `Flux` 或 `Mono` 的，那么会给它注明前缀。
公共的操作符没有前缀。如果一个具体的用例涉及多个操作符的组合，这里以方法调用的方式展现，
会以一个点 (.) 开头，并将参数置于圆括号内，比如：`.methodCall(parameter)`。

1) 创建一个新序列，它...

1.1 发出一个T，用just:

`Mono` ,是指最多只能触发(emit) (事件)一次。

`Mono` 对应于 RxJava 库的 `Single` 和 `Maybe` 类型或者是java的Optional。

因此一个异步任务，如果只是想要在完成时给出完成信号，就可以使用 `Mono<Void>`。

```
@Test
public void testMonoBasic(){
    Mono.fromSupplier(() -> "Hello").subscribe(System.out::println);
    Mono.justOrEmpty(Optional.of("Hello")).subscribe(System.out::println);
    Mono.create(sink -> sink.success("Hello")).subscribe(System.out::println);
}
```

- 基于一个 `Optional<T>`: `Mono#justOrEmpty(Optional<T>)`
- 基于一个可能为 `null` 的 `T`: `Mono#justOrEmpty(T)`

`Flux` 相当于一个 RxJava `Observable`，能够发出 0~N 个数据项，然后（可选地）completing 或 erroring。Flux处理多个数据项作为stream。

```
@Test
public void testBasic(){
    Flux.just("Hello", "World").subscribe(System.out::println);
    Flux.fromArray(new Integer[] {1, 2, 3}).subscribe(System.out::println);
    Flux.empty().subscribe(System.out::println);
    Flux.range(1, 10).subscribe(System.out::println);
    Flux.interval(Duration.of(10,
ChronoUnit.SECONDS)).subscribe(System.out::println);
}
```

1.2 发出一个“懒”创建的T

- 还是由 just返回但是“懒”创建的:

使用 `Mono#fromSupplier` 或用 `defer` 包装 `just`

```
@Test
public void defer(){
    //声明阶段创建DeferredClass对象

    Mono<Date> m1 = Mono.just(new Date());
    Mono<Date> m2 = Mono.defer(()->Mono.just(new Date()));
    m1.subscribe(System.out::println);
    m2.subscribe(System.out::println);
    //延迟5秒钟
    try {
        Thread.sleep(5000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    m1.subscribe(System.out::println);
    m2.subscribe(System.out::println);
}
```

1.3发出许多 T,

- 这些元素我可以明确列举出来: `Flux#just(T...)`
- 基于迭代数据结构:
 - 一个数组: `Flux#fromArray`
 - 一个集合或 iterable: `Flux#fromIterable`
 - 一个 Integer 的 range: `Flux#range`
 - 一个 Stream 提供给每一个订阅: `Flux#fromStream(Supplier<Stream>)`
- 基于一个参数值给出的源:
 - 一个 `Supplier<T>`: `Mono#fromSupplier`
 - 一个任务: `Mono#fromCallable`, `Mono#fromRunnable`
 - 一个 `CompletableFuture<T>`: `Mono#fromFuture`
- 直接完成: `empty`
- 立即生成错误: `error`
 - ...“懒”的方式生成 `Throwable`: `error(Supplier<Throwable>)`
- 什么都不做: `never`

- 订阅时才决定: `defer`
- 依赖一个可回收的资源: `using`
- 可编程地生成事件 (可以使用状态):
 - 同步且逐个的: `Flux#generate`
 - 异步 (也可同步) 的, 每次尽可能多发出元素: `Flux#create` (`Mono#create` 也类似的, 只不过只能发一个)

2) 对序列进行转化

- 我想转化一个序列:
 - 1对1地转化 (比如字符串转化为它的长度): `map`
 - ...类型转化: `cast`
 - ...为了获得每个元素的序号: `Flux#index`
 - 1对n地转化 (如字符串转化为一串字符): `flatMap` + 使用一个工厂方法
 - 1对n地转化可自定义转化方法和/或状态: `handle`
 - 对每一个元素执行一个异步操作 (如对 url 执行 http 请求): `flatMap` + 一个异步的返回类型为 `Publisher` 的方法
 - ...忽略一些数据: 在 `flatMap` lambda 中根据条件返回一个 `Mono.empty()`
 - ...保留原来的序列顺序: `Flux#flatMapSequential` (对每个元素的异步任务会立即执行, 但会将结果按照原序列顺序排序)
 - ...当 `Mono` 元素的异步任务会返回多个元素的序列时: `Mono#flatMapMany`
- 我想添加一些数据元素到一个现有的序列:
 - 在开头添加: `Flux#startWith(T...)`
 - 在最后添加: `Flux#concatWith(T...)`
- 我想将 `Flux` 转化为集合 (一下都是针对 `Flux` 的)
 - 转化为 List: `collectList`, `collectSortedList`
 - 转化为 Map: `collectMap`, `collectMultiMap`
 - 转化为自定义集合: `collect`
 - 计数: `count`
 - reduce 算法 (将上个元素的reduce结果与当前元素值作为输入执行reduce方法, 如sum) `reduce`
 - ...将每次 reduce 的结果立即发出: `scan`
 - 转化为一个 boolean 值:
 - 对所有元素判断都为true: `all`
 - 对至少一个元素判断为true: `any`
 - 判断序列是否有元素 (不为空): `hasElements`
 - 判断序列中是否有匹配的元素: `hasElement`
- 我想合并 publishers...
 - 按序连接: `Flux#concat` 或 `.concatWith(other)`
 - ...即使有错误, 也会等所有的 publishers 连接完成: `Flux#concatDelayError`
 - ...按订阅顺序连接 (这里的合并仍然可以理解成序列的连接): `Flux#mergeSequential`
 - 按元素发出的顺序合并 (无论哪个序列的, 元素先到先合并): `Flux#merge / .mergeWith(other)`
 - ...元素类型会发生变化: `Flux#zip / Flux#zipWith`
 - 将元素组合:
 - 2个 Monos 组成1个 `Tuple2`: `Mono#zipWith`

- n个 Monos 的元素都发出来后组成一个 Tuple: `Mono#zip`
- 在终止信号出现时“采取行动”:
- 在 Mono 终止时转换为一个 `Mono<Void>`: `Mono#and`
- 当 n 个 Mono 都终止时返回 `Mono<Void>`: `Mono#when`
- 返回一个存放组合数据的类型, 对于被合并的多个序列:
 - 每个序列都发出一个元素时: `Flux#zip`
 - 任何一个序列发出元素时: `Flux#combineLatest`
- 只取各个序列的第一个元素: `Flux#first`, `Mono#first`,
`mono.or(otherMono).or(thirdMono)`, `flux.or(otherFlux).or(thirdFlux)`
- 由一个序列触发 (类似于 `flatMap`, 不过“喜新厌旧”): `switchMap`
- 由每个新序列开始时触发 (也是“喜新厌旧”风格): `switchOnNext`
- 我想重复一个序列: `repeat`
 - ...但是以一定的间隔重复: `Flux.interval(duration).flatMap(tick -> myExistingPublisher)`
- 我有一个空序列, 但是...
 - 我想要一个缺省值来代替: `defaultIfEmpty`
 - 我想要一个缺省的序列来代替: `switchIfEmpty`
- 我有一个序列, 但是我对序列的元素值不感兴趣: `ignoreElements`
 - ...并且我希望用 `Mono` 来表示序列已经结束: `then`
 - ...并且我想在序列结束后等待另一个任务完成: `thenEmpty`
 - ...并且我想在序列结束之后返回一个 `Mono`: `Mono#then(mono)`
 - ...并且我想在序列结束之后返回一个值: `Mono#thenReturn(T)`
 - ...并且我想在序列结束之后返回一个 `Flux`: `thenMany`
- 我有一个 Mono 但我想延迟完成...
 - ...当有1个或N个其他 publishers 都发出 (或结束) 时才完成: `Mono#delayUntilOther`
 - ...使用一个函数式来定义如何获取“其他 publisher”: `Mono#delayUntil(Function)`
- 我想基于一个递归的生成序列的规则扩展每一个元素, 然后合并为一个序列发出:
 - ...广度优先: `expand(Function)`
 - ...深度优先: `expandDeep(Function)`

3) “窥视” (只读) 序列

- 再不对序列造成改变的情况下, 我想:
 - 得到通知或执行一些操作:
 - 发出元素: `doOnNext`
 - 序列完成: `Flux#doOnComplete`, `Mono#doOnSuccess`
 - 因错误终止: `doOnError`
 - 取消: `doOnCancel`
 - 订阅时: `doOnSubscribe`
 - 请求时: `doOnRequest`
 - 完成或错误终止: `doOnTerminate` (Mono的方法可能包含有结果)
 - 但是在终止信号向下游传递 之后: `doAfterTerminate`
 - 所有类型的信号 (`Signal`): `Flux#doOnEach`

- 所有结束的情况（完成complete、错误error、取消cancel）：`doFinally`
- 记录日志：`log`
- 我想知道所有的事件：
 - 每一个事件都体现为一个 `single` 对象：
 - 执行 callback：`doOnEach`
 - 每个元素转化为single对象：`materialize`
 - ...在转化回元素：`dematerialize`
 - 转化为一行日志：`log`

4) 过滤序列

- 我想过滤一个序列
 - 基于给定的判断条件：`filter`
 - ...异步地进行判断：`filterWhen`
 - 仅限于指定类型的对象：`ofType`
 - 忽略所有元素：`ignoreElements`
 - 去重：
 - 对于整个序列：`Flux#distinct`
 - 去掉连续重复的元素：`Flux#distinctUntilChanged`
- 我只想要一部分序列：
 - 只要 N 个元素：
 - 从序列的第一个元素开始算：`Flux#take(long)`
 - ...取一段时间内发出的元素：`Flux#take(Duration)`
 - ...只取第一个元素放到 `Mono` 中返回：`Flux#next()`
 - ...使用 `request(N)` 而不是取消：`Flux#limitRequest(long)`
 - 从序列的最后一个元素倒数：`Flux#takeLast`
 - 直到满足某个条件（包含）：`Flux#takeUntil`（基于判断条件），
`Flux#takeUntilOther`（基于对 publisher 的比较）
 - 直到满足某个条件（不包含）：`Flux#takeWhile`
 - 最多只取 1 个元素：
 - 给定序号：`Flux#elementAt`
 - 最后一个：

```
.takeLast(1)
```

- ...如果为序列空则发出错误信号：`Flux#last()`
- ...如果序列为空则返回默认值：`Flux#last(T)`
- 跳过一些元素：
- 从序列的第一个元素开始跳过：`Flux#skip(long)`
 - ...跳过一段时间内发出的元素：`Flux#skip(Duration)`
- 跳过最后的 n 个元素：`Flux#skipLast`
- 直到满足某个条件（包含）：`Flux#skipUntil`（基于判断条件），`Flux#skipUntilOther`
（基于对 publisher 的比较）
- 直到满足某个条件（不包含）：`Flux#skipWhile`

- 采样：
 - 给定采样周期：`Flux#sample(Duration)`
 - 取采样周期里的第一个元素而不是最后一个：`sampleFirst`
- 基于另一个 publisher：`Flux#sample(Publisher)`
- 基于 publisher“超时”：`Flux#sampleTimeout`（每一个元素会触发一个 publisher，如果这个 publisher 不被下一个元素触发的 publisher 覆盖就发出这个元素）
- 我只想要一个元素（如果多于一个就返回错误）...
 - 如果序列为空，发出错误信号：`Flux#single()`
 - 如果序列为空，发出一个缺省值：`Flux#single(T)`
 - 如果序列为空就返回一个空序列：`Flux#singleOrNull`

5) 错误处理

- 我想创建一个错误序列：`error...`
 - ...替换一个完成的 Flux：`.concat(Flux.error(e))`
 - ...替换一个完成的 Mono：`.then(Mono.error(e))`
 - ...如果元素超时未发出：`timeout`
 - ...“懒”创建：`error(Supplier<Throwable>)`
- 我想要类似 try/catch 的表达方式：
 - 抛出异常：`error`
 - 捕获异常：
 - 然后返回缺省值：`onErrorReturn`
 - 然后返回一个 Flux 或 Mono：`onErrorResume`
 - 包装异常后再抛出：`.onErrorMap(t -> new RuntimeException(t))`
 - finally 代码块：`doFinally`
 - Java 7 之后的 try-with-resources 写法：`using` 工厂方法
- 我想从错误中恢复...
 - 返回一个缺省的：
 - 的值：`onErrorReturn`
 - Publisher：`Flux#onErrorResume` 和 `Mono#onErrorResume`
 - 重试：`retry`
 - ...由一个用于伴随 Flux 触发：`retryWhen`
- 我想处理回压错误（向上游发出“MAX”的 request，如果下游的 request 比较少，则应用策略）...
 - 抛出 `IllegalStateException`：`Flux#onBackpressureError`
 - 丢弃策略：`Flux#onBackpressureDrop`
 - ...但是不丢弃最后一个元素：`Flux#onBackpressureLatest`
 - 缓存策略（有限或无限）：`Flux#onBackpressureBuffer`
 - ...当有限的缓存空间用满则应用给定策略：`Flux#onBackpressureBuffer` 带有策略 `BufferOverflowStrategy`

6) 基于时间的操作

- 我想将元素转换为带有时间信息的 `Tuple2<Long, T>` ...
 - 从订阅时开始：`elapsed`
 - 记录时间戳：`timestamp`
- 如果元素间延迟过长则中止序列：`timeout`
- 以固定的周期发出元素：`Flux#interval`

- 在一个给定的延迟后发出 0：static Mono.delay.
- 我想引入延迟：
 - 对每一个元素：Mono#delayElement, Flux#delayElements
 - 延迟订阅：delaySubscription

7) 拆分 Flux

- 我想将一个 Flux<T> 拆分为一个 Flux<Flux<T>>：
 - 以个数为界：window(int)
 - ...会出现重叠或丢弃的情况：window(int, int)
 - 以时间为界：window(Duration)
 - ...会出现重叠或丢弃的情况：window(Duration, Duration)
 - 以个数或时间为界：windowTimeout(int, Duration)
 - 基于对元素的判断条件：windowUntil
 - ...触发判断条件的元素会分到下一波（cutBefore 变量）：.windowUntil(predicate, true)
 - ...满足条件的元素在一波，直到不满足条件的元素发出开始下一波：windowWhile（不满足条件的元素会被丢弃）
 - 通过另一个 Publisher 的每一个 onNext 信号来拆分序列：window(Publisher), windowWhen
- 我想将一个 Flux<T> 的元素拆分到集合...
 - 拆分为一个一个的 List：
 - ...会出现重叠或丢弃的情况：buffer(int, int)
 - 以时间为界：buffer(Duration)
 - ...会出现重叠或丢弃的情况：buffer(Duration, Duration)
 - 以个数或时间为界：bufferTimeout(int, Duration)
 - 基于对元素的判断条件：bufferUntil(Predicate)
 - ...触发判断条件的元素会分到下一个buffer：.bufferUntil(predicate, true)
 - ...满足条件的元素在一个buffer，直到不满足条件的元素发出开始下一buffer：bufferWhile(Predicate)
 - 通过另一个 Publisher 的每一个 onNext 信号来拆分序列：buffer(Publisher), bufferWhen
 - 拆分到指定类型的“collection”：buffer(int, Supplier<C>)
- 我想将 Flux<T> 中具有共同特征的元素分组到子 Flux：groupBy(Function<T,K>)（注意返回值是 Flux<GroupedFlux<K, T>>, 每一个 GroupedFlux 具有相同的 key 值 K, 可以通过 key() 方法获取）。

8) 回到同步的世界

- 我有一个 Flux<T>, 我想：
 - 在拿到第一个元素前阻塞：Flux#blockFirst
 - ...并给出超时时限：Flux#blockFirst(Duration)
 - 在拿到最后一个元素前阻塞（如果序列为空则返回 null）：Flux#blockLast
 - ...并给出超时时限：Flux#blockLast(Duration)
 - 同步地转换为 Iterable<T>：Flux#toIterable
 - 同步地转换为 Java 8 Stream<T>：Flux#toStream

- 我有一个 `Mono<T>`，我想：
 - 在拿到元素前阻塞：`Mono#block`
 - ...并给出超时时限：`Mono#block(Duration)`
 - 转换为 `CompletableFuture<T>`：`Mono#toFuture`

处理错误

在响应式流中，错误（error）是终止（terminal）事件。

当有错误发生时，它会导致流序列停止，并且错误信号会沿着操作链条向下传递，直至遇到定义的 Subscriber 及其 `onError` 方法。

在 try-catch 代码块中处理异常的几种方法。常见的包括如下几种：

1. 捕获并返回一个静态的缺省值。
2. 捕获并执行一个异常处理方法。
3. 捕获并动态计算一个候补值来顶替。
4. 捕获，并再包装为某一个业务相关的异常，然后再抛出业务异常。
5. 捕获，记录错误日志，然后继续抛出。
6. 使用 `finally` 来清理资源

以上所有这些在 Reactor 都有等效的 操作符处理方式。

与第 (1) 条（捕获并返回一个静态的缺省值）对应的是 `onErrorReturn`：

```
Flux.just(10)
    .flatMap(this::function)
    .onErrorReturn("Error");//返回一个静态的缺省值
```

根据错误类型返回对应值

```
Flux.just(10)
    .flatMap(this::function)
    .onErrorReturn(e -> e.getMessage().equals("boom-1"), "Error-1");
```

与第 (2、3、4) 条（捕获并执行一个异常处理方法）对应的是 `onErrorResume`

```
Flux.just(10)
    .flatMap(m -> function(k).onErrorResume(e -> handleErr(k)));

Flux.just(10)
    .flatMap(m -> function(k).onErrorResume(e -> errFunc(k)));

Flux.just(10)
    .flatMap(m -> function(k)
        .onErrorResume(e -> Flux.error(new Exception(k)))
    );
```

对应第 (5) 条（捕获，记录错误日志，并继续抛出）

```
Flux.just(10)
    .flatMap(k -> function(k))
    .doOnError(e ->
        log.error(e.getLocalizedMessage(), e);
    );
```

对应 (6) doFinally 在序列终止 (无论是 onComplete、onError 还是取消) 的时候被执行, 并且能够判断是什么类型的终止事件

```
Flux.just(10)
    .doFinally(type -> {
        if (type == SignalType.CANCEL){
            log.info("a log");
        }
    })
```

流的并行 (或者异步) 与多线程

Reactor 被视为与并发无关的。

也就是说, 获得 Flux 或 Mono 并不一定意味着它在专用线程中运行。

取而代之的是, 大多数 Operator 会继续在执行前一个 Operator 的线程中工作。

除非指定, 否则最顶层的 Operator (源) 本身运行在进行 subscribe() 调用的线程上。

先分享一个案例:

在进行 DB 这种耗时的操作时, 我们不想在 IO 线程上执行, 而是在专用的线程池里边执行, 从而使得 IO 线程不会阻塞。

来看看下面的代码:

```
//获得所有的用户
public Mono<ServerResponse> getAllUser(ServerRequest request) {
    return ServerResponse.ok()
        .contentType(MediaType.APPLICATION_JSON)
        .body(userRepository.findAll(), User.class);
}
```

代码中没有使用 subscribeOn 或者 publishOn 操作符, userRepository.findAll() 会使后续的操作符都在 lettuce-1 线程 (IO) 线程中执行, 所以, 并且整个流的执行都是串行的,

我们需要将查询过程改为异步查询。

调度程序(Schedulers)和线程(threads)

Project Reactor 中的调度程序 (Schedulers) 用于指示多线程 (multi-threading)。

某些运算符具有将 Scheduler 作为参数的变体。这些指示操作员在特定的调度程序上执行其部分或全部工作。

Project Reactor 附带了一组预配置的调度程序 (Schedulers), 都可以通过 `Schedulers` 类进行访问:

- `Schedulers.parallel()`: 执行诸如事件循环和回调处理之类的计算工作。
- `Schedulers.immediate()`: 在当前线程中立即执行工作
- `Schedulers.elastic()`: 执行I/O绑定的工作，例如阻塞I/O的异步性能，此调度程序由线程池支持，该线程池将根据需要增长
- `Schedulers.newSingle()`: 在新线程上执行工作
- `Schedulers.fromExecutor()`: 从`java.util.concurrent.Executor`创建调度程序
- `Schedulers.timer()`: 创建或重新使用分辨率为50ms的基于hash-wheel的`TimedScheduler`。

不要将计算调度程序用于I/O。

调度程序可以通过以下几种不同的方式执行发布者：

- 使用利用调度程序的运算符
- 明确地通过将调度程序传递给这样的运算符
- 通过使用 `subscribeOn(Scheduler)`
- 通过使用 `publishOn(Scheduler)`

如果没有其他说明，默认情况下，诸如 `buffer`, `replay`, `skip`, `delay`, `parallel` 等操作符将使用调度程序。

如果需要，所有列出的运算符都允许你传入自定义调度程序。大多数时候都使用默认值是一个好主意。

如果希望在特定的调度程序上执行订阅链，请使用 `subscribeOn()` 运算符。

该代码在未设置调度程序的情况下在主线程上执行：

```
Flux.just("Ben", "Michael", "Mark").flatMap(key -> {
    System.out.println("Map 1: " + key + " (" +
Thread.currentThread().getName() + ")");
    return Flux.just(key);
})
.flatMap(value -> {
    System.out.println("Map 2: " + value + " (" +
Thread.currentThread().getName() + ")");
    return Flux.just(value);
})
.subscribe();
```

该示例打印以下行：

```
Map 1: Ben (main)
Map 2: Ben (main)
Map 1: Michael (main)
Map 2: Michael (main)
Map 1: Mark (main)
Map 2: Mark (main)
```

此示例显示了添加到流中的 `subscribeOn()` 方法（在哪里添加都无所谓）：

```
Flux.just("Ben", "Michael", "Mark").flatMap(key -> {
    System.out.println("Map 1: " + key + " (" +
Thread.currentThread().getName() + ")");
    return Flux.just(key);
})
.flatMap(value -> {
    System.out.println("Map 2: " + value + " (" +
Thread.currentThread().getName() + ")");
    return Flux.just(value);
})
.subscribeOn(Schedulers.parallel()).subscribe();
```

该示例的输出显示了 `subscribeOn()` 的效果。

你可以看到Publisher在同一线程上执行，但在计算线程池上执行：

```
Map 1: Ben (parallel-1)
Map 2: Ben (parallel-1)
Map 1: Michael (parallel-1)
Map 2: Michael (parallel-1)
Map 1: Mark (parallel-1)
Map 2: Mark (parallel-1)
```

如果将相同的代码应用于Lettuce，你将注意到执行第二个 `flatMap()` 的线程有所不同：

```
Flux.just("Ben", "Michael", "Mark").flatMap(key -> {
    System.out.println("Map 1: " + key + " (" + Thread.currentThread().getName()
+ ")");
    return commands.set(key, key);
})
.flatMap(value -> {
    System.out.println("Map 2: " + value + " (" +
Thread.currentThread().getName() + ")");
    return Flux.just(value);
})
.subscribeOn(Schedulers.parallel()).subscribe();
```

该示例打印以下行：

```
Map 1: Ben (parallel-1)
Map 1: Michael (parallel-1)
Map 1: Mark (parallel-1)
Map 2: OK (lettuce-nioEventLoop-3-1)
Map 2: OK (lettuce-nioEventLoop-3-1)
Map 2: OK (lettuce-nioEventLoop-3-1)
```

与独立示例有两点不同：

- 这些值是同时设置的，而不是顺序设置的
- 第二个 `flatMap()` 转换输出netty EventLoop线程名称这是因为默认情况下，Lettuce发布者是在netty EventLoop线程上执行和完成的。

`publishOn` 指示发布者在特定的Scheduler上调用其观察者的 `nNext`，`onError` 和 `onCompleted` 方法。

在这里，顺序很重要：

```
Flux.just("Ben", "Michael", "Mark").flatMap(key -> {
    System.out.println("Map 1: " + key + " (" + Thread.currentThread().getName()
+ ")");
    return commands.set(key, key);
}).publishOn(Schedulers.parallel()).flatMap(value -> {
    System.out.println("Map 2: " + value + " (" +
Thread.currentThread().getName() + ")");
    return Flux.just(value);
}).subscribe();
```

`publishOn()` 调用之前的所有操作均在main中执行，而调度器中的以下所有操作均在其中执行：

```
Map 1: Ben (main)
Map 1: Michael (main)
Map 1: Mark (main)
Map 2: OK (parallel-1)
Map 2: OK (parallel-1)
Map 2: OK (parallel-1)
```

调度程序(Schedulers)允许直接调度操作。有关更多信息，请参考[Project Reactor文档](#)。

两种在流中切换执行上下文（或Scheduler）的方式

Reactor提供了两种在流中切换执行上下文（或Scheduler）的方式：

- `publishOn`
- `subscribeOn`

我们可以通过这两种方式达到异步执行的目的

publishOn :

此运算符影响线程上下文，它下面的链中的其余运算符将在其中执行，直到新出现的`publishOn`。

```
void demo() {
    final Flux<Long> flux = Flux.interval(Duration.of(100, ChronoUnit.MILLIS))
        .map(log())//1
        .publishOn(Schedulers.parallel())//2
        .map(log())//3
        .publishOn(Schedulers.elastic())//4
        .flatMap(logOfMono()); //5
    flux.subscribe();
}
```

上面代码中：

1. 会在调用`flux.subscribe()`的线程上执行
2. 指定后续的运算符执行的上下文
3. 会在`parallel`线程上执行
4. 指定后续的运算符执行的上下文
5. 会在`elastic`线程上执行

subscribeOn :

整个流在指定的Scheduler的Scheduler.Worker上运行，直至出现`publishOn`，`publishOn`后续的操作符由`publishOn`决定执行上下文。

```

void demo() {
    final Flux<Long> flux = Flux.interval(Duration.of(100, ChronoUnit.MILLIS))
        .map(log())//1
        .publishOn(Schedulers.parallel())
        .map(log())//2
        .subscribeOn(Schedulers.elastic())
        .flatMap(logOfMono());
    flux.subscribe();
}

```

上面代码中：

1. 会在elastic线程执行
2. 会在parallel线程上执行，publishOn会“覆盖”subscribeOn的行为

这样我们就可以异步的去执行方法了。

Flux依次发出了1, 2, 3三个元素 无论是subscribeOn、还是publishOn，经过flatMap 都会在parallel-1 线程上执行，也就是说，Flux中的所有元素只是从主线程发出，在另一个线程中执行。

如何实现流的并行

在调用一些阻塞方法时（rpc、redis、io），我们期望每个元素经过flatMap 中时可以运行在不同线程上（串行-> 并发），应该怎么做？

并行的方法

- 1 使用ParallelFlux
将Flux转为ParallelFlux，使用runOn来指明需要的线程池
- 2 内部通过publishOn表明执行上下文
flatMap中调用的方法需要在内部通过publishOn表明执行上下文。
- 3 创建异步流

1 使用ParallelFlux

将Flux转为ParallelFlux，使用runOn来指明需要的线程池

要获得ParallelFlux，可以在Flux 上使用parallel()运算符。

为了告诉ParallelFlux在哪里运行每个元素，必须使用runOn(Scheduler)。

如果并行处理后，您想恢复到“正常”状态 Flux并以顺序方式应用运算符链的其余部分，可以使用sequential()

```

void demo() {
    final Flux<Long> flux = Flux.fromIterable(Lists.newArrayList(3L, 1L, 2L))
        .parallel().runOn(Schedulers.elastic())
        .flatMap(logOfMono())
        .sequential();
    flux.subscribe();
}

```

2 内部通过publishOn表明执行上下文

flatMap中调用的方法需要在内部通过publishOn表明执行上下文。

```
void demo() {
    Flux.fromIterable(Lists.newArrayList(3L, 1L, 2L))
        .flatMap(this::blockFunction)
        .subscribe();
}

//通过publishOn 来表明 blockFunction() 的上下文
Mono<Long> blockFunctionAsync(Long i) {
    return
    Mono.just(i).publishOn(Schedulers.elastic()).flatMap(this::blockFunction);
}

Mono<Long> blockFunction(Long i) {
    Thread.sleep(i * 1000);
    return Mono.just(i);
}
```

3 创建异步流

1. 使用 Mono.fromFuture() 创建流

```
Mono.fromFuture(CompletableFuture.supplyAsync(() -> blockFunction()));
```

Mono.fromFuture() 创建一个Mono， 需要提供一个 CompletableFuture 产生它的值。

CompletableFuture.supplyAsync() 返回一个 CompletableFuture， 它将在 ForkJoinPool.commonPool() 上运行任务， 异步完成。

```
java.util.concurrent.CompletableFuture<T>
@NotNull
@Contract("_->new")
public static <U> CompletableFuture<U> supplyAsync(@NotNull java.util.function.Supplier<U> supplier)

Returns a new CompletableFuture that is asynchronously completed by a task running in the ForkJoinPool.commonPool() with the value
obtained by calling the given Supplier.

Params:
    supplier – a function returning the value to be used to complete the returned CompletableFuture
Type parameters:
    <U> – the function's return type
Returns:
    the new CompletableFuture
Inferred annotations: Method supplyAsync: @org.jetbrains.annotations.NotNull @org.jetbrains.annotations.Contract("_->new")
Parameter supplier: @org.jetbrains.annotations.NotNull
```

ForkJoinPool [一个全局线程池](#)， 主要应用于计算密集型的场景。

2. 使用Mono.create()、Flux.create() 方法创建流

```
//自定义线程池
final Executor executor = new ThreadPoolExecutor(...);

//创建Mono、Flux时 指明上下文
Mono.create(sink -> executor.execute(() -> {
    sink.success(blockFunction());
}));

Mono<User> userMono = Mono.create(sink -> bizPool.execute(() -> {
    sink.success(jpaEntityService.selectOne(dto.getUserId()));
}));
```

目前我们推荐使用 这种方法来创建Mono、Flux来实现异步

这样做的好处：

1. Reactor对新手来说有一定理解成本，在调用一个返回值为 Publisher 的类型的的方法的时候，不点进去看看，无法知道这是同步方法还是异步响应式方法，这样显示开发人员知道这是一个 响应式方法，避免对于调用者造成的心智负担。
2. 更方便得使用自定义线程池

以上3中流并行方法，如何选择？

建议查询业务场景 采取合适的方法：

方法1：

ParallelFlux 不能保证上游元素 返回顺序，不满足多个 sql 有序执行，或者 上游 元素有序执行的 场景。

方法2：

Reactor对新手来说有一定理解成本，在调用一个返回值为 Publisher 的类型的的方法时，不点进去无法知道这是同步方法还是异步方法，对于调用者存在心智负担。

方法3.1：

无法指定自定义线程池

方法3.2：

这样做的好处：

1. Reactor对新手来说有一定理解成本，在调用一个返回值为 Publisher 的类型的的方法的时候，不点进去看看，无法知道这是同步方法还是异步响应式方法，这样显示开发人员知道这是一个 响应式方法，避免对于调用者造成的心智负担。
2. 更方便得使用自定义线程池

响应式的超价值（4次迭代，让我的HttpClient提速100倍）

在尼恩的一个生产项目中，经常需要调用第三方接口。

有一个比较特殊的请求，一次请求，包含 10个 Web 外部服务调用，每个服务调用的预定时间200ms。

为了并行，使用了100个线程的线程池

注意，这里100个线程已经很多了，因为其他的环节也需要很多线程。所以不能太多了。

尼恩和团队小伙伴，经过4次迭代把性能提升了100倍。

接下来看看，尼恩团队是怎么做到的。

第1次迭代：使用 CompletableFuture 完成并发

使用CompletableFuture 并发10个远程api 接口，有两个方法

- 方法1：CompletableFuture 和 CountDownLatch
- 方法2：CompletableFuture.allOf 算子

在代码实现的时候，使用了结合使用CompletableFuture 和 CountdownLatch 进行并发回调

为了大家能体验效果，给大家提供了一段原始代码的参考程序，具体如下：

```
package com.crazymaker.springcloud;

import com.crazymaker.springcloud.common.util.ThreadUtil;
import org.junit.Test;

import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;

public class CoCurrentDemo {

    /**
     * 使用CompletableFuture 和 CountdownLatch 进行并发回调
     */

    @Test
    public void testMuticallBack() {
        CountDownLatch countDownLatch = new CountDownLatch(10);
        //批量异步
        ExecutorService executor = ThreadUtil.getIoIntenseTargetThreadPool();
        long start = System.currentTimeMillis();
        for (int i = 0; i < 10; i++) {
            CompletableFuture<Long> future = CompletableFuture.supplyAsync(() ->
            {
                long tid = ThreadUtil.getCurThreadId();
                try {
                    doRpc(tid); //异步执行 远程调用
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                return tid;
            }, executor);

            future.thenAccept((tid) -> {
                System.out.println("线程" + tid + "结束了");
                countDownLatch.countDown();
            });
        }
        try {
            countDownLatch.await();
            //输出统计结果
            float time = System.currentTimeMillis() - start;

            System.out.println("所有任务已经执行完毕");
            System.out.println("运行的时长为(ms): " + time);

        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```

    }

    private void doRpc(long tid) throws InterruptedException {
        System.out.println("线程" + tid + "开始了,模拟一下远程调用");
        Thread.sleep(200);
    }
}

```

第2次迭代：响应式Flux流进行异步回调改造

在以上示例中，一次请求，触发 10 个 Web 服务调用。

请记住，执行一次WEB调用操作所需的实际工作非常少：

- 生成并触发一个请求
- 等等一段时间，会收到一个响应。

而且，其中有个关键环节非常低性能，

原因是：在请求和响应之间，线程根本没有做任何工作，而且在等待，等待200ms。

如何要让这些线程不等待，该当如何？这是使用异步框架。

使用异步框架发出 Web 请求的核心优势之一，在请求进行中时您不会占用任何线程。

使用异步框架的核心逻辑，如下所以：

```

Flux.range(0,100)
    .subscribeOn(Schedulers.boundedElastic())
    .flatMap(i -> webClientCallApi(i))
    .subscribe(...) //定义结果

```

响应式的本质，是回调。

其原理，具体请参考 尼恩的深度文章：[京东一面：20种异步，你知道几种？含协程](#)

在响应式框架中：

我们的线程，把异步任务加入响应式组件的内部的队列之后，就去干起别的事情了，不需要去死死等 200ms。

so，我们进行了第2次迭代：响应式Flux流进行异步回调改造。

为了大家能体验效果，给大家提供了一段原始代码的参考程序，具体如下：

```

private Flux<Integer> doRpcFlux(int key) {
    return Flux.create(sink -> {
        log.info("模拟远程调用，参数为：" + key);
        ThreadUtil.sleepMilliseconds(100);
        sink.next(key);
        sink.complete();
    });
}

@Test
public void mergeTest5() throws InterruptedException {
    long startTime = LocalTiker.SINGLETON.now();

```

```

        Iterable<Integer> targetKeys = Arrays.asList( 101, 102, 103, 104, 105);
        Flux.fromIterable(targetKeys)
            .publishOn(Schedulers.fromExecutorService(
                ThreadUtil.getMixedTargetThreadPool()))
            .flatMap(key -> doRpcFlux(key))
            .doOnError(ReactorParallelDemo::doOnError)
            .doOnComplete(ReactorParallelDemo::doOnComplete)
            .doFinally(signalType -> log.info("并发执行的时间: " +
                LocalTiker.SINGLETON.gapFrom(startTime)))
            .subscribe(responseData -> log.info(responseData.toString()), e
                -> log.info("error:" + e.getMessage()));
        Thread.sleep(200000);
    }

```

第3次迭代：响应式parallel Flux 流进行并行异步回调改造

第2次迭代响应式Flux流进行异步回调改造，以为十全十美，改造成功。

实际不然，

虽然线程资源得到了有效的利用，但是由于前期对flux流的机制，不是太清楚。所以，性能反而下降了。

flux的内部机制：flux流里边的任务，默认是串行执行的。

所以上面的代码，实际上是让10次调用，串行执行。

怎么回到并行呢？需要使用 Flux parallel流。

下面是一段 使用parallel Flux 的demo:

```

Flux.range(0,100)
    .parallel()
    .runOn(Schedulers.boundedElastic())
    .flatMap(i -> webClientCallApi(i))
    .sequential().collecttoList();

```

so，我们进行了第2次迭代：响应式Flux流进行异步回调改造。

为了大家能体验效果，给大家提供了一段原始代码的参考程序，具体如下：

```

@Test
    public void mergeTest6() throws InterruptedException {
        long startTime = LocalTiker.SINGLETON.now();
        Iterable<Integer> targetKeys = Arrays.asList(100, 101, 102, 103, 104,
            105, 106, 107);
        Flux.fromIterable(targetKeys)
            .parallel()

            .runOn(Schedulers.fromExecutorService(ThreadUtil.getMixedTargetThreadPool()))
            .flatMap(key -> doRpcFlux(key))
            .sequential()
            .doOnError(ReactorParallelDemo::doOnError)

```

```

        .doOnComplete(ReactorParallelDemo::doOnComplete)
        .doFinally(signalType -> log.info("并发执行的时间: " +
LocalTiker.SINGLETON.gapFrom(startTime)))
        .subscribe(i -> log.info("->" + i));

    Thread.sleep(200000);
}

private static void doOnComplete() {
    log.info("并发远程调用异常完成");
}
}

```

运行的结果如下:

```

12:31:34.138 [Biz-1-cpu-2] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - this taskB start
12:31:34.138 [Biz-1-cpu-1] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - this taskA start
12:31:34.227 [main] DEBUG reactor.util.Loggers$LoggerFactory - UsingSlf4j
logging framework
12:31:34.296 [Biz-2-mixed-18] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 模拟远程调用, 参数为: 101
12:31:34.296 [Biz-2-mixed-22] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 模拟远程调用, 参数为: 105
12:31:34.296 [Biz-2-mixed-23] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 模拟远程调用, 参数为: 106
12:31:34.296 [Biz-2-mixed-24] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 模拟远程调用, 参数为: 107
12:31:34.296 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 模拟远程调用, 参数为: 103
12:31:34.296 [Biz-2-mixed-21] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 模拟远程调用, 参数为: 104
12:31:34.296 [Biz-2-mixed-17] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 模拟远程调用, 参数为: 100
12:31:34.296 [Biz-2-mixed-19] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 模拟远程调用, 参数为: 102
12:31:34.398 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 并发执行的结果->103
12:31:34.398 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 并发执行的结果->100
12:31:34.398 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 并发执行的结果->101
12:31:34.398 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 并发执行的结果->102
12:31:34.398 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 并发执行的结果->104
12:31:34.398 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 并发执行的结果->105
12:31:34.398 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 并发执行的结果->106
12:31:34.398 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 并发执行的结果->107
12:31:34.398 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 并发远程调用异常完成
12:31:34.398 [Biz-2-mixed-20] INFO
com.crazymaker.springcloud.user.main.ReactorParallelDemo - 并发执行的时间: 105

```

响应式Flux parallel流进行并行异步回调改造，看上去已经圆满了。

实现了两个效果：

- 并行
- 没有线程阻塞等待

虽然底层IO组件，还是存在线程阻塞，只是这个下一次迭代再说。

至少从业务维度，第三次迭代之后，没有线程阻塞等待。

parallel Flux 流的使用场景

Flux 本质上是将通量元素划分为单独的轨道，元素之间是串行。Flux parallel 模式，就是将通量元素划分为并行的轨道，才是真正并行的。

Flux parallel 并行流，在生产场景，经常用到。

下面是来自尼恩的社群50+群中的一个生产问题：

@架构师尼恩

假设我上游有一个元素，下游需要根据这个元素有多个Task处理，但是需要等多个Task处理完成后才能返回，如果我不用ConnectFlux的话，还有什么其他方法解决吗？

类似于 `Mono.just(1).map(a->{ executeTaskA(a); executeTaskB(a); }).subscribe();`

我这个executeTaskA 和 executeTaskB 是并行操作的，不存在依赖关系

以上这样的生产问题，都可以使用并行流。

parallel Flux 的底层原理

1. parallel FLux 分配上游元素过程是

parallel操作符执行过后，会生成一个新的flux 即parallelFLux。parallelFLux能够接受多个订阅者，他会从原始flux中请求prefetch个元素，将上游传递push过来的元素依次按顺序分配给子订阅者。

parallelFLux分配上游元素过程是：

依次分配给下游每个订阅者，当某个子订阅者请求数已经满足时，将跳过此订阅者继续向下一个子订阅者分配。

他有两个参数 parallelism 和 prefetch，

- parallelism 第一个参数表示划分子流的数量，默认等于cpu核心数，
- prefetch表示向原始流预取的数量。

parallel本质上是包装每个子订阅者，将接受到的数据存入子订阅者的queue中，然后使用线程池处理queue内的数据。

2. parallel操作符后接map/flapMap操作符

map/flapMap操作符会将真实订阅者生成代理，他将上游onNext的值使用map/flapMap操作符进行转换。

在parallelFlux场景下，它会将每个子订阅者包装起来进行转换。

3. parallel后面可以接sequential操作符

sequential 可以将parallelFlux转成普通flux，

他的原理是创建parallel需要的子订阅者，使用子订阅者订阅parallelFlux，然后将数据依次从每个子订阅者中获取到数据，向下游发送。

在sequential操作执行时，数据可能来自多个线程，这里采用的是哪个线程先到则负责将其他子订阅者队列中的值向下发，没获取到锁的线程存入队列直接退出。

允许传递参数，表示每个子订阅者身上能存储的元素最大数。

4.parallel后接runOn操作符

parallel本质上是包装每个子订阅者，将接受到的数据存入子订阅者的queue中，然后使用线程池处理queue内的数据。

runOn处理的逻辑也是和上面sequential类似，使用AtomicInteger限制每个子订阅者同时只会有一个线程在执行，如果当前已经存在一个线程在执行，后来的数据存入queue后会直接退出。

它还可以传递第二个参数表示每个子订阅者预取的数量，这里为什么要预取呢？

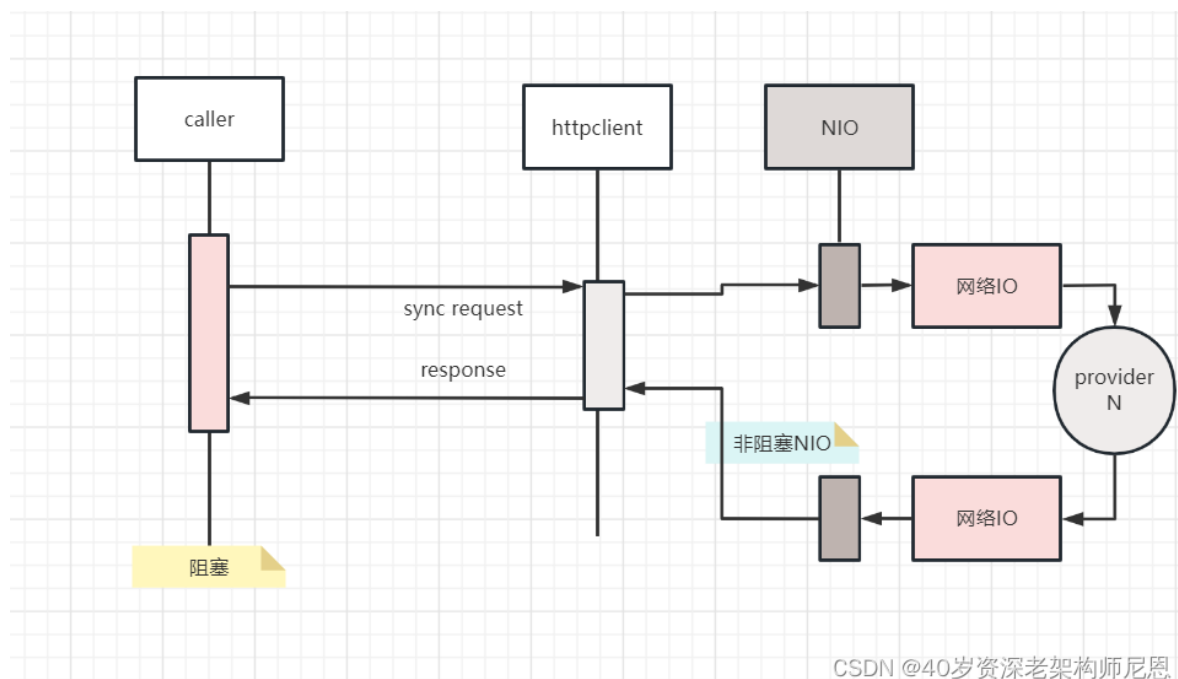
是因为如果下游订阅者请求了非常大的数量，而runOn将任务分配到线程池中本身是非堵塞的，也就是任务全部堆积在线程池中，可能导致内存溢出。所以使用预取参数分批次满足下游的请求数量。

第4次迭代：响应式WebClient组件去掉底层的io线程阻塞

虽然httpClient也支持高性能的nio 模式，但是httpClient 线程模型是阻塞的，

httpClient没有使用Netty这样的reactor反应器线程模型。

底层的io线程是阻塞式的，具体如下图：



如果底层的io传输，也需要使用反应器线程，怎么办？

3个措施：

- 那么要么使用netty,
- 要么使用基于netty的http组件。
- 或者基于netty自己进行封组

这里使用了响应式的http组件， WebClient组件。

使用 WebClient 进行了第四次迭代。为了大家能体验效果，给大家提供了一段原始代码的参考程序，具体如下：

```
@Test
public void mergeTest7() throws InterruptedException {
    long startTime = LocalTiker.SINGLETON.now();
    Iterable<Integer> targetKeys = Arrays.asList( 101, 102, 103, 104, 105);
    Flux.fromIterable(targetKeys)
        .parallel()

    .runOn(Schedulers.fromExecutorService(ThreadUtil.getMixedTargetThreadPool()))
        .flatMap(key -> doRpcUsewebClient(key))
        .sequential()
        .doOnError(ReactorPrallelDemo::doOnError)
        .doOnComplete(ReactorPrallelDemo::doOnComplete)
        .doFinally(signalType -> log.info("并发执行的时间：" +
LocalTiker.SINGLETON.gapFrom(startTime)))
        .subscribe(responseData -> log.info(responseData.toString()), e
-> log.info("error:" + e.getMessage()));
    Thread.sleep(200000);
}

private Flux<RestOut<JSONObject>> doRpcUsewebClient(int key) {

    //响应式客户端
    WebClient client = null;

    WebClient.RequestBodySpec request = null;

    //方式一：极简创建
    //方式二：使用builder创建
    client = WebClient.builder()
        .baseUrl("")
        .defaultHeader(HttpHeaders.CONTENT_TYPE, "application/json")
        .defaultHeader(HttpHeaders.USER_AGENT, "Spring 5 WebClient")
        .build();
    String restUrl = "http://crazy.api/demo/hello/v1?key=" + key;
    /**
     * 是通过 WebClient 组件构建请求
     */
    request = client
        // 请求方法
        .method(HttpMethod.GET)
        // 请求url 和 参数
        .uri(restUrl, params)
        .uri(restUrl)
        // 媒体的类型
        .accept(MediaType.APPLICATION_JSON);
```

```

webClient.ResponseSpec retrieve = request.retrieve();
// 处理异常 请求发出去之后判断一下返回码
retrieve.onStatus(status -> status.value() == 404,
    response -> Mono.just(new RuntimeException("Not Found")));

ParameterizedTypeReference<RestOut<JSONObject>>
parameterizedTypeReference =
    new ParameterizedTypeReference<RestOut<JSONObject>>() {
};
// 返回流
Mono<RestOut<JSONObject>> resp =
retrieve.bodyToMono(parameterizedTypeReference);
return Flux.from(resp);
}

```

让我的HttpClient提速100倍

实际效果，提升了100多倍。

我们从理论上计算一下，这里的4次迭代，为啥性能提升了100多倍？

一次请求，包含 10个 Web 外部服务调用，每个服务调用的预定时间200ms，

为了并行，使用了100个线程的线程池

注意，这里100个线程已经很多了，因为其他的环节，也需要很多线程。

100个线程，同时可以并发完成10个请求，1s可以并发50个请求，吞吐量为 50qps

经过异步改造之后，线程没有任何阻塞，在网卡（千兆）限制范围内 + 和后端的吞吐量保证 下，可以轻松实现5000qps，甚至更多。

最终，性能轻松提速了100倍+。

Webflux 概述

简单来说，Webflux 是响应式编程的框架，与其对等的概念是 SpringMVC。

两者的不同之处在于 Webflux 框架是异步非阻塞的，其可以通过较少的线程处理高并发请求。

Webflux 的框架底层采用了 Reactor响应式编程框架以及 Netty，关于这两部分内容可以参看我之前的3高基础书籍：

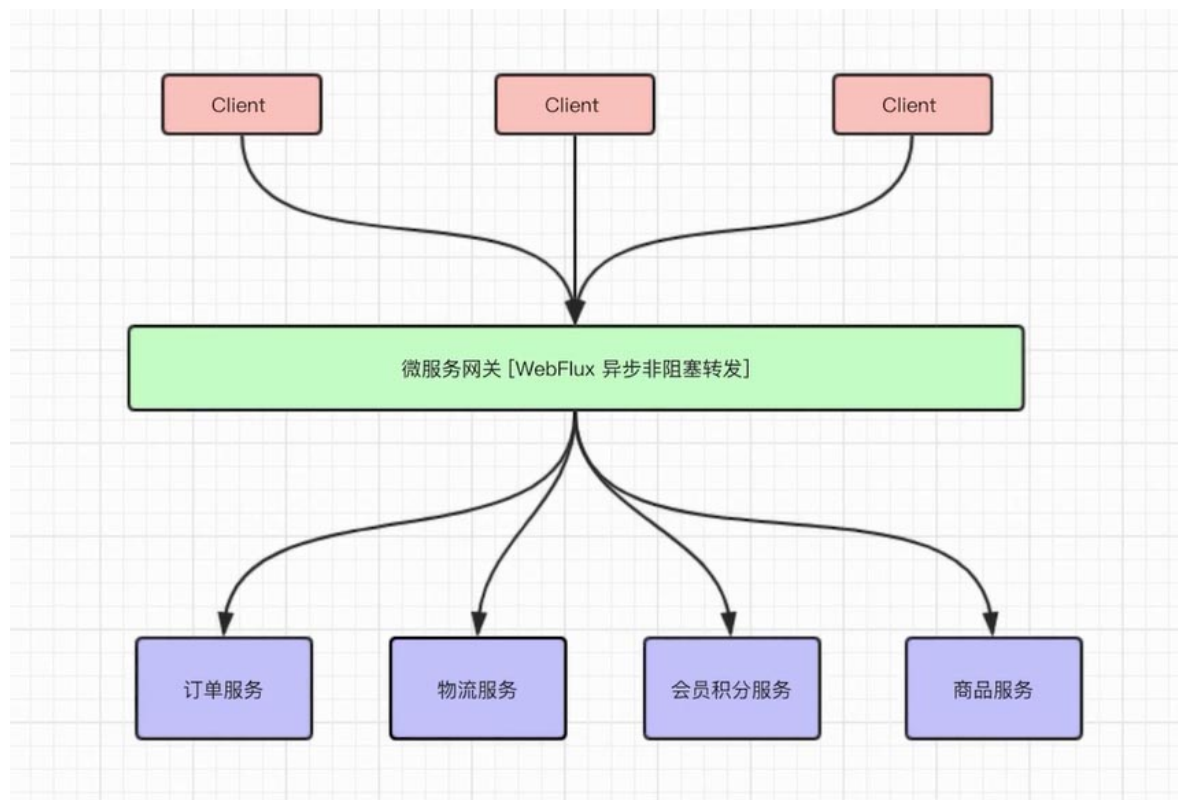
《[Java高并发核心编程 卷1 加强版](#)》

作为一个异步框架来说，必须保证整个程序链中的每一步都是异步操作，如果在某一步出现了同步阻塞（如等待数据库 IO），则整个程序还是回出现阻塞的问题。

webflux应用场景

既适合IO密集型、磁盘IO密集、网络IO密集等服务场景，也适用于高并发、高吞吐业务场景的响应式改造。

比如微服务网关，就可以使用webflux技术来显著的提升网关对下游服务的吞吐量，spring cloud gateway就使用了webflux这门技术



Spring boot webflux介绍

先提一下Spring Boot 2.0

spring.io 官网有句醒目的话是：

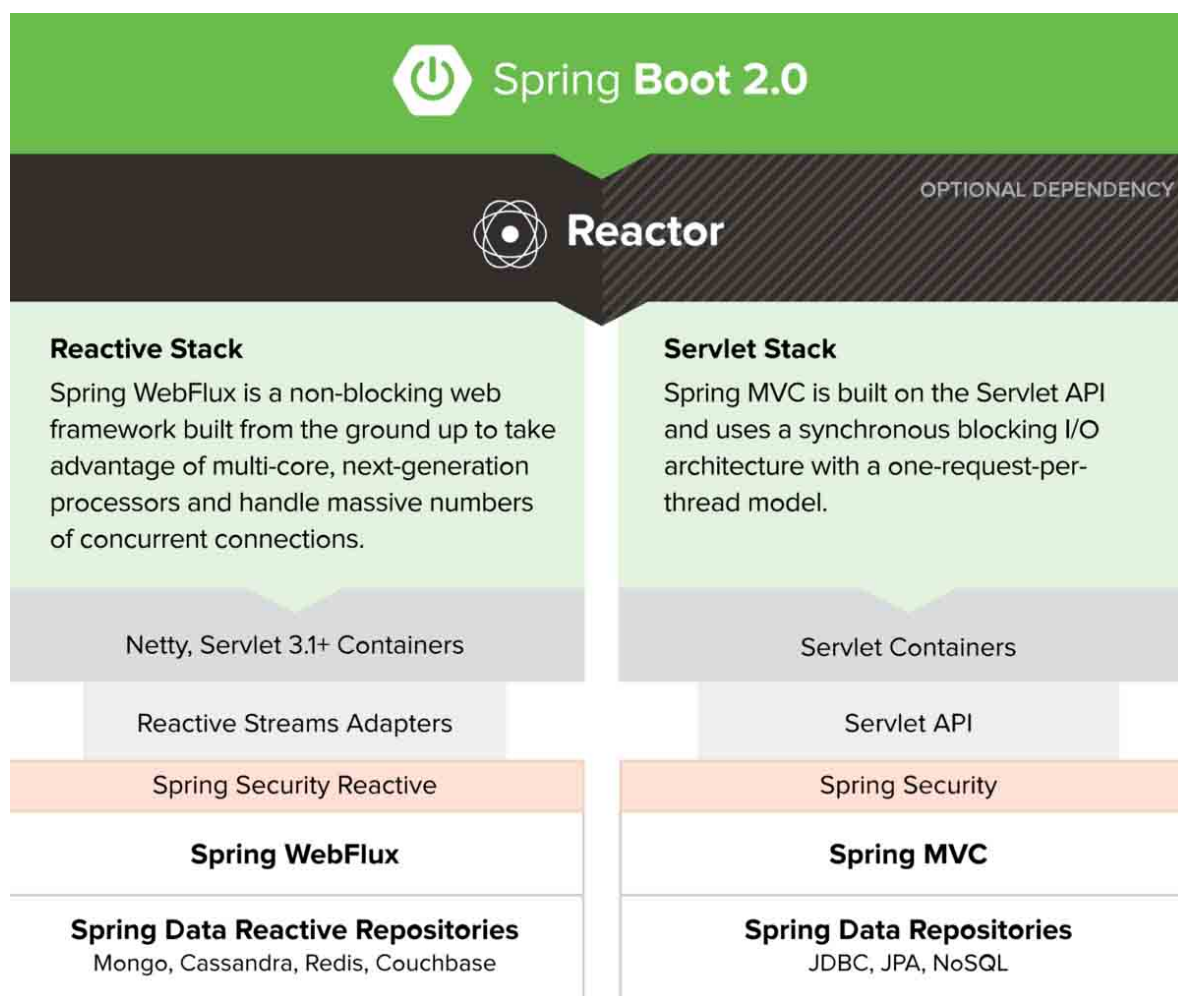
BUILD ANYTHING WITH SPRING BOOT

Spring Boot （Boot 顾名思义，是引导的意思）框架是用于简化 Spring 应用从搭建到开发的过程。

应用开箱即用，只要通过一个指令，包括命令行 `java -jar`、SpringApplication 应用启动类、Spring Boot Maven 插件等，就可以启动应用了。

另外，Spring Boot 强调只需要很少的配置文件，所以在开发生产级 Spring 应用中，让开发变得更加高效和简易。

目前，Spring Boot 2.x 包括 WebFlux。



Spring Boot 2.0 WebFlux

Spring Boot Webflux 就是基于 Reactive Streams 实现的。Spring Boot 2.0 包括一个新的 spring-webflux 模块。

所以，了解 WebFlux，首先了解下什么是 Reactive Streams。

Reactive Streams 是 JVM 中面向流的库标准和规范：

- 处理可能无限数量的元素
- 按顺序处理
- 组件之间异步传递
- 强制性非阻塞背压 (Backpressure)

Backpressure (背压)

背压是一种常用策略，使得发布者拥有无限限制的缓冲区存储元素，用于确保发布者发布元素太快时，不会去压制订阅者。

响应式 API

Reactor 框架是 Reactive Streams 的一个实现框架，也是 Spring Boot Webflux 响应库依赖，通过 Reactor 并与其他响应库交互。

Webflux 提供了两种响应式 API：Mono 和 Flux。

Webflux 的处理流程是：一般是将 Publisher 作为输入，在框架内部转换成 Reactor 类型并处理逻辑，然后返回 Flux 或 Mono 作为输出。

Spring Webflux

Spring Boot Webflux 有两种编程模型实现，一种类似 Spring MVC 注解方式，另一种是使用其功能性端点方式。

Spring Boot 2.0 WebFlux 特性

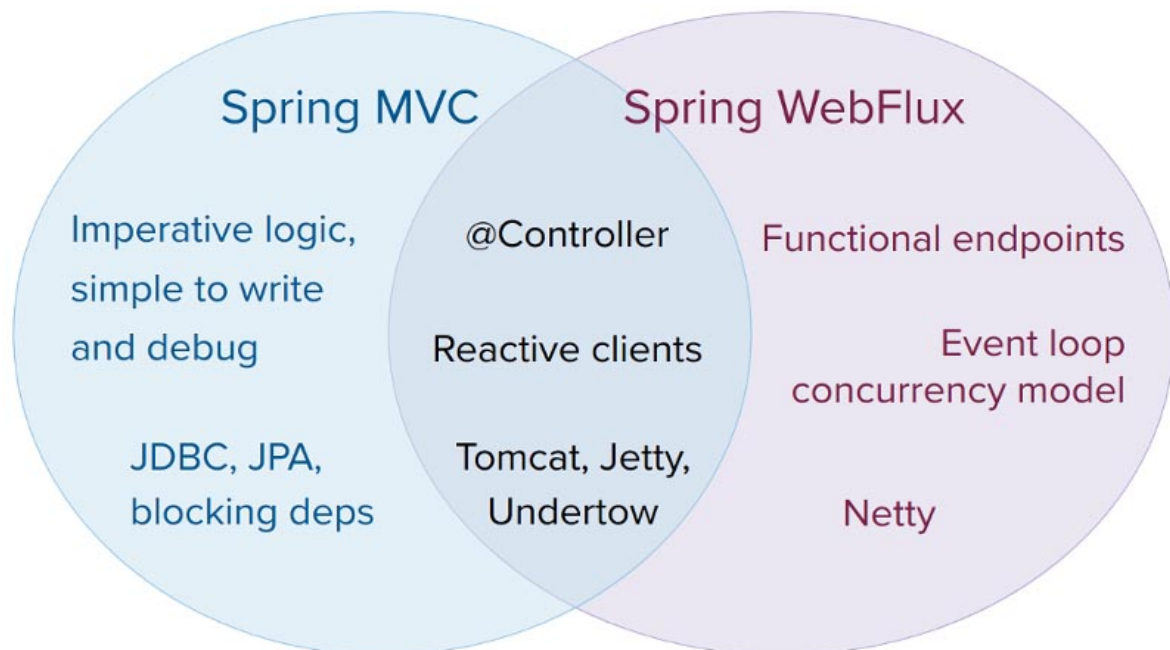
常用的 Spring Boot 2.0 WebFlux 生产的特性如下：

- 响应式 API
- 编程模型
- 适用性
- 内嵌容器
- Starter 组件

还有对日志、Web、消息、测试及扩展等支持。

spring webflux和spring mvc的异同点

WebFlux 和 MVC 的交集



一图就很明确了，WebFlux 和 MVC 有交集，方便大家迁移。但是注意：

- MVC 能满足场景的，就不需要更改为 WebFlux。
- 要注意容器的支持，可以看看下面内嵌容器的支持。
- 微服务体系结构，WebFlux 和 MVC 可以混合使用。尤其开发 IO 密集型服务的时候，选择 WebFlux 去实现。
- spring mvc是一个命令式的编程方式采用同步阻塞方式，方便开发人员编写代码和调试；spring webflux调试会非常不方便
- JDBC连接池和JPA等技术还是阻塞模型，传统的关系型数据库如MySQL也不支持非阻塞的方式获取数据，目前只有非关系型数据库如Redis、Mongodb支持非阻塞方式获取数据

编程模型

Spring 5 web 模块包含了 Spring WebFlux 的 HTTP 抽象。类似 Servlet API , WebFlux 提供了 WebHandler API 去定义非阻塞 API 抽象接口。可以选择以下两种编程模型实现：

- 注解控制层。和 MVC 保持一致，WebFlux 也支持响应性 @RequestBody 注解。

- 功能性端点。基于 lambda 轻量级编程模型，用来路由和处理请求的小工具。和上面最大的区别就是，这种模型，全程控制了请求 - 响应的生命流程

内嵌容器

跟 Spring Boot 大框架一样启动应用，但 WebFlux 默认是通过 Netty 启动，并且自动设置了默认端口为 8080。另外还提供了对 Jetty、Undertow 等容器的支持。

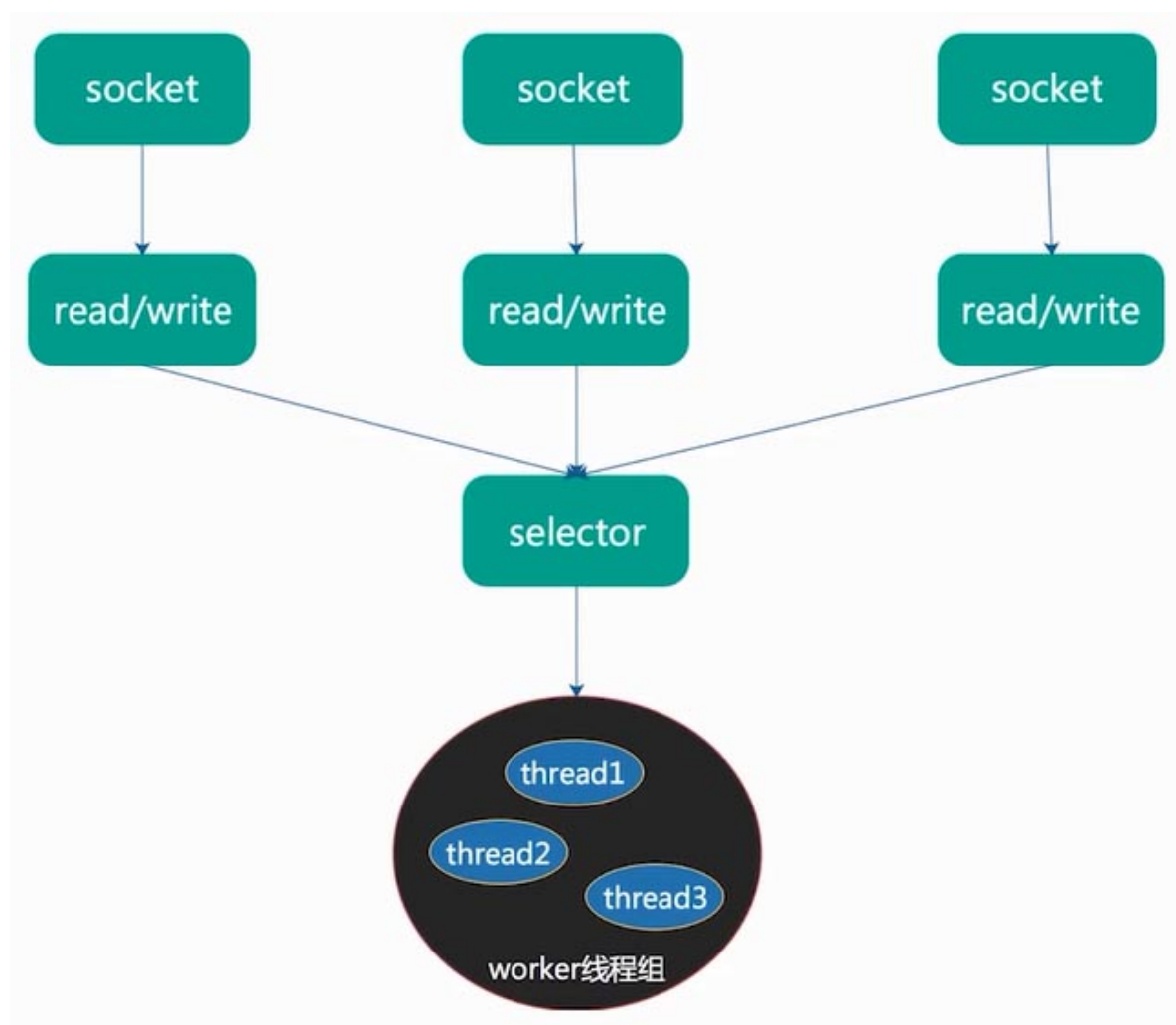
开发者自行在添加对应的容器 Starter 组件依赖，即可配置并使用对应内嵌容器实例。

但是要注意，必须是 Servlet 3.1+ 容器，如 Tomcat、Jetty；或者非 Servlet 容器，如 Netty 和 Undertow。

Netty优点

- API使用简单、易上手
- 功能强大、支持多种主流协议
- 定制能力强、可扩展性高
- 性能高、综合性能最优
- 成熟稳定、久经考验
- 社区活跃、学习资料多

Netty selector模型

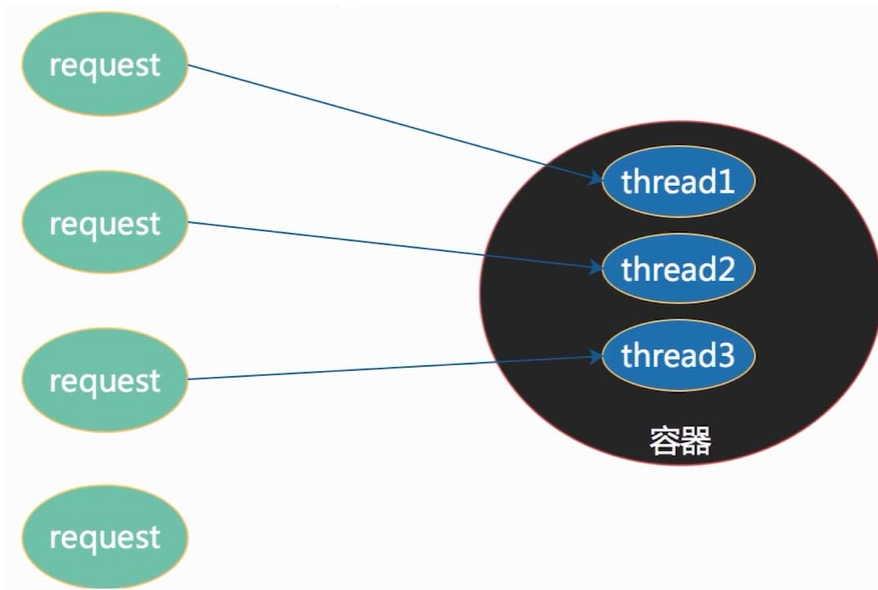


SpringMVC 与Spring WebFlux 的根本区别

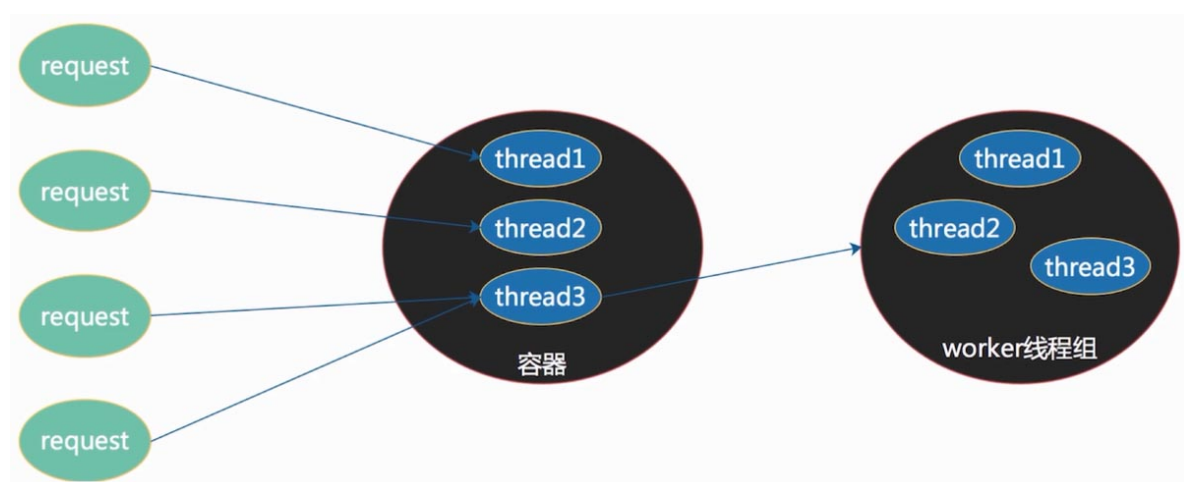
Spring Webflux模块包含对响应式 HTTP 和 WebSocket 客户端的支持，以及对 REST，HTML 和 WebSocket 交互等程序的支持。

一般来说，Spring MVC 用于同步处理，Spring Webflux 用于异步处理。

回顾一下，传统的以SpringMVC为代表的webmvc技术使用的是同步阻塞式IO模型



而Spring WebFlux是一个异步非阻塞式IO模型，可以用少量的容器线程支撑大量的并发访问，所以Spring WebFlux可以提升吞吐量和伸缩性，但是接口的响应时间并不会缩短，其处理结果还是得由worker线程处理完成之后在返回给请求



Webflux 基本使用

首先创建 maven 项目，在项目的 pom 文件中引入相应的依赖

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.7.3</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-r2dbc</artifactId>
  </dependency>
</dependencies>
```

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>dev.miku</groupId>
    <artifactId>r2dbc-mysql</artifactId>
    <version>0.8.2.RELEASE</version>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.24</version>
</dependency>
</dependencies>

```

创建项目的启动类

```

@SpringBootApplication
public class WebfluxDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(WebfluxDemoApplication.class);
    }
}

```

此时，我们就可以编写一个简单的 Controller 来感受一下 Webflux 框架异步相应的概念

```

package com.crazymaker.springcloud.reactive.user.info.controller;

import com.crazymaker.springcloud.common.result.RestOut;
import com.crazymaker.springcloud.common.util.ThreadUtil;
import com.crazymaker.springcloud.reactive.user.info.dto.User;
import com.crazymaker.springcloud.reactive.user.info.service.impl.JpaEntityServiceImpl;
import io.swagger.annotations.Api;
import io.swagger.annotations.ApiImplicitParam;
import io.swagger.annotations.ApiImplicitParams;
import io.swagger.annotations.ApiOperation;
import lombok.extern.slf4j.Slf4j;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import javax.annotation.Resource;
import java.util.concurrent.ExecutorService;
import java.util.function.Supplier;

import static com.crazymaker.springcloud.reactive.user.info.parallel.DBFunctionWrapper.dbFunWrapSafe;

```

```

/**
 * Mono 和 Flux 适用于两个场景，即：
 * Mono: 实现发布者，并返回 0 或 1 个元素，即单对象。
 * Flux: 实现发布者，并返回 N 个元素，即 List 列表对象。
 * 有人会问，这为啥不直接返回对象，比如返回 City/Long/List。
 * 原因是，直接使用 Flux 和 Mono 是非阻塞写法，相当于回调方式。
 * 利用函数式可以减少回调，因此会看不到相关接口。这恰恰是 webFlux 的好处：集合了非阻塞 + 异步
 */
@Slf4j
@Api(value = "用户信息、基础学习DEMO", tags = {"用户信息DEMO"})
@RestController
@RequestMapping("/api/user")
public class UserReactiveController {

    static ExecutorService bizPool = ThreadUtil.getIoIntenseTargetThreadPool();

    @ApiOperation(value = "回显测试", notes = "提示接口使用者注意事项", httpMethod = "GET")
    @RequestMapping(value = "/hello")
    @ApiImplicitParams({
        @ApiImplicitParam(paramType = "query", dataType = "string",
            dataTypeClass = String.class, name = "name", value = "名称", required = true)})
    public Mono<RestOut<String>> hello(@RequestParam(name = "name") String name)
    {
        log.info("方法 hello 被调用了");

        return Mono.just(RestOut.succeed("hello " + name));
    }
}

```

上述代码中，我们定义了一个异步响应的接口，启动程序调用相应接口，可以看到效果

WebFlux和Spring Web MVC 的组件对比

Spring WebFlux不依赖于Servlet API，它可以运行在非Servlet容器Netty、Undertow和任何Servlet 3.1+的Servlet容器（Tomcat, Jetty）之上。

虽然Servlet 3.1提供了非阻碍I/O的API，但是有很多其它的API依然是同步或阻碍式的；这使得Spring需要重新构建完全基于异步和非阻碍式的运行环境。

Spring WebFlux最底层的组件是 `HttpHandler`，它用来适配不同的服务器引擎（Netty、Undertow、Tomcat、Jetty）。

在通过 `HttpHandler` 消除了服务器引擎的异构行后，Spring WebFlux的API设计与Spring MVC是高度一致的。

Spring WebFlux与Spring MVC在概念上有对应的关系：

Spring WebFlux	Spring Web MVC
<code>DispatcherHandler</code>	<code>DispatcherServlet</code>
<code>WebFilter</code>	<code>Filter</code>
<code>HttpMessageWriter</code> <code>HttpMessageReader</code>	<code>HttpMessageConverter</code>
<code>HandlerMapping</code>	<code>HandlerMapping</code>
<code>HandlerAdapter</code>	<code>HandlerAdapter</code>
<code>ServerHttpRequest</code> <code>ServerHttpResponse</code>	<code>ServletRequest</code> <code>ServletResponse</code>

Spring WebFlux支持两种编程模型：

- 注解控制器：和Spring MVC使用的注解式保持一致。
 - `RequestMappingHandlerMapping`：映射请求与 `@RequestMapping` 控制器类和方法；
 - `RequestMappingHandlerAdapter`：调用 `@RequestMapping` 注解的方法。
Spring MVC的两个类与这两个类名称一致，在不同的包里，有不同的实现，但功能保持一致。
- 函数式端点：基于Lambada表达式、轻量级的函数式编程模型。
 - `RouterFunctionMapping`：用来支持 `RouterFunction`；
 - `HandlerFunctionAdapter`：用来支持 `HandlerFunctions`。
 从Spring 5.2或Spring 2.2.x以后Spring MVC也支持这种编程模型。

Spring Boot的自动配置

Spring Boot提供的自动配置主要有：

- `CodecsAutoConfiguration`：Spring WebFlux使用 `HttpMessageReader` 和 `HttpMessageWriter` 接口来转换HTTP请求和返回。本配置类为我们注册了 `CodecCustomizer` 的 Bean，默认使用 `Jackson2JsonEncoder` 和 `Jackson2JsonDecoder`。
- `ReactiveWebServerFactoryAutoConfiguration`：为响应式Web服务器进行自动配置。
- `WebFluxAutoConfiguration`：使用等同于 `@EnableWebFlux` 的配置开启WebFlux的支持。可通过 `WebFluxProperties` 使用 `spring.webflux.*` 来对WebFlux进行配置：

```
spring:
  webflux:
    date-format: yyyy-MM-dd # 日期格式
    static-path-pattern: /resouces/static/** # 静态资源目录
```

- `WebClientAutoConfiguration`：为 `webClient` 进行自动配置。

传统代码转为Reactive的桥梁

Mono/FluxCreate ——传统代码转为Reactive的桥梁

Reactive Programming 作为观察者模式（Observer）的延伸，不同于传统的命令编程方式（Imperative programming）同步拉取数据的方式，如迭代器模式（Iterator）。

而是采用数据发布者同步或异步地推送到数据流（Data Streams）的方案。当该数据流（Data Steams）订阅者监听到传播变化时，立即作出响应动作。

在实现层面上，Reactive Programming 可结合函数式编程简化面向对象语言语法的臃肿性，屏蔽并发实现的复杂细节，提供数据流的有序操作，从而达到提升代码的可读性，以及减少 Bugs 出现的目的。

同时，Reactive Programming 结合背压（Backpressure）的技术解决发布端生成数据的速率高于订阅端消费的问题。

Project Reactor 提供了很多创建Mono/Flux的静态方法，而最常用的就是Mono#create方法，通过该方法能把以前命令式的程序转化为Reactive的编程方式。

众所周知，Reactive Programming是一种Pull-Push模型，其中Pull用于实现back-pressure，

Iterator 属于推模式（push-based），Reactive Flux/Mono 属于拉模式（pull-based）

下面以一个常见的Pull模型迭代器Iterator来说明如何将传统代码转为Reactive的代码。

Iterator 推模式-> Flux 拉模式

```
//创建一个迭代器
Iterator it = Arrays.asList<>(1,2,3).iterator();

//使用迭代器
while(it.hasNext()) {
    //模拟业务逻辑 —— 这里直接打印value
    System.out.println(it.next());
}
```

上面是一个常见的迭代器使用方式，下面看看是如何将迭代器转换成Flux的:

```
@Test
public void fluxExample() throws InterruptedException {
    //创建迭代器
    Iterator it = Arrays.asList(1, 2, 3).iterator();

    Flux<Integer> iteratorFlux = Flux.create(sink -> {
        while (it.hasNext()) {
            Integer data = (Integer) it.next();
            sink.next(data); //利用FluxSink实现data的Push
        }
        sink.complete(); //发送结束的Signal
    });

    //进行订阅，进行业务逻辑操作
    iteratorFlux.log().subscribe(System.out::println);
}
```

输出

```
23:29:15.727 [main] INFO reactor.Flux.Create.1 -  
onSubscribe(FluxCreate.BufferAsyncSink)  
23:29:15.730 [main] INFO reactor.Flux.Create.1 - request(unbounded)  
23:29:20.931 [main] INFO reactor.Flux.Create.1 - onNext(1)  
1  
23:29:20.931 [main] INFO reactor.Flux.Create.1 - onNext(2)  
2  
23:29:20.931 [main] INFO reactor.Flux.Create.1 - onNext(3)  
3  
23:29:20.932 [main] INFO reactor.Flux.Create.1 - onComplete()  
Disconnected from the target VM, address: '127.0.0.1:58005', transport: 'socket'  
  
Process finished with exit code 0
```

MonoCreate常见的两者使用方式

传统命令式编程除了Iterator的Pull模式外，通常还有Observable以及Callback这两种Push模式，下面分别举例讲讲这两种模式。

Observable -> MonoCreate

Observable原始代码举例：

```
observable observable = new Observable() {  
    //需要重写Observable，默认是setChanged与notifyObservers分离，实现先提交再通知的效果  
    //这里为了简单起见，将通知与提交放在了一起  
    @Override  
    public void notifyObservers(Object arg) {  
        setChanged();  
        super.notifyObservers(arg);  
    }  
};  
Observer first = (ob,value) -> {  
    System.out.println("value is " + value);  
};  
observable.addObserver(first);  
observable.notifyObservers("42");  
  
//    after some time, cancel observer to dispose resource  
observable.deleteObserver(first);
```

MonoCreate的转化示例：

```

Mono<Object> observableMono = Mono.create(sink -> {
    observer first = (ob, value) -> {
        sink.success(value);
    };
    observable.addObserver(first);
    observable.notifyObservers("42");
    sink.onDispose(() -> observable.deleteObserver(first));
});
observableMono.subscribe(v -> System.out.println("value is " + v));

```

Callback -> MonoCreate

```

@Test
public void callbackExample() throws InterruptedException {
    ListeningExecutorService service =
        MoreExecutors.listeningDecorator(bizPool);
    ListenableFuture future = service.submit(new Runnable() {
        @Override
        public void run() {
            log.info("mydebug    run, " + Thread.currentThread().getName() );
        }
    });

    Futures.addCallback(future, new FutureCallback() {
        @Override
        public void onSuccess(Object result) {
            log.info("mydebug    onSuccess, " + Thread.currentThread().getName()
        );
        }
        @Override
        public void onFailure(Throwable thrown) {
            log.info("mydebug    onFailure, " + Thread.currentThread() .getName()
        );
        }
    },bizPool);
    Thread.sleep(10000);
}

```

MonoCreate的转化示例:

```

@Data
static class CallbackHandlerInner {
    private MonoSink monoSink;
    private FutureCallback<Object> responseCallback;

    public CallbackHandlerInner(MonoSink monoSink) {
        this.monoSink = monoSink;
        responseCallback = new FutureCallback<Object>() {
            @Override
            public void onSuccess(@Nullable Object o) {
                log.info(" .... 2: FutureCallback    run, " +
                    Thread.currentThread().getName());
                monoSink.success(o);
            }

            @Override

```

```

        public void onFailure(Throwable throwable) {
            monoSink.error(throwable);
        }

    };
}

@Test
public void MonoCreateExample() throws InterruptedException {
    ListeningExecutorService service =
        MoreExecutors.listeningDecorator(bizPool);
    ListenableFuture future = service.submit(new Callable<Object>() {
        @Override
        public Object call() {
            log.info(" .... 1: mydebug    run, " +
                Thread.currentThread().getName());
            return "async out";
        }
    });

    Mono<Object> responseMono = Mono.create(monoSink -> {
        // callback的处理类，并传入monoSink供使用
        CallbackHandlerInner callbackHandler = new
            CallbackHandlerInner(monoSink);

        Futures.addCallback(future, callbackHandler.getResponseCallback(),
            dbPool);

    });
    responseMono.subscribeOn(Schedulers.single())
        .subscribe(out -> log.info(" .... 3: final out:, " + out.toString()));

    ThreadUtil.sleepMilliseconds(1111);
}

```

MonoSink

从前面已经可以看到，将传统代码转变为Reactive方式的关键是在于sink，

在创建Mono/FluxCreate的时候，Mono/Flux都会提供相应的sink给使用方来使用。

MonoSink相比FluxSink要简单的多，为了简单起见，我们从MonoSink来了解sink的运行原理。下面就来探探Mono下的MonoSink究竟到底是什么。

再深入MonoSink之前，我们先来看看MonoCreate是怎么使用MonoSink的，对于Reactor来说，所有的入口都是 `subscribe` 方法，所以先来看看MonoCreate的subscribe方法：

```

public void subscribe(CoreSubscriber<? super T> actual) {
    //1. 创建MonoSink实例，供MonoCreate来使用
    //如变量名字emitter一样，MonoSink的作用其实就是信号的发射器（signal emitter）
    DefaultMonoSink<T> emitter = new DefaultMonoSink<>(actual);

    //2. emitter除了是sink外，也实现了subscription，供Subscriber使用
    //这一步，调用Subscriber的onSubscribe方法，其内部则会调用subscription的request方法
    //（后续会重点说DefaultMonoSink的request方法）
    actual.onSubscribe(emitter);
}

```

```

try {
    //3. callback就是在Mono.create时候传入的Mono构造器
    //此步骤即调用Mono构造器函数，并将sink传入
    callback.accept(emitter);
}
catch (Throwable ex) {
    emitter.error(Operators.onOperatorError(ex, actual.currentContext()));
}
}

```

从上面的源代码可以看出，整个MonoCreate订阅过程很简单，主要是分为三个步骤：

1. 创建DefaultMonoSink (通过这一步可以看出，一个Subscriber是独占一个MonoSink的)
2. 实现Subscriber的onSubscribe的方法
3. 调用Mono#create的构造器函数

以上三个步骤是从整体视角来看的，我们进一步进入DefaultMonoSink，以它的内部视角，来看看到底作为signal emitter的MonoSink做了些什么。

MonoSink 内部状态

MonoSink内部主要有4个状态:

```

volatile int state; //初始默认状态0，即未调用Request且未赋值

static final int NO_REQUEST_HAS_VALUE = 1; //未调用Request但已经赋值
static final int HAS_REQUEST_NO_VALUE = 2; //调用了Request但还未赋值
static final int HAS_REQUEST_HAS_VALUE = 3; //调用了Request且已经赋值了

```

这三个状态主要取决于request和success(或者error)的调用时机，调用了request方法则会是HAS_REQUEST，调用了success(或者error)方法则会是HAS_VALUE，其中request方法调用是由Subscriber#onSubscribe调用的，success或者error则是由具体使用者来调用的，如Callback。

由于success或者error调用时机往往不可能确定（通常是异步的），所以才产生了上述4种状态。

以同步的角度思考，通常是先调用request然后再调用success或者error方法，其中success会对应调用Subscriber的onNext与onComplete方法，error方法则会调用对应的Subscriber#onError方法。但事情往往没这么简单，就如前面提到的，request方法与success/error方法是乱序的，很有可能在request的时候，success/error方法已经调用结束了。

为了解决这个问题，每个方法都引入了for-loop加CAS的多线程操作，变得相对复杂了，但只要知道其内部原理，再复杂的代码看起来就都有线索了，下面以request方法为例，来讲讲是MonoSink是如何解决多线程问题的。

MonoSink request方法解释

```

public void request(long n) {
    if (Operators.validate(n)) {
        LongConsumer consumer = requestConsumer;
        //1. 如果传入了requestConsumer，则调用
        //requestConsumer是通过onRequest方法传入的
        if (consumer != null) {
            consumer.accept(n);
        }
        //2. 进入for loop来实现自旋
        for (; ; ) {

```

```

        int s = state;
        //2.1 HAS_Request: 已经调用过了，直接退出
        if (s == HAS_REQUEST_NO_VALUE || s == HAS_REQUEST_HAS_VALUE) {
            return;
        }
        if (s == NO_REQUEST_HAS_VALUE) {
            // 2.2 double check 是否已经有值
            // 如果是，执行onNext/onComplete方法，并设置完成状态：
            HAS_REQUEST_HAS_VALUE
            // 如果不是，double check失败，直接退出，说明有别的线程已经执行了该方法了
            if (STATE.compareAndSet(this, s, HAS_REQUEST_HAS_VALUE)) {
                try {
                    actual.onNext(value);
                    actual.onComplete();
                }
                finally {
                    //释放资源 - 具体调用的disposable对象由onDisposable方法赋值
                    disposeResource(false);
                }
            }
            return;
        }
        //2.3 正常流程，值没有被赋值，设置为HAS_REQUEST_NO_VALUE
        if (STATE.compareAndSet(this, s, HAS_REQUEST_NO_VALUE)) {
            return;
        }
    }
}
}

```

MonoSink回调方法

MonoSink除了request、success、error方法外，还提供了几个回调函数，以供使用者使用，主要有：

```

//request的时候会被调用，获取request的数量N
MonoSink<T> onRequest(LongConsumer consumer);

//Subscriber调用subscription.cancel是会调用该Disposable方法
MonoSink<T> onCancel(Disposable d);

//与onCancel类似，区别是，除了onCancel方法，在onComplete以及onError也会调用该Disposable方法
MonoSink<T> onDispose(Disposable d);

```

这里简单讲一下Reactor的代码命名规范，对于回调函数都是以onXXX方式命名，注意调用该onXXX方式的时候，**并不是直接调用，而只是传入该回调方法，待对应的事件信号发生时，才会真的被调用。**

这也是声明式编程的一个特色，先声明再执行。

全链路异步化改造，性能提升十倍

结论来自于，尼恩的实操，具体请参考下面的文章

[全链路异步，让你的 SpringCloud 性能优化10倍+](#)

案例分析：微服务迁移到WebFlux - allegro.tech

原文：

<https://blog.allegro.tech/2019/07/migrating-microservice-to-spring-webflux.html>

响应式编程在这几个月内一直是许多会议演讲的热门话题。找到简单的代码示例和教程并将它们应用于allegro新项目是毫不费力的。

当需要从现有解决方案迁移时，特别是它是具有**数百万用户**和**每秒数千个请求**的生产服务时，事情变得有点复杂。

在本文中，我想通过一个Allegro微服务的例子讨论从[Spring Web MVC到 [Spring WebFlux的迁移策略。

我将展示一些常见的陷阱，以及生产中的性能指标如何受迁移的影响。

改变的动机

在详细探讨迁移策略之前，让我们先讨论其变更的动机。

其中一个由我的团队开发和维护的**微服务**，参与了2018年7月18日的重大Allegro停运（详见[尸检](#)）。

虽然我们的微服务不是问题的根本原因，但由于**线程池饱和**，一些实例也崩溃了。

临时修复是：增加线程池大小并减少外部服务调用的超时；然而，这还不够。

临时解决方案仅略微提高了外部服务延迟的吞吐量和弹性。

我们决定转而使用**非阻塞方法**来彻底摆脱线程池作为并发的基础。

使用WebFlux的另一个动机是新项目，它使我们的微服务中的外部服务调用流程变得复杂。

无论复杂程度如何增加，我们都面临着保持代码库可维护性和可读性的挑战。

我们看到WebFlux比我们之前基于Java 8的解决方案（CompletableFuture可以模拟复杂的流程）更加友好。

什么时候迁移？

每一种新兴技术都倾向于其炒作周期。

玩一种新的解决方案，特别是在生产环境中，仅仅因为它新鲜，有光泽和嗡嗡声

尝鲜可能导致沮丧，有时甚至是灾难性的后果。

每个软件供应商都想宣传他的产品，并说服客户使用它。

但是，WebFlux的官方 Pivotal 表现得非常负责任，密切关注那些最好不要迁移到WebFlux的场景。官方文件的[第1.1.4部分](#)详细介绍了这一点。

官方文件最重要的几点是：

- **不要改变工作正常的东西。**

如果您的服务中没有性能问题，或扩展性问题，建议不要迁移，而是**找一个更好的地方来尝试WebFlux**。

- **阻塞式API和WebFlux不是最好的朋友。**

他们可以合作，但把阻塞式API迁移到响应式编程，不会有效率提升。

如果没有迁移恰当，一个阻塞调用可以锁定整个应用程序。

- **团队的学习曲线，特别是如果没有响应性编程的经验，可能会很陡峭。**

你应该在迁移过程中非常注意人为因素。

我们来谈谈性能。

对此有很多误解。反应Reactive并不意味着马上、自动大的提升性能，提升性能还有很多工作要做。WebFlux文档警告我们，以非阻塞方式执行操作需要做更多工作。

响应式闪耀点：等待其他服务响应不会阻塞线程。

因此，获得相同吞吐量所需的线程更少，线程越少意味着使用的内存越少。

始终建议检查独立来源以避免框架作者的偏见。

迁移到WebFlux有四个指标

总而言之，迁移到WebFlux有四个指标如果符合则可行：

- 当前的技术栈没有解决低性能和可**扩展性**的问题。
- 对外部服务或数据库的调用很多，响应速度可能很慢。
- 现有的阻塞依赖项可以很容易地被替换为响应式API。
- 开发团队面临新的挑战并愿意学习。

三阶段迁移策略

根据我们的迁移经验，我想介绍三阶段迁移策略。

为什么3个阶段？

如果我们谈论具有大型代码库的实时服务，每秒数千个请求和数百万用户，从头开始重写是一个相当大的风险。

让我们看看如何在后续的小步骤中将应用程序从Spring Web MVC迁移到Spring WebFlux，从而实现从阻塞到非阻塞世界的平滑过渡。

第1阶段，入门 - 迁移一小段代码

通常，首先在系统的非关键部分尝试新技术是一种很好的做法。

响应式技术也不例外。这个阶段的想法是只要找到一个非关键特性功能，它又是被封装在一个阻塞方法调用中，那就将其重写为非阻塞风格。

让我们看看执行此阻塞方法的示例，该方法用于RestTemplate从外部服务检索结果。

```
Pizza getPizzaBlocking(int id) {
    try {
        return restTemplate.getForObject("http://localhost:8080/pizza/" + id,
            Pizza.class);
    } catch (RestClientException ex) {
        throw new PizzaException(ex);
    }
}
```

我们从丰富的WebFlux功能集中选择一件事 - 反应式WebClient - 并使用它以非阻塞方式重写此方法：

```

Mono<Pizza> getPizzaReactive(int id) {
    return webClient
        .get()
        .uri("http://localhost:8080/pizza/" + id)
        .retrieve()
        .bodyToMono(Pizza.class)
        .onErrorMap(PizzaException::new);
}

```

现在是时候将我们的新方法与应用程序的其余部分连接起来了。

非阻塞方法返回Mono，但我们需要一个普通类型。

我们可以使用Mono.block()方法从中检索值。

```

Pizza getPizzaBlocking(int id) {
    return getPizzaReactive(id).block();
}

```

最终，我们的方法仍在都塞等待。

但是，它内部使用了非阻塞库。

此阶段的主要目标是熟悉非阻塞API。这种更改对应用程序的其余部分是透明的，易于测试并可部署到生产环境中。

第二阶段，主菜 - 将关键路径转换为非阻塞方法

在使用WebClient转换一小段代码后，我们准备更进一步。

第二阶段的目标是将应用程序的关键路径转换为非阻塞 - 从HTTP客户端到处理外部服务响应的类，再到控制器。

在这个阶段，重要的是**避免重写所有代码**。

应用程序中较不重要的部分，例如没有外部调用或很少使用的部分，应该保持不变。

我们需要关注非阻塞方法揭示其优势的领域。

```

//parallel call to two services using Java8 CompletableFuture
Food orderFoodBlocking(int id) {
    try {
        return CompletableFuture.completedFuture(new FoodBuilder())
            .thenCombine(CompletableFuture.supplyAsync(() ->
                pizzaService.getPizzaBlocking(id), executorService), FoodBuilder::withPizza)
            .thenCombine(CompletableFuture.supplyAsync(() ->
                hamburgerService.getHamburgerBlocking(id), executorService),
                FoodBuilder::withHamburger)
            .get()
            .build();
    } catch (ExecutionException | InterruptedException ex) {
        throw new FoodException(ex);
    }
}

//parallel call to two services using Reactor
Mono<Food> orderFoodReactive(int id) {
    return Mono.just(new FoodBuilder())

```

```

        .zipWith(pizzaService.getPizzaReactive(id), FoodBuilder::withPizza)
        .zipWith(hamburgerService.getHamburgerReactive(id),
FoodBuilder::withHamburger)
        .map(FoodBuilder::build)
        .onErrorMap(FoodException::new);
    }

```

使用.subscribeOn()方法，可以轻松地将阻塞部分系统与非阻塞代码合并。

可以使用默认的Reactor调度器之一，或者我们自己创建并提供的线程池ExecutorService。

```

Mono<Pizza> getPizzaReactive(int id) {
    return Mono.fromSupplier(() -> getPizzaBlocking(id))
        .subscribeOn(Schedulers.fromExecutorService(executorService));
}

```

此外，只需对控制器进行少量更改即可 - 将返回类型更改Foo为 Mono<Foo> 或 Flux<Foo>。

它甚至可以在Spring Web MVC中运行 - 您不需要将整个应用程序的堆栈更改为被动。

第2阶段的成功实施为我们提供了非阻塞方法的所有主要优点。是时候测量并检查我们的问题是否已解决。

第3阶段，甜点 - 让我们改变WebFlux的一切！

我们可以在第2阶段之后做更多的事情。

我们可以重写代码中不太关键的部分，并使用Netty服务器而不是servlet。

我们也可以删除@Controller注释并将端点重写为函数风格，尽管这是因为风格和个人偏好去改写，而非性能的问题去改写。

这里的关键问题是：这些优势的成本是多少？

代码可以一直重构，并且通常定义“足够好”的点是很有挑战性的。

在我们的案例中，我们没有决定更进一步。重写整个代码库需要很多工作。

帕累托原则结果证明是有效的一次。

我们认为我们已经取得了显着的收益，而后续的收益也相对较高。

作为一般规则 - 当我们从头开始编写新服务时，获得WebFlux的所有特权是很好的。

另一方面，当我们重构现有（微）服务时，通常最好尽可能少地完成工作。

迁移陷阱以及经验教训

问题1 - 缺乏订阅

WebFlux初学者有时会忘记反应流往往尽可能地会惰加载。

由于缺少订阅，以下功能永远不会向控制台打印任何内容：

```
Food orderFood(int id) {
    FoodBuilder builder = new FoodBuilder().withPizza(new Pizza("margherita"));

    hamburgerService.getHamburgerReactive(id).doOnNext(builder::withHamburger);
    //hamburger will never be set,
    //because Mono returned from getHamburgerReactive() is not subscribed to

    return builder.build();
}
```

教训：每一个Mono和Flux都需要进行订阅。

在控制器中返回的 响应式类型就是一种隐式订阅。

问题2 - .block()在Reactor线程中

正如我之前所展示的（在第1阶段），.block()有时用于将反应函数加入到阻塞代码中。

```
Food getFoodBlocking(int id) {
    return foodService.orderFoodReactive(id).block();
}
```

在Reactor线程中无法调用此函数。这种尝试会导致以下错误：

```
block()/blockFirst()/blockLast() are blocking, which is not supported in thread
reactor-http-nio-2
```

.block()只允许在其他线程中使用显式用法（请参阅参考资料.subscribeOn()）。

Reactor抛出一个异常并告知我们这个问题是有帮助的。

不幸的是，许多其他方案允许将阻塞代码插入到Reactor线程中，这不会自动检测到。

学到的经验教训：

- .block()只能在scheduler中执行的代码中使用。
- 更好的是避免使用.block()。

问题3 - 阻塞Reactor线程中的代码

没有什么能阻止我们将阻塞代码添加到被动流中。

而且，我们不需要使用.block()- 我们可以通过使用可以阻止当前线程的库无意识地引入阻塞。

请考虑以下代码示例。第一个类似于正确的“响应式”延迟。

```
Mono<Food> getFood(int id) {
    return foodService.orderFood(id)
        .delayElement(Duration.ofMillis(1000));
}
```

另一个示例模拟了一个危险的延迟，它阻塞了订户线程。

```
Mono<Food> getFood(int id) throws InterruptedException {  
    return foodService  
        .orderFood(id)  
        .doOnNext(food -> Thread.sleep(1000));  
}
```

一目了然，这两个版本似乎都有效。

当我们在localhost上运行此应用程序并尝试请求服务时，我们可以看到类似的行为。

“Hello, world!”在延迟1秒后返回。

然而，这种观察极具误导性。

实际上，具有反应式延迟（上一段代码）的版本在重负载下运行良好，另一方面，具有阻塞延迟（下一段代码）的版本重负载下运行糟糕。

所以，大家要重视对应用程序进行性能测试，尤其是考虑外部调用的延迟。

问题4 - 意外的代码执行

Reactor有许多有用的方法，有助于编写富有表现力和声明性的代码。

但是，其中一些可能有点棘手。

请考虑以下代码：

```
String valueFromCache = "some non-empty value";  
return Mono.justOrEmpty(valueFromCache)  
    .switchIfEmpty(Mono.just(getValueFromService()));
```

我们使用类似的代码检查特定值的缓存，然后在缺少值时调用外部服务。

作者的意图似乎很明确：getValueFromService() 仅在缺少缓存值的情况下执行。

但是，此代码每次都会运行，即使是缓存命中也是如此。

赋给.switchIfEmpty()的参数不是lambda，而是Mono.just()直接执行作为参数传递的代码。

显而易见的解决方案是使用Mono.fromSupplier()，并将条件代码作为lambda传递，如下例所示：

```
String valueFromCache = "some non-empty value";  
return Mono.justOrEmpty(valueFromCache)  
    .switchIfEmpty(Mono.fromSupplier(() -> getValueFromService()));
```

经验教训：Reactor API有许多不同的方法。

始终考虑一个问题：参数是应该按原样传递，还是用lambda包装。

迁移带来的好处

总结一下，在迁移到WebFlux之后检查我们服务的生产指标。

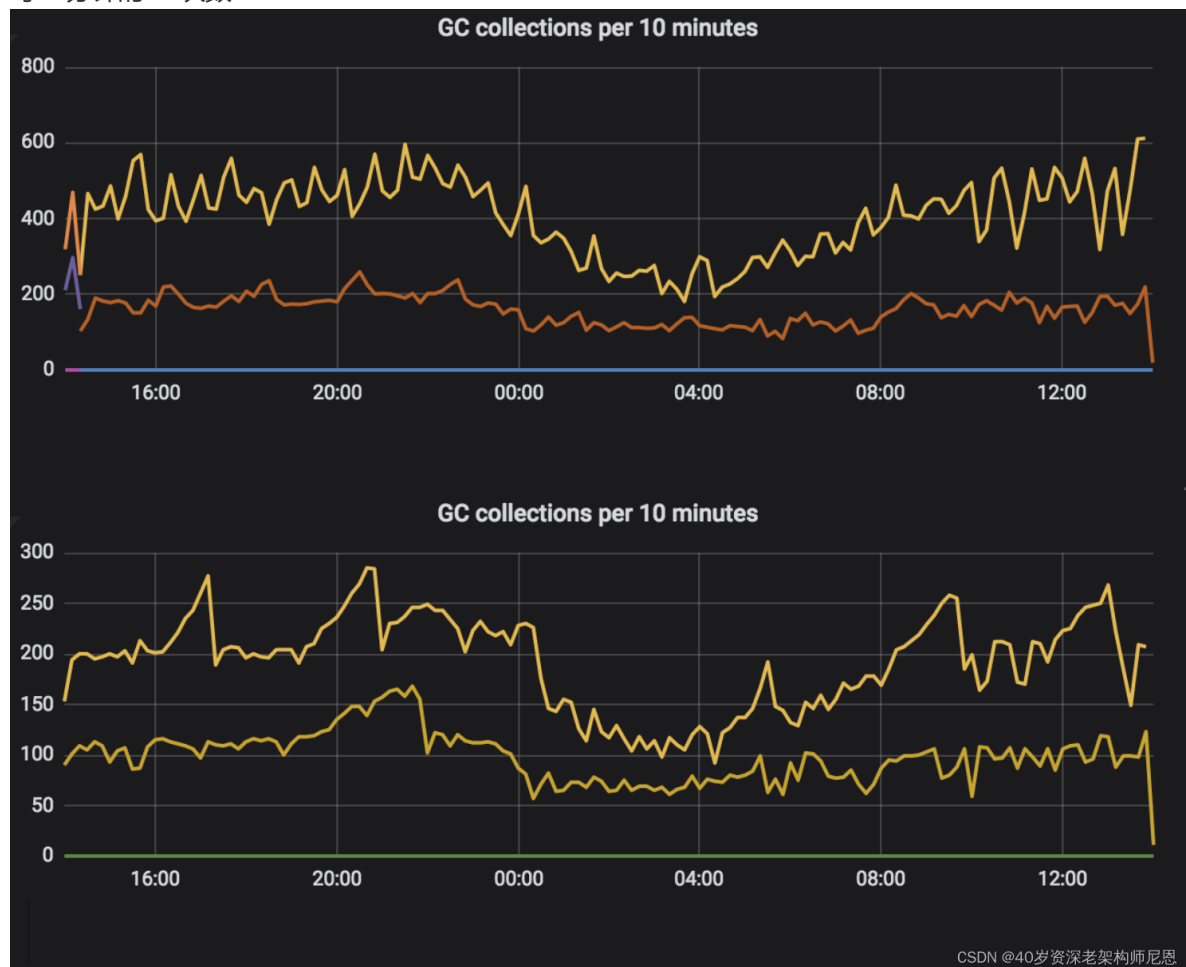
明显而直接的影响是应用程序使用的线程数量减少。

有趣的是，我们没有将应用程序类型更改为Reactive（我们仍然使用servlet，有关详细信息，请参阅第3阶段），但Undertow工作线程的使用也变小了一个数量级。

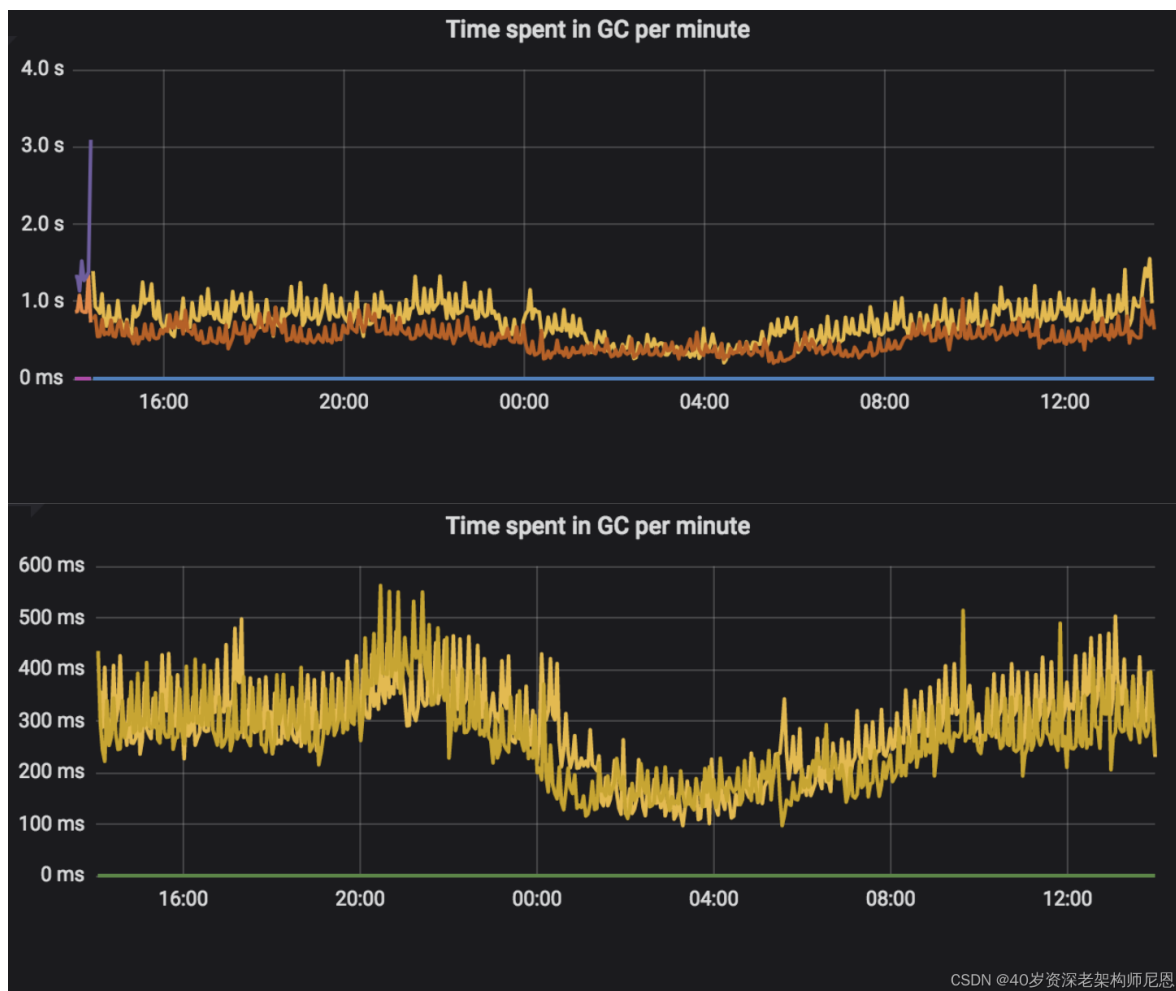
低级指标如何受到影响？

我们观察到更少的垃圾收集，并且他们花费的时间更少。

每10分钟的GC次数



每10分钟的GC耗时



此外，响应时间略有下降，但我们没有预料到这样的效果。

其他指标（如CPU负载，文件描述符使用情况和消耗的总内存）未发生变化。我们的服务也做了很多工作，这与调用无关。将流量迁移到HTTP客户端和控制器周围的响应是至关重要的，但在资源使用方面并不重要。

正如我在开始时所说的那样，**迁移的预期收益是延迟的可扩展性和弹性。我们确信我们已经实现了这一目标。

结论

您是否正在迁移现有的微服务？考虑到文章中涉及的因素，不仅仅是技术因素 - 检查时间和人员使用新解决方案的能力。

始终测试您的应用程序，在迁移过程中，覆盖外部耗时调用的集成和性能测试至关重要。

请记住，响应式思维不同于众所周知的阻塞式、命令式思维。

Lettuce Reactive API

spring默认redis连接库lettuce性能优化，突破性能天花板,获得官方建议方式2倍吞吐量

spring-data-redis性能优化，使用lettuce库连接池模式，比官方推荐方式性能翻倍。

所有命令都返回订阅者可以订阅的 `Flux<T>`，`Mono<T>` 或 `Mono<Void>`。

该订阅者对 `Publisher <T>` 发出的任何项目或项目序列做出反应。

此模式有助于并发操作，因为在等待 `Publisher<T>` 发出对象时不需要阻塞。相反，它以订阅者的形式创建一个哨兵，随时准备在 `Publisher<T>` 以后的任何时间做出适当的反应。

使用Lettuce创建Flux和Mono

建立发布者的方法有很多。

你已经看过 `just()`，`take()` 和 `collectList()`。有关可用于创建 `Flux` 和 `Mono` 的更多方法，请参考[Project Reactor文档](#)。

Lettuce发布者可用于初始和链接（chaining）操作。

使用Lettuce发布者时，你会注意到非阻塞行为。

这是因为所有的I/O和命令处理都是使用netty EventLoop异步处理的。

连接到Redis非常简单：

```
RedisClient client = RedisClient.create("redis://localhost");
RedisStringReactiveCommands<String, String> commands =
    client.connect().reactive();
```

下一步，从键获取值需要GET操作：

```
commands.get("key").subscribe(new Consumer<String>() {

    public void accept(String value) {
        System.out.println(value);
    }

});
```

或者，用Java 8 lambdas编写：

```
commands
    .get("key")
    .subscribe(value -> System.out.println(value));
```

执行是异步处理的，并且在Netty EventLoop线程上完成操作时，可以使用调用线程在处理中进行处理。由于其解耦性质，可以在完成 `Publisher<T>` 的执行之前保留调用方法。

可以在链接（chaining）的上下文中使用Lettuce发布者来异步加载多个键：

```
Flux.just("Ben", "Michael", "Mark")
    .flatMap(key -> commands.get(key))
    .subscribe(value -> System.out.println("Got value: " + value));
```

性能验证

文章已经太长，这里不做赘述，具体请参考[尼恩3高架构笔记](#)

Spring Cloud Gateway的开发

姊妹篇：关于 SpringCloud Gateway 简介

SpringCloud Gateway 是 Spring Cloud 的一个全新项目，该项目是基于 Spring 5.0, Spring Boot 2.0 和 Project Reactor 等技术开发的网关，它旨在为微服务架构提供一种简单有效的统一的 API 路由管理方式。

SpringCloud Gateway 作为 Spring Cloud 生态系统中的网关，目标是替代 Zuul，在Spring Cloud 2.0以上版本中，没有对新版本的Zuul 2.0以上最新高性能版本进行集成，仍然还是使用的Zuul 2.0之前的非Reactor模式的老版本。

而为了提升网关的性能，SpringCloud Gateway是基于WebFlux框架实现的，而WebFlux框架底层则使用了高性能的Reactor模式通信框架Netty。

Spring Cloud Gateway 的目标，不仅提供统一的路由方式，并且基于 Filter 链的方式提供了网关基本的功能，例如：安全，监控/指标，和限流。

有关 Spring Cloud Gateway 实战，具体请参考尼恩的 深度文章：

[《SpringCloud gateway（史上最全）》](#)

特别说明：

Spring Cloud Gateway 底层使用了高性能的通信框架Netty。

Netty 是高性能中间件的通讯底座，rocketmq、seata、nacos、sentinel、redisson、dubbo 等太多、太多的的大名鼎鼎的中间件，无一例外都是基于netty。

可以毫不夸张的说：**netty 是进入大厂、走向高端 的必备技能。**

要想深入了解springcloud gateway，**最好是掌握netty 编程。**

有关 netty学习 具体请参见机工社出版、尼恩的畅销书：[《Java高并发核心编程卷 1》](#)

ServerWebExchange交换机

理解ServerWebExchange

先看 ServerWebExchange 的注释：

Contract for an HTTP request-response interaction.

Provides access to the HTTP request and response and also exposes additional server-side processing related properties and features such as request attributes.

翻译一下大概是：

ServerWebExchange是一个HTTP请求-响应交互的契约。

提供对HTTP请求和响应的访问，并公开额外的服务器端处理相关属性和特性，如请求属性。

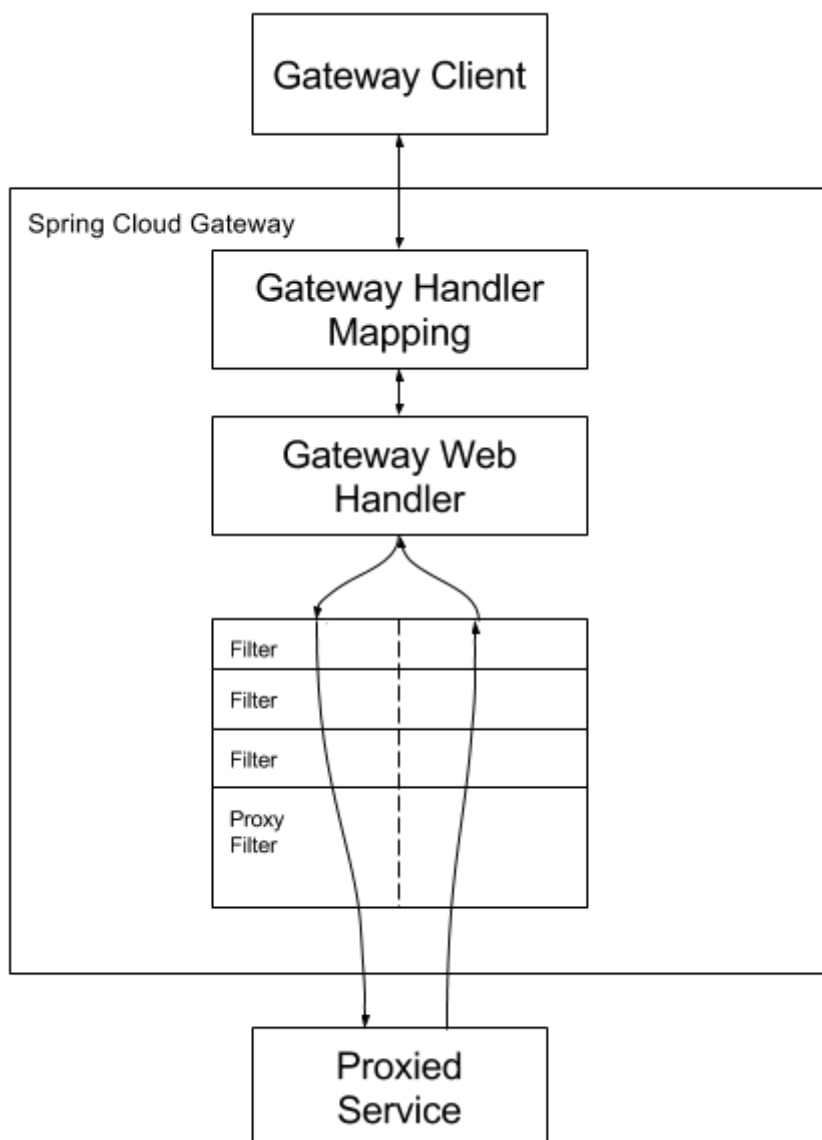
其实，`ServerWebExchange` 命名为**服务网络交换器**，存放着重要的请求-响应属性、请求实例和响应实例等等，有点像 `Context` 的角色。

ServerWebExchange 与过滤器的关系：

Spring Cloud Gateway同zuul类似，有“pre”和“post”两种方式的filter。

客户端的请求先经过“pre”类型的filter，然后将请求转发到具体的业务服务，收到业务服务的响应之后，再经过“post”类型的filter处理，最后返回响应到客户端。

引用Spring Cloud Gateway官网上的一张图：



与zuul不同的是，filter除了分为“pre”和“post”两种方式的filter外，在Spring Cloud Gateway中，filter从作用范围可分为另外两种，

一种是**针对于单个路由的gateway filter**，它在配置文件中的写法同predict类似；

一种是**针对于所有路由的global gateway filter**。

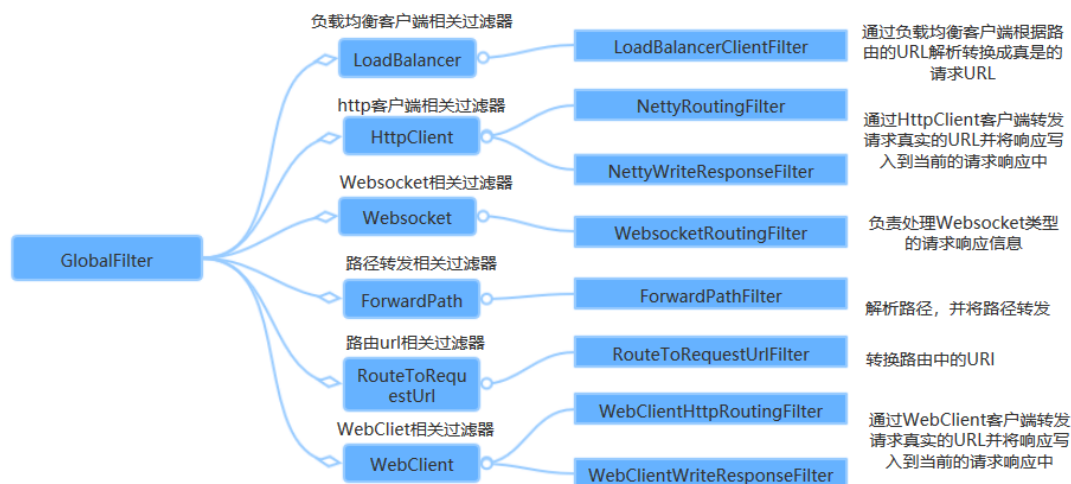
现在从作用范围划分的维度来讲解这两种filter。

我们在使用 Spring Cloud Gateway 的时候，注意到过滤器（包括 `GatewayFilter`、`GlobalFilter` 和过滤器链 `GatewayFilterChain`）。

Spring Cloud Gateway根据作用范围划分为GatewayFilter和GlobalFilter，二者区别如下：

- **GatewayFilter**：需要通过spring.cloud.routes.filters 配置在具体路由下，只作用在当前路由上或通过spring.cloud.default-filters配置在全局，作用在所有路由上
- **GlobalFilter**：全局过滤器，不需要在配置文件中配置，作用在所有的路由上，最终通过GatewayFilterAdapter包装成GatewayFilterChain可识别的过滤器，它为请求业务以及路由的URI转换为真实业务服务的请求地址的核心过滤器，不需要配置，系统初始化时加载，并作用在每个路由上。

Spring Cloud Gateway框架内置的GlobalFilter如下：



上图中每一个GlobalFilter都作用在每一个router上，能够满足大多数的需求。

但是如果遇到业务上的定制，可能需要编写满足自己需求的GlobalFilter。

过滤器都依赖到 `ServerWebExchange`：

```

public interface GlobalFilter {

    Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain);
}

public interface GatewayFilter extends ShortcutConfigurable {

    Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain);
}

public interface GatewayFilterChain {

    Mono<Void> filter(ServerWebExchange exchange);
}
  
```

这里的设计和 `Servlet` 中的 `Filter` 是相似的，

当前过滤器可以决定是否执行下一个过滤器的逻辑，由 `GatewayFilterChain#filter()` 是否被调用来决定。

而 `ServerWebExchange` 就相当于当前请求和响应的上下文。

`ServerWebExchange` 实例不单存储了 `Request` 和 `Response` 对象，还提供了一些扩展方法，如果想实现改造请求参数或者响应参数，就必须深入了解 `ServerWebExchange`。

ServerWebExchange接口

`ServerWebExchange` 接口的所有方法：

```

public interface ServerWebExchange {

    // 日志前缀属性的KEY，值为
    org.springframework.web.server.ServerWebExchange.LOG_ID
  }
  
```

```

// 可以理解为
attributes.set("org.springframework.web.server.ServerWebExchange.LOG_ID", "日志前缀的具体值");

// 作用是打印日志的时候会拼接这个KEY对饮的前缀值，默认值为""
String LOG_ID_ATTRIBUTE = ServerWebExchange.class.getName() + ".LOG_ID";
String getLogPrefix();

// 获取ServerHttpRequest对象
ServerHttpRequest getRequest();

// 获取ServerHttpResponse对象
ServerHttpResponse getResponse();

// 返回当前exchange的请求属性，返回结果是一个可变的Map
Map<String, Object> getAttributes();

// 根据KEY获取请求属性
@Nullable
default <T> T getAttribute(String name) {
    return (T) getAttributes().get(name);
}

// 根据KEY获取请求属性，做了非空判断
@SuppressWarnings("unchecked")
default <T> T getRequiredAttribute(String name) {
    T value = getAttribute(name);
    Assert.notNull(value, () -> "Required attribute '" + name + "' is missing");
    return value;
}

// 根据KEY获取请求属性，需要提供默认值
@SuppressWarnings("unchecked")
default <T> T getAttributeOrDefault(String name, T defaultValue) {
    return (T) getAttributes().getOrDefault(name, defaultValue);
}

// 返回当前请求的网络会话
Mono<WebSession> getSession();

// 返回当前请求的认证用户，如果存在的话
<T extends Principal> Mono<T> getPrincipal();

// 返回请求的表单数据或者一个空的Map，只有Content-Type为application/x-www-form-urlencoded的时候这个方法才会返回一个非空的Map -- 这个一般是表单数据提交用到
Mono<Multimap<String, String>> getFormData();

// 返回multipart请求的part数据或者一个空的Map，只有Content-Type为multipart/form-data的时候这个方法才会返回一个非空的Map -- 这个一般是文件上传用到
Mono<Multimap<String, Part>> getMultipartData();

// 返回Spring的上下文
@Nullable
ApplicationContext getApplicationContext();

// 这几个方法和lastModified属性相关
boolean isNotModified();
boolean checkNotModified(Instant lastModified);

```

```

boolean checkNotModified(String etag);
boolean checkNotModified(@Nullable String etag, Instant lastModified);

// URL转换
String transformUrl(String url);

// URL转换映射
void addUrlTransformer(Function<String, String> transformer);

// 注意这个方法，方法名是：改变，这个是修改ServerWebExchange属性的方法，返回的是一个
Builder实例，Builder是ServerWebExchange的内部类
default Builder mutate() {
    return new DefaultServerWebExchangeBuilder(this);
}

interface Builder {

    // 覆盖ServerHttpRequest
    Builder request(Consumer<ServerHttpRequest.Builder>
requestBuilderConsumer);
    Builder request(ServerHttpRequest request);

    // 覆盖ServerHttpResponse
    Builder response(ServerHttpResponse response);

    // 覆盖当前请求的认证用户
    Builder principal(Mono<Principal> principalMono);

    // 构建新的ServerWebExchange实例
    ServerWebExchange build();
}
}

```

ServerWebExchange#mutate() 方法

注意到 `ServerWebExchange#mutate()` 方法，`ServerWebExchange` 实例可以理解为不可变实例，如果我们想要修改它，需要通过 `mutate()` 方法生成一个新的实例，例如这样：

```

public class CustomGlobalFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
chain) {
        ServerHttpRequest request = exchange.getRequest();
        // 这里可以修改ServerHttpRequest实例
        ServerHttpRequest newRequest = ...
        ServerHttpResponse response = exchange.getResponse();
        // 这里可以修改ServerHttpResponse实例
        ServerHttpResponse newResponse = ...
        // 构建新的ServerWebExchange实例
        ServerWebExchange newExchange =
exchange.mutate().request(newRequest).response(newResponse).build();
        return chain.filter(newExchange);
    }
}

```

ServerHttpRequest接口

`ServerHttpRequest` 实例是用于承载请求相关的属性和请求体，

`Spring Cloud Gateway` 中底层使用 `Netty` 处理网络请求，通过追溯源码，

可以从 `ReactorHttpHandlerAdapter` 中得知 `ServerWebExchange` 实例中持有的

`ServerHttpRequest` 实例的具体实现是 `ReactorServerHttpRequest`。

之所以列出这些实例之间的关系，是因为这样比较容易理清一些隐含的问题，例如：

- `ReactorServerHttpRequest` 的父类 `AbstractServerHttpRequest` 中初始化内部属性 `headers` 的时候把请求的HTTP头部封装为只读的实例：

```
public AbstractServerHttpRequest(URI uri, @Nullable String contextPath,
HttpHeaders headers) {
    this.uri = uri;
    this.path = RequestPath.parse(uri, contextPath);
    this.headers = HttpHeaders.readOnlyHttpHeaders(headers);
}

// HttpHeaders类中的readOnlyHttpHeaders方法，
// ReadOnlyHttpHeaders屏蔽了所有修改请求头的方法，直接抛出
// UnsupportedOperationException
public static HttpHeaders readOnlyHttpHeaders(HttpHeaders headers) {
    Assert.notNull(headers, "HttpHeaders must not be null");
    if (headers instanceof ReadOnlyHttpHeaders) {
        return headers;
    }
    else {
        return new ReadOnlyHttpHeaders(headers);
    }
}
```

所以，不能直接从 `ServerHttpRequest` 实例中直接获取请求头 `HttpHeaders` 实例并且进行修改。

`ServerHttpRequest` 接口如下：

```
public interface HttpMessage {

    // 获取请求头，目前的实现中返回的是ReadOnlyHttpHeaders实例，只读
    HttpHeaders getHeaders();
}

public interface ReactiveHttpInputMessage extends HttpMessage {

    // 返回请求体的Flux封装
    Flux<DataBuffer> getBody();
}

public interface HttpRequest extends HttpMessage {

    // 返回HTTP请求方法，解析为HttpMethod实例
    @Nullable
    default HttpMethod getMethod() {
        return HttpMethod.resolve(getMethodValue());
    }
}
```

```

// 返回HTTP请求方法，字符串
String getMethodValue();

// 请求的URI
URI getURI();
}

public interface ServerHttpRequest extends HttpRequest, ReactiveHttpInputMessage
{

    // 连接的唯一标识或者用于日志处理标识
    String getId();

    // 获取请求路径，封装为RequestPath对象
    RequestPath getPath();

    // 返回查询参数，是只读的MultiValueMap实例
    MultiValueMap<String, String> getQueryParams();

    // 返回Cookie集合，是只读的MultiValueMap实例
    MultiValueMap<String, HttpCookie> getCookies();

    // 远程服务器地址信息
    @Nullable
    default InetSocketAddress getRemoteAddress() {
        return null;
    }

    // SSL会话实现的相关信息
    @Nullable
    default SslInfo getSslInfo() {
        return null;
    }

    // 修改请求的方法，返回一个建造器实例Builder，Builder是内部类
    default ServerHttpRequest.Builder mutate() {
        return new DefaultServerHttpRequestBuilder(this);
    }

    interface Builder {

        // 覆盖请求方法
        Builder method(HttpMethod httpMethod);

        // 覆盖请求的URI、请求路径或者上下文，这三者相互有制约关系，具体可以参考API注释
        Builder uri(URI uri);
        Builder path(String path);
        Builder contextPath(String contextPath);

        // 覆盖请求头
        Builder header(String key, String value);
        Builder headers(Consumer<HttpHeaders> headersConsumer);

        // 覆盖SslInfo
        Builder sslInfo(SslInfo sslInfo);

        // 构建一个新的ServerHttpRequest实例
        ServerHttpRequest build();
    }
}

```

```
}  
}
```

注意:

`ServerHttpRequest` 或者说 `HttpMessage` 接口提供的获取请求头方法 `HttpHeaders getHeaders()`;

返回结果是一个只读的实例，具体是 `ReadOnlyHttpHeaders` 类型，

如果要修改 `ServerHttpRequest` 实例，那么需要这样做：

```
ServerHttpRequest request = exchange.getRequest();  
ServerHttpRequest newRequest =  
request.mutate().header("key", "value").path("/myPath").build();
```

ServerHttpResponse接口

`ServerHttpResponse` 实例是用于承载响应相关的属性和响应体，

`Spring Cloud Gateway` 中底层使用 `Netty` 处理网络请求，通过追溯源码，可以从

`ReactorHandlerAdapter` 中得知 `ServerWebExchange` 实例中持有的 `ServerHttpResponse` 实例的具体实现是 `ReactorServerHttpResponse`。

之所以列出这些实例之间的关系，是因为这样比较容易理清一些隐含的问题，例如：

```
// ReactorServerHttpResponse的父类  
public AbstractServerHttpResponse(DataBufferFactory dataBufferFactory,  
HttpHeaders headers) {  
    Assert.notNull(dataBufferFactory, "DataBufferFactory must not be null");  
    Assert.notNull(headers, "HttpHeaders must not be null");  
    this.dataBufferFactory = dataBufferFactory;  
    this.headers = headers;  
    this.cookies = new LinkedMultiValueMap<>();  
}  
  
public ReactorServerHttpResponse(HttpServerResponse response, DataBufferFactory  
bufferFactory) {  
    super(bufferFactory, new HttpHeaders(new  
NettyHeadersAdapter(response.responseHeaders())));  
    Assert.notNull(response, "HttpServerResponse must not be null");  
    this.response = response;  
}
```

可知 `ReactorServerHttpResponse` 构造函数初始化实例的时候，存放响应Header的是 `HttpHeaders` 实例，也就是响应Header是可以直接修改的。

`ServerHttpResponse` 接口如下：

```
public interface HttpMessage {  
  
    // 获取响应Header，目前的实现中返回的是HttpHeaders实例，可以直接修改  
    HttpHeaders getHeaders();  
}
```

```

public interface ReactiveHttpOutputMessage extends HttpResponseMessage {

    // 获取DataBufferFactory实例，用于包装或者生成数据缓冲区DataBuffer实例(创建响应体)
    DataBufferFactory bufferFactory();

    // 注册一个动作，在HttpOutputMessage提交之前此动作会进行回调
    void beforeCommit(Supplier<? extends Mono<Void>> action);

    // 判断HttpOutputMessage是否已经提交
    boolean isCommitted();

    // 写入消息体到HTTP协议层
    Mono<Void> writeWith(Publisher<? extends DataBuffer> body);

    // 写入消息体到HTTP协议层并且刷新缓冲区
    Mono<Void> writeAndFlushWith(Publisher<? extends Publisher<? extends
DataBuffer>>> body);

    // 指明消息处理已经结束，一般在消息处理结束自动调用此方法，多次调用不会产生副作用
    Mono<Void> setComplete();
}

public interface ServerHttpResponse extends ReactiveHttpOutputMessage {

    // 设置响应状态码
    boolean setStatusCode(@Nullable HttpStatus status);

    // 获取响应状态码
    @Nullable
    HttpStatus getStatusCode();

    // 获取响应Cookie，封装为MultiValueMap实例，可以修改
    MultiValueMap<String, ResponseCookie> getCookies();

    // 添加响应Cookie
    void addCookie(ResponseCookie cookie);
}

```

这里可以看到除了响应体比较难修改之外，其他的属性都是可变的。

ServerWebExchangeUtils和上下文属性

ServerWebExchangeUtils 里面存放了很多静态公有的字符串KEY值

(这些字符串KEY的实际值是

org.springframework.cloud.gateway.support.ServerWebExchangeUtils. + 下面任意的静态公有KEY),

这些字符串KEY值一般是用于 ServerWebExchange 的属性(Attribute, 见上文的 ServerWebExchange#getAttributes() 方法)的KEY, 这些属性值都是有特殊的含义, 在使用过滤器的时候如果时机适当可以直接取出来使用, 下面逐个分析。

- PRESERVE_HOST_HEADER_ATTRIBUTE: 是否保存Host属性, 值是布尔值类型, 写入位置是 PreserveHostHeaderGatewayFilterFactory, 使用的位置是 NettyRoutingFilter, 作用是如果设置为true, HTTP请求头中的Host属性会写到底层Reactor-Netty的请求Header属性中。

- `CLIENT_RESPONSE_ATTR`：保存底层Reactor-Netty的响应对象，类型是 `reactor.netty.http.client.HttpClientResponse`。
- `CLIENT_RESPONSE_CONN_ATTR`：保存底层Reactor-Netty的连接对象，类型是 `reactor.netty.Connection`。
- `URI_TEMPLATE_VARIABLES_ATTRIBUTE`： `PathRoutePredicateFactory` 解析路径参数完成之后，把解析完成后的占位符KEY-路径Path映射存放在 `ServerWebExchange` 的属性中，KEY就是 `URI_TEMPLATE_VARIABLES_ATTRIBUTE`。
- `CLIENT_RESPONSE_HEADER_NAMES`：保存底层Reactor-Netty的响应Header的名称集合。
- `GATEWAY_ROUTE_ATTR`：用于存放 `RoutePredicateHandlerMapping` 中匹配出来的具体的路由 (`org.springframework.cloud.gateway.route.Route`)实例，通过这个路由实例可以得知当前请求会路由到下游哪个服务。
- `GATEWAY_REQUEST_URL_ATTR`： `java.net.URI` 类型的实例，这个实例代表直接请求或者负载均衡处理之后需要请求到下游服务的真实URI。
- `GATEWAY_ORIGINAL_REQUEST_URL_ATTR`： `java.net.URI` 类型的实例，需要重写请求URI的时候，保存原始的请求URI。
- `GATEWAY_HANDLER_MAPPER_ATTR`：保存当前使用的 `HandlerMapping` 具体实例的类型简称(一般是字符串"RoutePredicateHandlerMapping")。
- `GATEWAY_SCHEME_PREFIX_ATTR`：确定目标路由URI中如果存在 `schemeSpecificPart` 属性，则保存该URI的scheme在此属性中，路由URI会被重新构造，见 `RouteToRequestUrlFilter`。
- `GATEWAY_PREDICATE_ROUTE_ATTR`：用于存放 `RoutePredicateHandlerMapping` 中匹配出来的具体的路由 (`org.springframework.cloud.gateway.route.Route`)实例的ID。
- `WEIGHT_ATTR`：实验性功能(此版本还不建议在正式版本使用)存放分组权重相关属性，见 `WeightCalculatorWebFilter`。
- `ORIGINAL_RESPONSE_CONTENT_TYPE_ATTR`：存放响应Header中的Content-Type的值。
- `HYSTRIX_EXECUTION_EXCEPTION_ATTR`： `Throwable` 的实例，存放的是Hystrix执行异常时候的异常实例，见 `HystrixGatewayFilterFactory`。
- `GATEWAY_ALREADY_ROUTED_ATTR`：布尔值，用于判断是否已经进行了路由，见 `NettyRoutingFilter`。
- `GATEWAY_ALREADY_PREFIXED_ATTR`：布尔值，用于判断请求路径是否被添加了前置部分，见 `PrefixPathGatewayFilterFactory`。

`ServerWebExchangeUtils` 提供的上下文属性用于 Spring Cloud Gateway 的 `ServerWebExchange` 组件处理请求和响应的时候，内部一些重要实例或者标识属性的安全传输和使用，使用它们可能存在一定的风险，

因为没有人可以确定在版本升级之后，原有的属性KEY或者VALUE是否会发生改变，如果评估过风险或者规避了风险之后，可以安心使用。

例如我们在做请求和响应日志(类似Nginx的Access Log)的时候，可以依赖到 `GATEWAY_ROUTE_ATTR`，因为我们要打印路由的目标信息。举个简单例子：

```
@Slf4j
@Component
public class AccessLogFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest request = exchange.getRequest();
        String path = request.getPath().pathWithinApplication().value();
        HttpMethod method = request.getMethod();
        // 获取路由的目标URI
    }
}
```

```

        URI targetUri =
exchange.getAttribute(ServerWebExchangeUtils.GATEWAY_REQUEST_URL_ATTR);
        InetSocketAddress remoteAddress = request.getRemoteAddress();
        return chain.filter(exchange.mutate().build()).then(Mono.fromRunnable(()
-> {
            ServerHttpResponse response = exchange.getResponse();
            HttpStatus statusCode = response.getStatusCode();
            log.info("请求路径:{},客户端远程IP地址:{},请求方法:{},目标URI:{},响应码:
{}",
                    path, remoteAddress, method, targetUri, statusCode);
        }));
    }
}

```

修改请求体和响应体

修改请求体

修改请求体是一个比较常见的需求。

例如我们使用 `Spring Cloud Gateway` 实现网关的时候，要实现一个功能：

把存放在请求头中的JWT解析后，提取里面的用户ID，然后写入到请求体中。

我们简化这个场景，假设我们把userId明文存放在请求头中的accessToken中，请求体是一个JSON结构：

```

{
    "serialNumber": "请求流水号",
    "payload" : {
        // ... 这里是有效载荷，存放具体的数据
    }
}

```

我们需要提取accessToken，也就是userId插入到请求体JSON中如下：

```

{
    "userId": "用户ID",
    "serialNumber": "请求流水号",
    "payload" : {
        // ... 这里是有效载荷，存放具体的数据
    }
}

```

这里为了简化设计，用全局过滤器 `GlobalFilter` 实现，实际需要结合具体场景考虑：

```

@Slf4j
@Component
public class ModifyRequestBodyGlobalFilter implements GlobalFilter {

    private final DataBufferFactory dataBufferFactory = new
NettyDataBufferFactory(ByteBufAllocator.DEFAULT);

    @Autowired

```

```

private ObjectMapper objectMapper;

@Override
public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
chain) {
    ServerHttpRequest request = exchange.getRequest();
    String accessToken = request.getHeaders().getFirst("accessToken");
    if (!StringUtils.hasLength(accessToken)) {
        throw new IllegalArgumentException("accessToken");
    }
    // 新建一个ServerHttpRequest装饰器,覆盖需要装饰的方法
    ServerHttpRequestDecorator decorator = new
ServerHttpRequestDecorator(request) {

        @Override
        public Flux<DataBuffer> getBody() {
            Flux<DataBuffer> body = super.getBody();
            InputStreamHolder holder = new InputStreamHolder();
            body.subscribe(buffer -> holder.inputStream =
buffer.asInputStream());
            if (null != holder.inputStream) {
                try {

                    // 解析JSON的节点
                    JsonNode jsonNode =
objectMapper.readTree(holder.inputStream);
                    Assert.isTrue(jsonNode instanceof ObjectNode, "JSON格式异
常");

                    ObjectNode objectNode = (ObjectNode) jsonNode;

                    // JSON节点最外层写入新的属性
                    objectNode.put("userId", accessToken);
                    DataBuffer dataBuffer =
dataBufferFactory.allocateBuffer();
                    String json = objectNode.toString();
                    log.info("最终的JSON数据为:{}", json);
                    dataBuffer.write(json.getBytes(StandardCharsets.UTF_8));

                    return Flux.just(dataBuffer);
                } catch (Exception e) {
                    throw new IllegalStateException(e);
                }
            } else {
                return super.getBody();
            }
        }
    };
    // 使用修改后的ServerHttpRequestDecorator重新生成一个新的ServerWebExchange
    return chain.filter(exchange.mutate().request(decorator).build());
}

private class InputStreamHolder {

    InputStream inputStream;

}
}

```

测试一下：

```
// HTTP
POST /order/json HTTP/1.1
Host: localhost:9090
Content-Type: application/json
accessToken: 10086
Accept: */*
Cache-Control: no-cache
Host: localhost:9090
accept-encoding: gzip, deflate
content-length: 94
Connection: keep-alive
cache-control: no-cache

{
  "serialNumber": "请求流水号",
  "payload": {
    "name": "doge"
  }
}

// 日志输出
最终的JSON数据为:{"serialNumber":"请求流水号","payload":
{"name":"doge"},"userId":"10086"}
```

最重要的是用到了 `ServerHttpRequest` 装饰器 `ServerHttpRequestDecorator`，主要覆盖对应获取请求体数据缓冲区的方法即可，至于怎么处理其他逻辑需要自行考虑，这里只是做一个简单的示范。

一般的代码逻辑如下：

```
ServerHttpRequest request = exchange.getRequest();
ServerHttpRequestDecorator requestDecorator = new
ServerHttpRequestDecorator(request) {

    @Override
    public Flux<DataBuffer> getBody() {
        // 拿到承载原始请求体的Flux
        Flux<DataBuffer> body = super.getBody();
        // 这里通过自定义方式生成新的承载请求体的Flux
        Flux<DataBuffer> newBody = ...
        return newBody;
    }
}
return chain.filter(exchange.mutate().request(requestDecorator).build());
```

修改响应体

修改响应体的需求也是比较常见的，具体的做法和修改请求体差不多。

例如我们想要实现下面的功能：第三方服务请求经过网关，原始报文是密文，我们需要在网关实现密文解密，然后把解密后的明文路由到下游服务，下游服务处理成功响应明文，需要在网关把明文加密成密文再返回到第三方服务。

现在简化整个流程，用AES加密算法，统一密码为字符串"throwable"，假设请求报文和响应报文明文如下：

```
// 请求密文
{
    "serialNumber": "请求流水号",
    "payload" : "加密后的请求消息载荷"
}

// 请求明文（仅仅作为提示）
{
    "serialNumber": "请求流水号",
    "payload" : "{\"name\":\"doge\"}"
}

// 响应密文
{
    "code": 200,
    "message": "ok",
    "payload" : "加密后的响应消息载荷"
}

// 响应明文（仅仅作为提示）
{
    "code": 200,
    "message": "ok",
    "payload" : "{\"name\":\"doge\",\"age\":26}"
}
```

为了方便一些加解密或者编码解码的实现，需要引入 Apache 的 `commons-codec` 类库：

```
<dependency>
  <groupId>commons-codec</groupId>
  <artifactId>commons-codec</artifactId>
  <version>1.12</version>
</dependency>
```

这里定义一个全局过滤器专门处理加解密，实际上最好结合真实的场景决定是否适合全局过滤器，这里只是一个示例：

```
// AES加解密工具类
public enum AesUtils {

    // 单例
    X;

    private static final String PASSWORD = "throwable";
    private static final String KEY_ALGORITHM = "AES";
    private static final String SECURE_RANDOM_ALGORITHM = "SHA1PRNG";
    private static final String DEFAULT_CIPHER_ALGORITHM =
        "AES/ECB/PKCS5Padding";

    public String encrypt(String content) {
        try {
            Cipher cipher = Cipher.getInstance(DEFAULT_CIPHER_ALGORITHM);
            cipher.init(Cipher.ENCRYPT_MODE, provideSecretKey());
            return
                Hex.encodeHexString(cipher.doFinal(content.getBytes(StandardCharsets.UTF_8)));
        } catch (Exception e) {
```

```

        throw new IllegalArgumentException(e);
    }
}

public byte[] decrypt(String content) {
    try {
        Cipher cipher = Cipher.getInstance(DEFAULT_CIPHER_ALGORITHM);
        cipher.init(Cipher.DECRYPT_MODE, provideSecretKey());
        return cipher.doFinal(Hex.decodeHex(content));
    } catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}

private SecretKey provideSecretKey() {
    try {
        KeyGenerator keyGen = KeyGenerator.getInstance(KEY_ALGORITHM);
        SecureRandom secureRandom =
SecureRandom.getInstance(SECURE_RANDOM_ALGORITHM);
        secureRandom.setSeed(PASSWORD.getBytes(StandardCharsets.UTF_8));
        keyGen.init(128, secureRandom);
        return new SecretKeySpec(keyGen.generateKey().getEncoded(),
KEY_ALGORITHM);
    } catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}
}

// EncryptionGlobalFilter
@Slf4j
@Component
public class EncryptionGlobalFilter implements GlobalFilter, Ordered {

    @Autowired
    private ObjectMapper objectMapper;

    @Override
    public int getOrder() {
        return -2;
    }

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
chain) {
        ServerHttpRequest request = exchange.getRequest();
        // 响应体
        ServerHttpResponse response = exchange.getResponse();
        DataBufferFactory bufferFactory =
exchange.getResponse().bufferFactory();
        ServerHttpRequestDecorator requestDecorator = processRequest(request,
bufferFactory);
        ServerHttpResponseDecorator responseDecorator =
processResponse(response, bufferFactory);
        return
chain.filter(exchange.mutate().request(requestDecorator).response(responseDecora
tor).build());
    }
}

```

```

private ServerHttpRequestDecorator processRequest(ServerHttpRequest request,
DataBufferFactory bufferFactory) {
    Flux<DataBuffer> body = request.getBody();
    DataBufferHolder holder = new DataBufferHolder();
    body.subscribe(dataBuffer -> {
        int len = dataBuffer.readableByteCount();
        holder.length = len;
        byte[] bytes = new byte[len];
        dataBuffer.read(bytes);
        DataBufferUtils.release(dataBuffer);
        String text = new String(bytes, StandardCharsets.UTF_8);
        JsonNode jsonNode = readNode(text);
        JsonNode payload = jsonNode.get("payload");
        String payloadText = payload.asText();
        byte[] content = AesUtils.X.decrypt(payloadText);
        String requestBody = new String(content, StandardCharsets.UTF_8);
        log.info("修改请求体payload,修改前:{}",修改后:{}", payloadText,
requestBody);
        rewritePayloadNode(requestBody, jsonNode);
        DataBuffer data = bufferFactory.allocateBuffer();
        data.write(jsonNode.toString().getBytes(StandardCharsets.UTF_8));
        holder.dataBuffer = data;
    });
    HttpHeaders headers = new HttpHeaders();
    headers.putAll(request.getHeaders());
    headers.remove(HttpHeaders.CONTENT_LENGTH);
    return new ServerHttpRequestDecorator(request) {

        @Override
        public HttpHeaders getHeaders() {
            int contentLength = holder.length;
            if (contentLength > 0) {
                headers.setContentLength(contentLength);
            } else {
                headers.set(HttpHeaders.TRANSFER_ENCODING, "chunked");
            }
            return headers;
        }

        @Override
        public Flux<DataBuffer> getBody() {
            return Flux.just(holder.dataBuffer);
        }
    };
}

private ServerHttpResponseDecorator processResponse(ServerHttpResponse
response, DataBufferFactory bufferFactory) {
    return new ServerHttpResponseDecorator(response) {

        @SuppressWarnings("unchecked")
        @Override
        public Mono<Void> writewith(Publisher<? extends DataBuffer> body) {
            if (body instanceof Flux) {
                Flux<? extends DataBuffer> flux = (Flux<? extends
DataBuffer>) body;
                return super.writewith(flux.map(buffer -> {

```

```

        CharBuffer charBuffer =
StandardCharsets.UTF_8.decode(buffer.asByteBuffer());
        DataBufferUtils.release(buffer);
        JsonNode jsonNode = readNode(charBuffer.toString());
        JsonNode payload = jsonNode.get("payload");
        String text = payload.toString();
        String content = AesUtils.X.encrypt(text);
        log.info("修改响应体payload,修改前:{},修改后:{}", text,
content);

        setPayloadTextNode(content, jsonNode);
        return
bufferFactory.wrap(jsonNode.toString().getBytes(StandardCharsets.UTF_8));
    });
}
return super.writewith(body);
}
};
}

private void rewritePayloadNode(String text, JsonNode root) {
    try {
        JsonNode node = objectMapper.readTree(text);
        ObjectNode objectNode = (ObjectNode) root;
        objectNode.set("payload", node);
    } catch (Exception e) {
        throw new IllegalStateException(e);
    }
}

private void setPayloadTextNode(String text, JsonNode root) {
    try {
        ObjectNode objectNode = (ObjectNode) root;
        objectNode.set("payload", new TextNode(text));
    } catch (Exception e) {
        throw new IllegalStateException(e);
    }
}

private JsonNode readNode(String in) {
    try {
        return objectMapper.readTree(in);
    } catch (Exception e) {
        throw new IllegalStateException(e);
    }
}

private class DataBufferHolder {

    DataBuffer dataBuffer;
    int length;
}
}

```

先准备一份密文：

```

Map<String, Object> json = new HashMap<>(8);
json.put("serialNumber", "请求流水号");
String content = "{\"name\": \"doge\"}";
json.put("payload", AesUtils.X.encrypt(content));
System.out.println(new ObjectMapper().writeValueAsString(json));

// 输出
{"serialNumber": "请求流水号", "payload": "144e3dc734743f5709f1adf857bca473da683246fd612f86ac70edeb5f2d2729"}

```

模拟请求:

```

POST /order/json HTTP/1.1
Host: localhost:9090
accessToken: 10086
Content-Type: application/json
User-Agent: PostmanRuntime/7.13.0
Accept: */*
Cache-Control: no-cache
Postman-Token: bda07fc3-ea1a-478c-b4d7-754fe6f37200,634734d9-feed-4fc9-ba20-7618bd986e1c
Host: localhost:9090
cookie: customCookieName=customCookieValue
accept-encoding: gzip, deflate
content-length: 104
Connection: keep-alive
cache-control: no-cache

{
  "serialNumber": "请求流水号",
  "payload": "FE49xzR0P1cJ8a34V7ykc9poMkb9YS+GrHdt618tJyk="
}

// 响应结果
{
  "serialNumber": "请求流水号",
  "payload": "oo/K1igg2t/S8EEExkBVGWOfI1gAh5pBpz0wyjNPW6e8=" # <--- 解密后:
{"name": "doge", "age": 26}
}

```

遇到的问题:

- 必须实现 Ordered 接口, 返回一个小于-1的order值, 这是因为 NettyWriteResponseFilter 的 order值为-1, 我们需要覆盖返回响应体的逻辑, 自定义的 GlobalFilter 必须比 NettyWriteResponseFilter 优先执行。
- 网关每次重启之后, 第一个请求总是无法从原始的 ServerHttpRequest 读取到有效的Body, 准确来说出现的现象是 NettyRoutingFilter 调用 ServerHttpRequest#getBody() 的时候获取到一个空的对象, 导致空指针; 奇怪的是从第二个请求开始就能正常调用。笔者把 Spring Cloud Gateway 的版本降低到 Finchley.SR3, Spring Boot 的版本降低到 2.0.8.RELEASE, 问题不再出现, 初步确定是 Spring Cloud Gateway 版本升级导致的兼容性问题或者是BUG。

最重要的是用到了 ServerHttpResponse 装饰器 ServerHttpResponseDecorator, 主要覆盖写入响应体数据缓冲区的部分, 至于怎么处理其他逻辑需要自行考虑, 这里只是做一个简单的示范。一般的代码逻辑如下:

```

ServerHttpResponse response = exchange.getResponse();
ServerHttpResponseDecorator responseDecorator = new
ServerHttpResponseDecorator(response) {

    @Override
    public Mono<Void> writeWith(Publisher<? extends DataBuffer> body) {
        if (body instanceof Flux) {
            Flux<? extends DataBuffer> flux = (Flux<? extends DataBuffer>) body;
            return super.writeWith(flux.map(buffer -> {
                // buffer就是原始的响应数据的缓冲区
                // 下面处理完毕之后返回新的响应数据的缓冲区即可
                return bufferFactory.wrap(...);
            }));
        }
        return super.writeWith(body);
    }
};
return chain.filter(exchange.mutate().response(responseDecorator).build());

```

Webflux + R2DBC 操作 MySQL

R2DBC 是一个异步操作数据库的驱动，区别于传统的同步数据库驱动 JDBC，R2DBC 与数据库的各种操作也是异步的，这将大量节省高并发系统的线程数量。

首先，创建一个 User 实体类用于测试，同时在 MySQL 中创建相应的数据库以及表结构

```

@Data
@AllArgsConstructor
@NoArgsConstructor
@Table("webflux_user")
public class User {

    @Id
    private int id;

    private String username;

    private String password;

}

```

编写数据仓库层，使用 Spring-data 封装好的简单 CRUD 接口（用法类似 JPA）

```

package com.crazymaker.springcloud.reactive.user.info.dao.impl;

import com.crazymaker.springcloud.reactive.user.info.dto.User;
import org.springframework.data.repository.reactive.ReactiveCrudRepository;

public interface UserRepository extends ReactiveCrudRepository<User, Integer> {

}

```

此时就可以调用封装好的 CRUD 方法进行简单的增删改查操作了。

在 Webflux 框架中，我们可以使用 SpringMVC 中 Controller + Service 的模式进行开发，也可以使用 Webflux 中 route + handler 的模式进行开发。

Controller + Service

编写 Service 调用 UserRepository

```
package com.crazymaker.springcloud.reactive.user.info.service.impl;

import com.crazymaker.springcloud.common.result.RestOut;
import com.crazymaker.springcloud.reactive.user.info.dao.impl.UserRepository;
import com.crazymaker.springcloud.reactive.user.info.dto.User;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public Mono<User> addUser(User user) {
        return userRepository.save(user);
    }

    public Mono<RestOut<Void>> delUser(long id) {
        return userRepository.findById(id)
            .flatMap(user -> userRepository.delete(user).then(Mono.just(
                RestOut.<Void>succeed("delete ok "))))
            .defaultIfEmpty(RestOut.<Void>succeed("没有找到数据 "));
    }

    public Mono<RestOut<User>> updateUser(User user) {
        return userRepository.findById(user.getUserId())
            .flatMap(user0 -> userRepository.save(user))
            .map(user0 -> RestOut.success(user))
            .defaultIfEmpty(RestOut.<User>succeed("没有找到数据 "));
    }

    public Flux<User> getAllUser() {
        return userRepository.findAll();
    }
}
```

编写 Controller 进行测试

```
package com.crazymaker.springcloud.reactive.user.info.controller;

import com.crazymaker.springcloud.common.result.RestOut;
import com.crazymaker.springcloud.reactive.user.info.dto.User;
import com.crazymaker.springcloud.reactive.user.info.service.impl.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.RestOut;
import org.springframework.web.bind.annotation.*;
```

```

import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

@RestController
@RequestMapping("/user")
public class UserController {

    @Autowired
    private UserService userService;

    @PostMapping
    public Mono<User> addUser(@RequestBody User user) {
        return userService.addUser(user);
    }

    @DeleteMapping("/{id}")
    public Mono<RestOut<Void>> delUser(@PathVariable int id) {
        return userService.delUser(id);
    }

    @PutMapping
    public Mono<RestOut<User>> updateUser(@RequestBody User user) {
        return userService.updateUser(user);
    }

    @GetMapping
    public Flux<User> getAllUser() {
        return userService.getAllUser();
    }
}

```

Route + Handler

handler 就相当于定义很多处理器，其中不同的方法负责处理不同路由的请求，其对应的是传统的 Service 层

```

package com.crazymaker.springcloud.reactive.user.info.handler;

import com.crazymaker.springcloud.reactive.user.info.dao.impl.UserRepository;
import com.crazymaker.springcloud.reactive.user.info.dto.User;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.server.ServerRequest;
import org.springframework.web.reactive.function.server.ServerResponse;
import reactor.core.publisher.Mono;

@Component
public class UserHandler {

    @Autowired
    private UserRepository userRepository;

    public Mono<ServerResponse> addUser(ServerRequest request) {
        return ServerResponse.ok()
            .contentType(MediaType.APPLICATION_JSON)

```

```

        .body(userRepository.saveAll(request.bodyToMono(User.class)),
User.class);
    }

    public Mono<ServerResponse> delUser(ServerRequest request) {
        return
userRepository.findById(Long.parseLong(request.pathVariable("id")))
        .flatMap(user ->
userRepository.delete(user).then(ServerResponse.ok().build()))
        .switchIfEmpty(ServerResponse.notFound().build());
    }

    public Mono<ServerResponse> updateUser(ServerRequest request) {
        return ServerResponse.ok()
            .contentType(MediaType.APPLICATION_JSON)
            .body(userRepository.saveAll(request.bodyToMono(User.class)),
User.class);
    }

    public Mono<ServerResponse> getAllUser(ServerRequest request) {
        return ServerResponse.ok()
            .contentType(MediaType.APPLICATION_JSON)
            .body(userRepository.findAll(), User.class);
    }

    public Mono<ServerResponse> getAllUserStream(ServerRequest request) {
        return ServerResponse.ok()
            .contentType(MediaType.TEXT_EVENT_STREAM)
            .body(userRepository.findAll(), User.class);
    }
}

```

route 就是路由配置，其规定路由的分发规则，将不同的请求路由分发给相应的 handler 进行业务逻辑的处理，其对应的就是传统的 Controller 层

```

@Configuration
public class RouteConfig {

    @Bean
    RouterFunction<ServerResponse> userRoute(UserHandler userHandler) {
        return RouterFunctions.nest(
            RequestPredicates.path("/userRoute"),
            RouterFunctions.route(RequestPredicates.POST(""),
userHandler::addUser)
                .andRoute(RequestPredicates.DELETE("/{id}"),
userHandler::delUser)
                .andRoute(RequestPredicates.PUT(""),
userHandler::updateUser)
                .andRoute(RequestPredicates.GET(""),
userHandler::getAllUser)
                .andRoute(RequestPredicates.GET("/stream"),
userHandler::getAllUserStream)
            );
    }
}

```

参考文献

<https://blog.csdn.net/wpc2018/article/details/122634049>

<https://www.jianshu.com/p/7d80b94068b3>

https://blog.csdn.net/yhj_911/article/details/119540000

<http://bjqianye.cn/detail/6845.html>

<https://blog.csdn.net/hao134838/article/details/110824092>

<https://blog.csdn.net/hao134838/article/details/110824092>

https://blog.csdn.net/weixin_34096182/article/details/91436704

<https://blog.csdn.net/fly910905/article/details/121682625>

<https://gaocher.github.io/2020/01/05/mono-create/>

推荐阅读：

《[全链路异步，让你的 SpringCloud 性能优化10倍+](#)》

《[Linux命令大全：2W多字，一次实现Linux自由](#)》

《[网易二面：CPU狂飙900%，该怎么处理？](#)》

《[阿里二面：千万级、亿级数据，如何性能优化？教科书级 答案来了](#)》

《[峰值21WQps、亿级DAU，小游戏《羊了个羊》是怎么架构的？](#)》

《[场景题：假设10W人突访，你的系统如何做到不 雪崩？](#)》

《[2个大厂 100亿级 超大流量 红包 架构方案](#)》

《[Nginx面试题（史上最全 + 持续更新）](#)》

《[K8S面试题（史上最全 + 持续更新）](#)》

《[操作系统面试题（史上最全、持续更新）](#)》

《[Docker面试题（史上最全 + 持续更新）](#)》

《[Springcloud gateway 底层原理、核心实战\(史上最全\)](#)》

《[Flux、Mono、Reactor 实战（史上最全）](#)》

《[sentinel（史上最全）](#)》

《[Nacos\(史上最全\)](#)》

《[TCP协议详解\(史上最全\)](#)》

《[分库分表 Sharding-JDBC 底层原理、核心实战（史上最全）](#)》

《[clickhouse 超底层原理 + 高可用实操（史上最全）](#)》

《[nacos高可用（图解+秒懂+史上最全）](#)》

《[队列之王：Disruptor 原理、架构、源码 一文穿透](#)》

《[环形队列、条带环形队列 Striped-RingBuffer（史上最全）](#)》

《[一文搞定：SpringBoot、SLF4j、Log4j、Logback、Netty之间混乱关系（史上最全）](#)》

《[单例模式（史上最全）](#)》

《[红黑树（图解 + 秒懂 + 史上最全）](#)》

《[分布式事务（秒懂）](#)》

《[缓存之王：Caffeine 源码、架构、原理（史上最全，10W字 超级长文）](#)》

《[缓存之王：Caffeine 的使用（史上最全）](#)》

《[Java Agent 探针、字节码增强 ByteBuddy（史上最全）](#)》

《[Docker原理（图解+秒懂+史上最全）](#)》

《[Redis分布式锁（图解 - 秒懂 - 史上最全）](#)》

《[Zookeeper 分布式锁 - 图解 - 秒懂](#)》

《[Zookeeper Curator 事件监听 - 10分钟看懂](#)》

《[Netty 粘包 拆包 | 史上最全解读](#)》

《[Netty 100万级高并发服务器配置](#)》

《[Springcloud 高并发 配置（一文全懂）](#)》

硬核推荐：尼恩Java硬核架构班

又名疯狂创客圈社群 VIP

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java 硬核架构班

已经发布

- ★ 《高能性能RPC的基础实操之：从0到1开始IM撸一个IM》
- ★ 《分布式高能性能RPC的基础实操之：千万级用户分布式IM实操-含简历指导》
- ★ 《亿级用户超高并发秒杀实操-含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023春招面试涨薪神器**
- ★ 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- ★ 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023春招面试涨薪大神器
- ★ 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- ★ 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- ★ 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- ★ 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- ★ 《架构师实操篇：redis cluster 工业级高可用实操》
- ★ 《架构师实操篇：100W级别QPS日志平台实操》
- ★ 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》

规划中

《架构师实操篇：基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架》

《架构师实操篇：go语言学习，以及基于 go 手写 rpc 框架》

《架构师实操篇：千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目》

《架构师实操篇：工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目》

特色

会员制

提供技术方向指导，
职业生涯指导，少躺坑，少弯路

简历指导

这个很重要，
对于挪窝涨薪来说

实操性

以上项目，都是老架构师
在生产上实操过的项目

非水货

40岁老架构师，不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

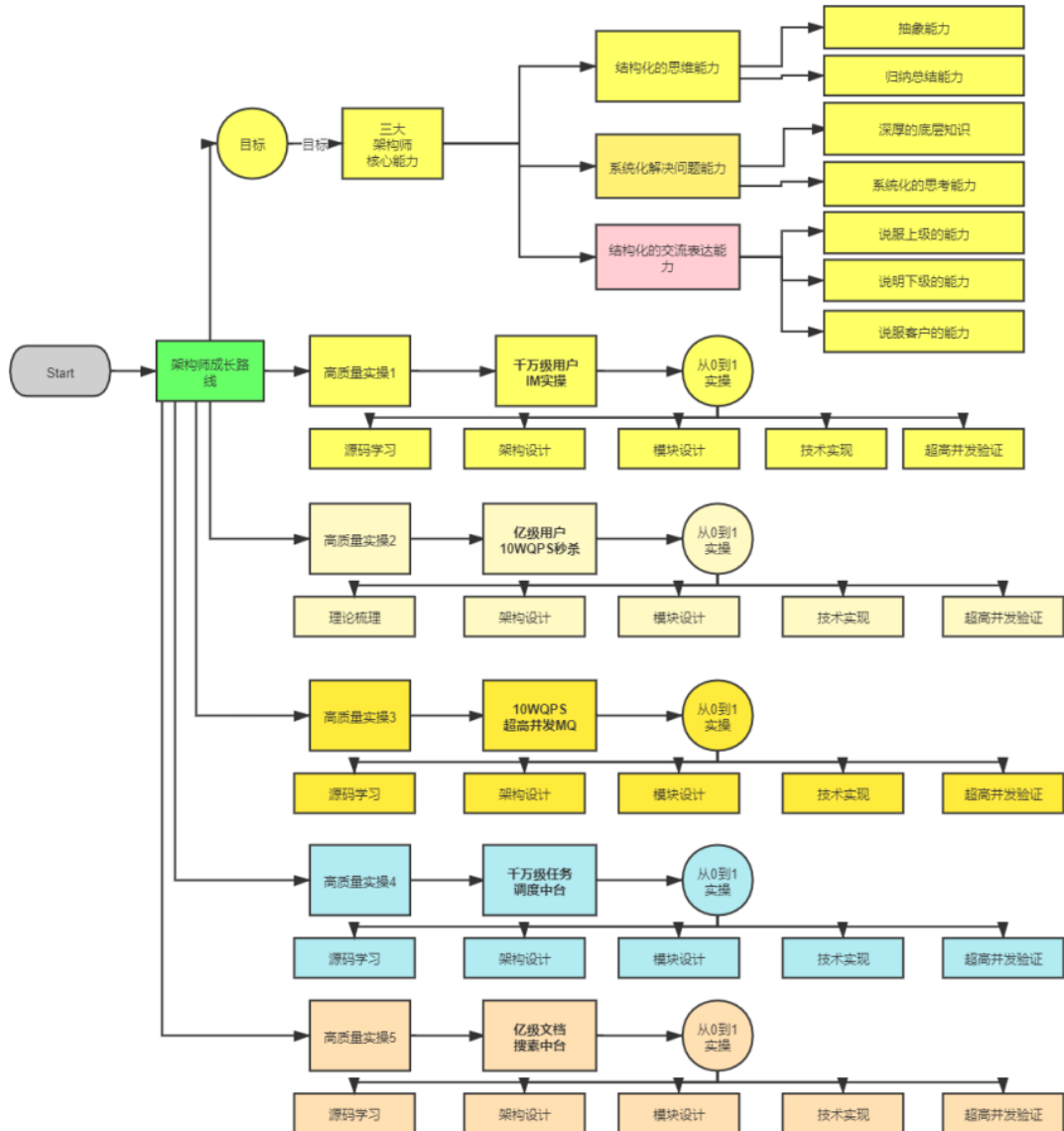
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



【样章】第 17 章:横扫全网Rocketmq 视频第 2 部曲: 工业级 rocketmq 高可用(HA) 底层原理和实操

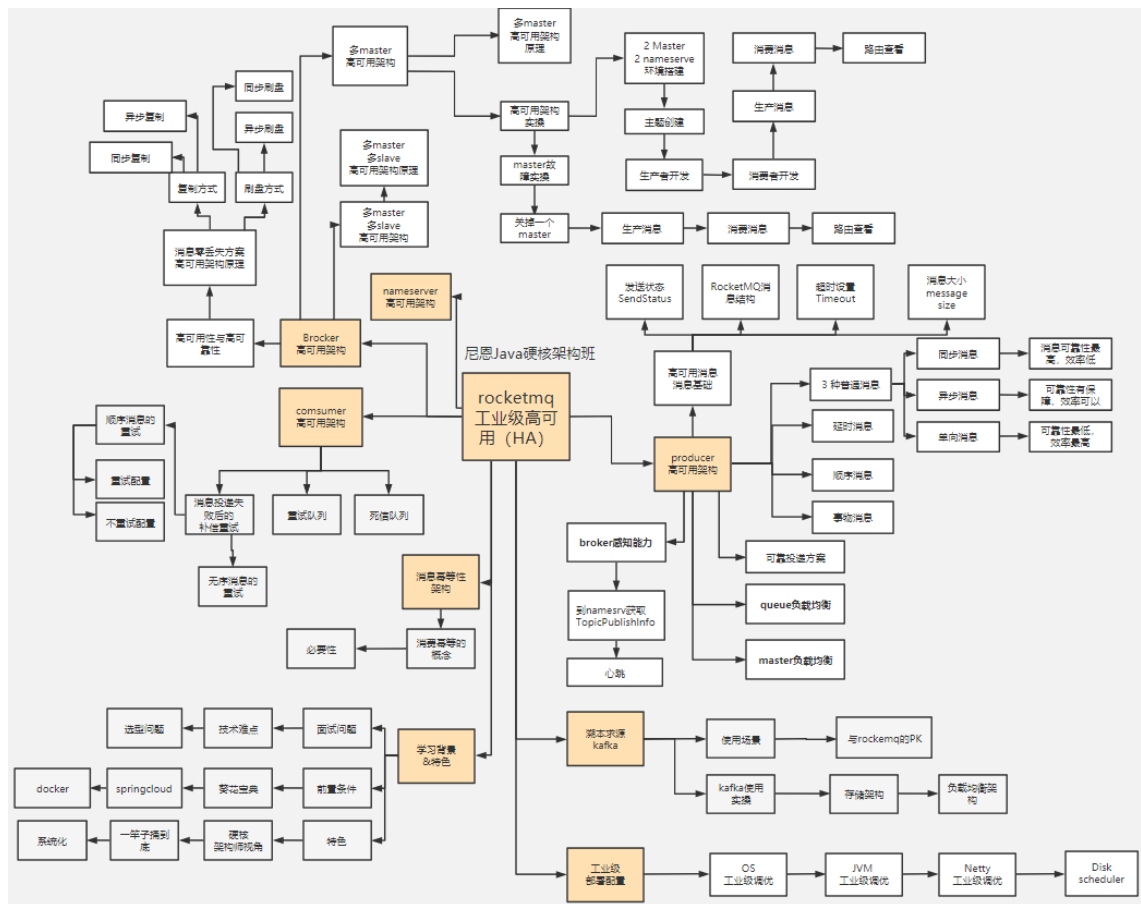
工业级 rocketmq 高可用底层原理, 包含: 消息消费、同步消息、异步消息、单向消息等不同消息的底层原理和源码实现; 消息队列非常底层的主从复制、高可用、同步刷盘、异步刷盘等底层原理。

工业级 rocketmq 高可用底层原理和搭建实操, 包含: 高可用集群的搭建。

解决以下难题:

- 1、技术难题: RocketMQ 如何最大限度的保证消息不丢失的呢? RocketMQ 消息如何做到高可靠投递?
- 2、技术难题: 基于消息的分布式事务, 核心原理不理解
- 3、选型难题: kafka or rocketmq , 该娶谁?

下图链接: <https://www.proceson.com/view/6178e8ae0e3e7416bde9da19>



成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情: <http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

[最新](#) [最后发表](#) [热门](#) [精华](#)

成功案例: [1057号卷王] 3年小伙拿到外企offer, 薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例: [645号卷王] 4年经验卷王逆袭, 被毕业后, 反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例: [878号卷王] 小伙8年经验, 年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例: 通过尼恩的指导, 小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例: [493号卷王] 5年小伙拿满意offer, 就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例: [250号卷王] 就业极寒时代, 收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例: [612号卷王] 就业极寒时代, 从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例: [913号卷王] 热烈祝贺6年经验卷王, 年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例: [959号卷王] 4年经验卷王, 喜获百度、Boss直聘等N个优质offer, 最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例: [529号卷王] 5年经验卷王喜收2大offer, 最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例: [811号卷王] 热烈祝贺7年经验卷王, 薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例: [287号卷王] 不惧大寒潮, 卷王逆市收4 offer, 涨30%, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [1002号卷王] 5月份“被毕业”, 改简历后, 斩获顶级央企Offer, 涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

① 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

① 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

① 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

① 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

① 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

① 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

① 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

① 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

① 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

① 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

① 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

① 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

① 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

① 卷王1号 超级版主 2022-3-25

成功案列: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

1 卷王1号 超级版主 2022-3-24

成功案列: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

1 卷王1号 超级版主 2022-3-21

成功案列: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

1 卷王1号 超级版主 2022-3-20

成功案列: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

1 卷王1号 超级版主 2022-3-16

简历案列: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

1 卷王1号 超级版主 2022-3-10

成功案列: 祝贺我国两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

1 卷王1号 超级版主 2022-3-10

成功案列: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

1 卷王1号 超级版主 2022-3-10

成功案列: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

尼恩 超级版主 2022-3-5

成功案列: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

1 卷王1号 超级版主 2022-2-27

成功案列: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

1 卷王1号 超级版主 2022-2-27

成功案列: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

1 卷王1号 超级版主 2022-2-27

成功案列: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

1 卷王1号 超级版主 2022-2-27

成功案列: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

1 卷王1号 超级版主 2022-2-27

成功案列: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

1 卷王1号 超级版主 2022-2-27

☐ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [449号卷王] 应届毕业生卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

☐ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

☐ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

☐ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP含免费简历优化）

9年 小伙伴拿到 年薪90W offer

9月11日改简历 11月29日晒offer

秘诀:
简历指导+ 狠狠卷

薪资高 稳定

这个是给你微调的

主要的工作是啥?

省略的地方, 需要你再补充一点

提升了的实力, 就不用怕

易所 关于数字资 的 易

po 也有 打盹的时候, 该裁员, 照着一个不少

上面你留着这个就行

9月11日 下午17:38

年包比 po 多 19w

这么多

po 估计有 70 万

那你不是有 90 个 W?

po 给我 68

就是吗

Java 开发 - 9 年 - 修改.docx 34.1 KB

小伙8年经验 年薪60w

7月12日改简历 8月10日晒offer

秘诀:
改简历+ 狠狠卷

涨幅多少呀?

或者, 幅度多少呀?

之前 36*15, 现在这个 39*15

今年行情不太好, 还有一丢 offer 基本都是平薪, 或降薪的。

OK

这个马上来

刚在指导简历

哈哈

稍等哈

辛苦明惠哥

这次找工作, 您的指导作用超关键了。

我这次面试基本看的都是咱们课程的 面试 案例。

明天晚上哈

好哒

7月12日 上午9:43

7月12日 上午10:56

7月12日 上午10:20

今晚还在外边应招, 估计回去比较晚

要不, 咱们延到明天, 如何

明天白天也行

好的, 白天吧, 答应别人明天给他们简历了。

7月12日 上午10:27

OK

那就上午 11 点左右哈

好的

6年小伙伴 年薪40w

9月6日改简历 9月21日晒offer

秘诀:
简历指导+ 狠狠卷

今年这行情, 也算可以了

总包多少呀, 让我也瞧瞧一下

大概 40w 吧

谢谢恩师的指导和鼓励

是在深圳

深圳的行情尤其难

能有面试电话就不错啦

是的

太牛啦

哈哈哈哈哈, 恩师的鼓励指导也很重要

给你前面调整了一下

9月6日 下午16:47

恩师 - Java - 6 年.docx 24.7 KB

恩师, 简历我改好了, 您再帮我看一下

9月6日 下午16:59

恩师 - Java - 6 年(1).docx 24.5 KB

恩师, 看第二个吧

9月6日 晚上19:41

9月6日 晚上19:45

5年小伙喜提3个offer 年薪 35个W

5月22日改简历 11月29日晒offer

秘诀:
简历指导+ 狠狠卷

恩师晚上好, 汇报下最近的 offer 情况, 最近面试收到了三个 offer, 两个是平薪, 一个是跳槽跨境电商公司的 offer, 涨幅暂时不到 20%, 通过这次面试也让我知道自己跟高级开发还有一点距离, 还要再多卷才能突破。

最后结合自己的情况, 先选择去跨境电商的公司再提升下

之前是 20K*13.5, 跨境电商这个是 23K* (14-15)

恩师恩师

恭喜恭喜, 喜提 3 个 offer, 已经很棒了, 已经是高手了

很多小伙伴, 面试电话一个都没接到, 简历没投 7000 份, 只收到 3 次面试机会, 没有一个拿到最终 offer

您这个年薪, 算下来也有 35W 了吧

车贷部分还要确认下, 大部分人说只有两个月

这个时间点, 拿到这个水平, 挺不错的哈

持续加油卷哈

把你视频都看看

哈哈! 看看明年自己有没有能力冲击离开

辛苦老师再帮忙指导下哈

5月22日 上午12:45

5月22日 上午12:45

恩师晚上好, 结合老师的指导, 我对自己的简历做了修改, 有感谢大家的感谢, 我也想让您也帮忙看看给一些建议~希望在当前环境有 offer 收获~

新简历 - Java 开发 - 本科统招 - 2022-5 年经验.docx 35.0 KB

恩师晚上好

我报好号, 回头找您哈

OK

1.5年小伙搞定15K offer

就业寒冬涨100%

5月7日改简历 11月21日晒offer

秘诀:
简历指导+ 狠狠卷

他那个不止再修修嘛，我才是两位，原先7k，现在15k

那也很牛

都翻倍了，都是牛人

正常就30%

好的

晚上指导你改

推送中台需要加不

我还没做完，准备八脱文暂时没时间搞这些，入职了会开始搞

OK

还有别的项目吗

一个项目，太少啦

那种练习项目也行

其实我工作一年，

下一步可以瞄准25k

一年经验，搞定15k offer，舍你其谁

那你就更牛逼啦

卷王逆袭成功案例

6年小伙从很少面试机会到搞定35K*14薪

3月5日改简历 4月11日拿offer

一个月拿到了理想的offer

面试邀请少 小伙很苦恼

理想很丰满 目标35K

面试法宝 rocketmq四部曲

6年 经验小伙伴

喜收25K offer

3月12日改简历 12月1日晒offer

秘诀:
简历指导+ 狠狠卷

咱们以这个项目为模板改哈

项目有多少人，你带领多少人？

你工作几年了？

6年了

改简历那晚，20涨到25k，高级开发，P7带的后端团队

任务重，道路远，并不是没有方向

7年经验卷王

薪酬涨30%

7月11日改简历 9月1日晒offer

秘诀:
改简历+ 狠狠卷

只要本事好，拿offer比较容易的

入职了，最终并不输于面试官，还差旧的大牛，只涨了30%

恭喜一下

现在不比往年

能涨30%是大牛

拿到offer都算大牛

现在很多人投几百发，连个电话都没有

你已经很牛啦

4年经验卷王逆袭 被毕业后，反涨24W

7月改简历 8月30日晒offer

**秘诀：
改简历 + 狠卷**

这就是你的简历
差得太多啦
老哥 我八月十号开始找工作，今天已入职了
能涨24W
原因大概是啥？
很多小伙伴，面试机会都没有
这个地方的要这样写
项目被终止
方便语音沟通不

尼恩 你这么指导，非常重要
老哥 我八月十号开始找工作，今天已入职了
现金基本持平，股票 +24W
总计涨了多少钱
股票这个吧只能到手了才算
也不错啦
继续跟着你学技术

小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历 7月5日晒offer

**秘诀：
简历指导 + 狠卷3高**

快速看书，就能不求甚解，把自读和场景大概一下，然后重点的地方，用到的地方，再去回顾
我被"毕业"了
这周末或下周找你改一下简历
毕业没有关系
ok，发我吧
R行业，就来说去，太复杂啦
嗯，其实有点心理准备
不太会写简历

尼恩 我拿到手写的offer了
涨20%，涨更多。结果人家都
看起来里面不差钱呀
超过了8000块
平均算下来
7000多
好的
有啥面试的心得吗
可以分享给其他小伙伴的

卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历 4月15日晒offer

**面试法宝：
改简历 + 实操**

尼恩老师，新年好！
能帮忙修改下简历吗？
金三银四准备啦
可以的
java开发-6年-简历-
340.2 KB
拜托了，尼恩，希望能拿25K回来给你报喜
好，我加一下
还有吗？

尼恩，决赛圈offer了
截图是我自认能写出来的评分了，麻烦帮我参考下
选择大于努力，尼恩助我上岸
这么多offer，我看看哈
都是尼恩指点有方，本来还有个新能源汽车的，35W给拒了，主要太远了
跟着尼恩的时间太短了，目前实力也只能到这了
这里边有个大数据的，感觉也不错

卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历 5月17日晒offer

**涨薪法宝：
改简历 + 狠卷**

Java开发工程师，
docx
52.5 KB
你看着我给你改的
好的呀
谢谢大佬
麻烦大佬了
这个你自己刷
不对的，你帮自己刷
那我照着这个改一下库存系统呀
一个简历，
这么漂亮的简历，涨50%，已经没啥问题
只要准备好，不出大错，基本没问题啦

尼恩 保证押金收起来，你的offer最高涨了9k，多返现100
后面继续跟着哈，感觉卷的时间越长，
越能感觉到，
跟着大佬一起
我周围好几个年薪百万的，都是这

卷王逆袭成功案例

5年经验小伙收2个offer
最高涨薪8k，年薪42W

5月9日改简历 5月30日晒offer

秘诀：
简历指导+ 狠卷3高

以此为例
大家狠狼卷
打造最卷IT社群

卷王逆袭成功案例

非全日制 6年经验卷王
喜提3个Offer，年包30W

5月9日改简历 5月18日晒offer

面试法宝：
改简历+ 狠狼卷

卷王逆袭成功案例

寒五冻六之际卷王大逆袭
收3大offer，涨30%

5月17日改简历 5月27日晒offer

秘诀：
简历指导+ 狠卷3高

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历 5月12日晒offer

涨薪法宝：
改简历+ 狠狼卷

4年小伙喜收百度、Boss直聘
等N个顶级Offer
最高涨幅100%

6月27日改简历

9月19日晒offer

秘诀:
改简历+狠狠卷

boss直聘和小满之间, boss那
边给的offer, 工资比小满高, 但是
小满那边给的offer, 福利比boss那
边好, 所以我就选择了小满。我
不知道该怎么选, 如果你应该怎么
选呢?

还有好几个offer, 还有一个养
育的。

总体上是想着boss和小满之间
选一个

了

boss比度小满年包多7w以上。

我这涨幅都快接近百分之百了

大牛说

长那个那个思路结合上次指导的
内容改了几个点, 发现确实高大了
很多, 加上一张指标的说明确实会
让人眼前一亮的感觉。

明天睡醒时候有空了再和我看下
下, 一是看两个点我实在不知道怎么
改了, 二是在排版表达这方面确实
不行, 还得让你帮忙把关。

卷王逆袭成功案例

4年卷王入收2个offer, 涨50%

3月23日改简历

5月12日晒offer

offer决策
offer决策: n+1 电商erp, 部门成
熟, 人数多, 加班少
offer决策: n+2 做业务中心 内部系
统, 新部门, 人数多, 加班多

又搞到个offer

能搞定两个offer, 不尴尬

现在很多小伙伴面试机会都没有

涨了没呀

地点在哪里

深圳

涨50%的样子

忘记你工作几年啦, 大概工作几年
呢?

这个不用加嘛

都是一样, 加粗了反而没有效果

涨薪法宝:
改简历+狠狠卷

小伙大三暑期很焦虑
跟着尼恩卷一年
校招斩获顶级央企Offer

去年5月19日加入VIP群

今年7月5日晒offer

秘诀:
狠狠卷书+视频

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个
国奖

不错不错, 这是央企

放空

这太牛啦

跟着你卷了一年 大学顺便拿了几个
国奖

其实拿到手的也就一个a美国一

邀请你加入社群

“尼恩老师”邀请你加入社群
疯狂创客圈VIP, 进入后可查看
详情。

2021年5月19日 晚上20:04

尼恩老师 大三的暑期实习找不到
现在准备秋招应该没关系吧

看身边 总是很焦虑 自己算法这一
块卡住了

网盘里边有算法书籍

去领一本吧

秋招来得及

谢谢大佬 不过经过几个月练习 看
你写的书比之前轻松多了

趁着还是学生这段时间 慢慢把知识吃透

嘿嘿, 我的书, 比较深

期待大佬的下一本书, 已经迫不及待去学习了

小伙高中学历
薪酬涨120%

5月6日改简历

7月22日晒offer

你这块估计要涨你668

¥668.00

哈哈, 不用了大佬, 你帮我涨了
就行 光今天晚上辅导就值几千了

老板很感谢

还有很多其他的模块

面试官要问

就是其他人做的

嘿嘿, 好

之前你的工资是多少的

翻了一番

我涨你多少涨上来的

不知道, 原价给老板就行了

就是这工资

咱们得说话算话呀

老板拿着您的那个简历开发改高亮
点, 所向披靡。

ok

从标题到面试官讲的清清楚楚

后面继续狠狠卷哈

秘诀:
改简历+狠狠卷

5年卷王喜收2大Offer 最高涨5K

5月19日改简历 **9月13日晒offer**

秘诀:
改简历 + 狠跟单

5月19日改简历:

17:32 对方: 好的，稍等哈

17:32 对方: 简历请简历 202205.docx (48.0 KB)

17:32 对方: 辛苦感谢了

17:32 对方: 目前薪资是 18K，目标想冲击到 24K

17:32 对方: 但现在在外单大环境感觉有点差，不知道能不能冲

17:32 对方: OK，我晚点看看回复哈

17:32 对方: 好的 谢谢

9月13日晒offer:

17:32 对方: OK，简历还需要好好改改哈

17:32 对方: 我主要介绍思路，方法

17:32 对方: 嗯嗯，我先消化一下。有问题请在请教

17:32 对方: OK

17:32 对方: 哥哥，我就搞成功了，目前手上有两家相对心仪的公司，您看，有空您来参考下呗

17:32 对方: 恭喜恭喜

17:32 对方: 一家是 18K，主做 18K 云相关的，给了 22K，一家是自研，做区块链的，给 22K

17:32 对方: 自研的是啥公司呢

17:32 对方: 薪资涨幅都差不多，涨幅都不高，都没达到你涨幅标准

17:32 对方: 感谢

公司	部门	岗位	薪资结构	备注
1. 某公司	能源数字化产品部	java后端开发	18.5k+14.5k+5k+200/月+500股票	<30w
2. 某公司	交易研发部	16k+14k		22.4w
3. 某公司	待定	java游戏开发	15k+15k+餐补+1000/月	22.5w
4. 某公司		全栈	11k+13k	14.3w
5. 某公司			14k	16.8w
6. 某公司			9k	14.3w
7. 某公司			9k+双休+饭钱交通	13.6w
8. 某公司			10k+双休+餐补+年终奖	16.8w
9. 某公司			10k+双休+餐补+年终奖	16.8w

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>